

# **AD24N 用户手册**

**珠海市杰理科技股份有限公司**

**Zhuhai Jieli Technologyco.,LTD**

**版权所有，未经许可，禁止外传**

**2025年5月**

# 目录

|                                                                                    |    |
|------------------------------------------------------------------------------------|----|
| 目录                                                                                 | 2  |
| 1 序言                                                                               | 9  |
| 2 总体介绍                                                                             | 10 |
| 2.1 概述                                                                             | 10 |
| 2.2 总体框图                                                                           | 11 |
| 3 IO_CONTROL                                                                       | 12 |
| 3.1 概述                                                                             | 12 |
| 3.2 数字模块控制寄存器                                                                      | 12 |
| 3.2.1 PORTX_IN: portx input control register:                                      | 12 |
| 3.2.2 PORTX_OUT: portx output control register:                                    | 12 |
| 3.2.3 PORTX_DIR: portx direction control register:                                 | 12 |
| 3.2.4 PORTX_DIE: portx input enable control register:                              | 12 |
| 3.2.5 PORTX_DIEH: portx input enable p33 control register:                         | 13 |
| 3.2.6 PORTX_PU0: portx pull up0 control register:                                  | 13 |
| 3.2.7 PORTX_PU1: portx pull up1 control register:                                  | 13 |
| 3.2.8 PORTX_PD0: portx pull down0 control register:                                | 13 |
| 3.2.9 PORTX_PD1: portx pull down1 control register:                                | 14 |
| 3.2.10 PORTX_HD0: portx high driver0 control register:                             | 14 |
| 3.2.11 PORTX_HD1: portx high driver1 control register:                             | 14 |
| 3.2.12 PORTX_SPL: portx pull-down resistor enable in output control register:      | 14 |
| 3.2.13 PORTX_OUT_BSR: portx bit set resistor in output control register:           | 14 |
| 3.2.14 PORTX_DIR_BSR: portx bit set resistor in dir control register:              | 15 |
| 3.2.15 PORTX_DIE_BSR: portx bit set resistor in die control register:              | 15 |
| 3.2.16 PORTX_DIEH_BSR: portx bit set resistor in dieh control register:            | 15 |
| 3.2.17 PORTX_PU0_BSR: portx bit set resistor in pu0 control register:              | 15 |
| 3.2.18 PORTX_PU1_BSR: portx bit set resistor in pu1 control register:              | 16 |
| 3.2.19 PORTX_PD0_BSR: portx bit set resistor in pd0 control register:              | 16 |
| 3.2.20 PORTX_PD1_BSR: portx bit set resistor in pd1 control register:              | 16 |
| 3.2.21 PORTX_HD0_BSR: portx bit set resistor in hd0 control register:              | 16 |
| 3.2.22 PORTX_HD1_BSR: portx bit set resistor in hd0 control register:              | 17 |
| 3.2.23 PORTX_SPL_BSR: portx bit set resistor in spl control register:              | 17 |
| 3.2.24 PORTX_CON_BSR: portx bit set resistor in con control register(USB_IO only): | 17 |

|                                                                |           |
|----------------------------------------------------------------|-----------|
| 3.3 USB_IO CONTROL                                             | 17        |
| 3.4 IO模拟状态设置                                                   | 19        |
| <b>4 IO_WAKEUP</b>                                             | <b>20</b> |
| 4.1 概述                                                         | 20        |
| 4.2 Wakeup唤醒源                                                  | 20        |
| 4.3 寄存器                                                        | 21        |
| 4.3.1 WKUP_CON0: wakeup enable control register                | 21        |
| 4.3.2 WKUP_CON1: wakeup edge control register                  | 21        |
| 4.3.3 WKUP_CON2: wakeup pending clear control                  | 21        |
| 4.3.4 WKUP_CON3: wakeup pending register                       | 22        |
| <b>5 IOMC</b>                                                  | <b>23</b> |
| 5.1 概述                                                         | 23        |
| 5.2 JL_IOMC控制寄存器                                               | 23        |
| 5.2.1 SPI_IOMC0: spi iomc control register 0                   | 23        |
| 5.2.2 SDC_IOMC0: sdc iomc control register 0                   | 23        |
| 5.2.3 sys iomc control register 0                              | 24        |
| 5.2.4 sys iomc control register 1                              | 25        |
| 5.2.5 ASS_IOMC0: audio iomc control register 0                 | 25        |
| 5.2.6 WL_IOMC0: wireless iomc control register 0               | 26        |
| 5.2.7 ICH_IOMC0: input channel control register0               | 26        |
| 5.2.8 ICH_IOMC1: input channel control register1               | 27        |
| 5.2.9 ICH_IOMC2: input channel control register2               | 27        |
| 5.2.10 ICH_IOMC3: input channel control register3              | 28        |
| 5.2.11 ICH_IOMC4: input channel control register4              | 28        |
| 5.2.12 ICH_IOMC5: input channel control register5              | 28        |
| 5.2.13 OCH_IOMC0: output channel control register0             | 29        |
| 5.2.14 OCH_IOMC1: output channel control register1             | 29        |
| 5.2.15 OCH_IOMC2/3/4/5: output channel control register2/3/4/5 | 30        |
| 5.2.16 FSPG_CON: FSPG control register                         | 30        |
| <b>6 PORT_Crossbar</b>                                         | <b>32</b> |
| 6.1 Crossbar功能结构图                                              | 32        |
| 6.2 寄存器功能                                                      | 32        |
| 6.2.1 JL_OMAP-> POUT                                           | 32        |
| 6.2.2 JL_IMAP-> FUN_IN                                         | 33        |
| <b>7 CLOCK_SYSTEM</b>                                          | <b>34</b> |

|                                                              |           |
|--------------------------------------------------------------|-----------|
| 7.1 原生时钟源                                                    | 34        |
| 7.2 衍生时钟源                                                    | 34        |
| 7.2.1 内部PLL 参考时钟源                                            | 34        |
| 7.2.2 SYSPLL                                                 | 34        |
| 7.3 标准时钟                                                     | 35        |
| 7.4 系统时钟                                                     | 36        |
| 7.4.1 HSB时钟源                                                 | 36        |
| 7.4.2 LSB时钟源                                                 | 37        |
| 7.5 外设时钟                                                     | 38        |
| 7.5.1 SFC 时钟源                                                | 38        |
| 7.5.2 音频时钟源                                                  | 39        |
| 7.5.3 外设时钟源                                                  | 40        |
| 7.6 数字模块控制寄存器                                                | 42        |
| 7.6.1 JL_LSBCK->SYS_CON0: lsbck system config register 0     | 42        |
| 7.6.2 LSB_SEL: lsb clock sel register                        | 42        |
| 7.6.3 JL_LSBCK->STD_CON0: standard clock register 0          | 43        |
| 7.6.4 JL_LSBCK->STD_CON1: standard clock register 1          | 43        |
| 7.6.5 JL_LSBCK->STD_CON2: standard clock register 2          | 43        |
| 7.6.6 JL_LSBCK->PRP_CON0: peripheral clock config register 0 | 44        |
| 7.6.7 JL_LSBCK->PRP_CON1: peripheral clock config register 1 | 44        |
| 7.6.8 JL_HSBCK->SYS_CON0: hsbck system config register 0     | 44        |
| 7.6.9 HSB_DIV: hsb clock div register                        | 45        |
| 7.6.10 HSB_SEL: hsb clock sel register                       | 45        |
| 7.6.11 SYSPLL_CON0: pll control register 0                   | 45        |
| 7.6.12 SYSPLL_CON1: pll control register 1                   | 46        |
| 7.6.13 SYSPLL_NR: pll config register nr                     | 47        |
| 7.6.14 SYSPLL_CON2: pll control register 2                   | 47        |
| <b>8 ADC</b>                                                 | <b>48</b> |
| 8.1 概述                                                       | 48        |
| 8.2 寄存器说明                                                    | 48        |
| 8.2.1 ADC_CON: ADC control register                          | 48        |
| 8.2.2 ADC_RES: ADC result register                           | 49        |
| <b>9 CRC</b>                                                 | <b>50</b> |
| 9.1 概述                                                       | 50        |
| 9.2 寄存器说明                                                    | 50        |

|                                                                |           |
|----------------------------------------------------------------|-----------|
| 9.2.1 REG: CRC16 校验码                                           | 50        |
| 9.2.2 FIFO: 运算数据输入                                             | 50        |
| <b>10 GPCNT</b>                                                | <b>52</b> |
| 10.1 概述                                                        | 52        |
| 10.2 寄存器说明                                                     | 52        |
| 10.2.1 JL_GPCNT->CON: GPCNT configuration register             | 52        |
| 10.2.2 JL_GPCNT->NUM: The number of GPCNT clock cycle register | 53        |
| <b>11 IIC</b>                                                  | <b>54</b> |
| 11.1 概述                                                        | 54        |
| 11.2 特殊注意点                                                     | 54        |
| 11.3 数字模块控制寄存器                                                 | 54        |
| 11.3.1 IIC_CON0: iic control register 0                        | 54        |
| 11.3.2 IIC_TASK: iic task register                             | 56        |
| 11.3.3 IIC_PND: iic pending register                           | 56        |
| 11.3.4 IIC_TXBUF: iic tx buff register                         | 58        |
| 11.3.5 IIC_RXBUF: iic tx buff register                         | 58        |
| 11.3.6 IIC_ADDR: iic device address register                   | 58        |
| 11.3.7 IIC_BAUD: iic baud clk register                         | 58        |
| 11.3.8 IIC_TSU: iic setup time register                        | 58        |
| 11.3.9 IIC_THD: iic hold time register                         | 59        |
| 11.3.10 IIC_DBG: iic debug register                            | 59        |
| 11.4 基本事务操作                                                    | 59        |
| 11.5 基本工作场景使用流程                                                | 60        |
| <b>12 MCPWM</b>                                                | <b>66</b> |
| 12.1 概述                                                        | 66        |
| 12.2 定时器 MCTIMER                                               | 66        |
| 12.3 TIMER模块寄存器                                                | 66        |
| 12.3.1 TMRx_CON: Timer contrl register                         | 66        |
| 12.3.2 TMRx_CNT: counter initial value register                | 67        |
| 12.3.3 TMR_PR: counter target value register                   | 67        |
| 12.4 模块引脚                                                      | 67        |
| 12.5 模块特性                                                      | 68        |
| 12.6 PWM模块控制寄存器                                                | 68        |
| 12.6.1 CHx_CON0: pwm control register0                         | 68        |
| 12.6.2 CHX_CON1: pwm control register1                         | 68        |

|                                                                 |           |
|-----------------------------------------------------------------|-----------|
| 12.6.3 CHX_CMPH: pwm high port compare register                 | 69        |
| 12.6.4 CHX_CMPL: pwm low port compare register                  | 69        |
| 12.6.5 FPIN_CON: input filter control register                  | 69        |
| 12.6.6 MCPWM_CON: mcpwm control register                        | 70        |
| <b>13 GPCRC</b>                                                 | <b>71</b> |
| 13.1 概述                                                         | 71        |
| 13.2 功能描述                                                       | 71        |
| 13.2.1 数据处理                                                     | 71        |
| 13.2.2 多项式配置                                                    | 71        |
| 13.3 数字模块控制寄存器                                                  | 72        |
| 13.3.1 GPCRC_CON: gpcrc control register                        | 72        |
| 13.3.2 GPCRC_POL: gpcrc poly register                           | 73        |
| 13.3.3 GPCRC_INIT: gpcrc initial register                       | 73        |
| 13.3.4 GPCRC_REG: gpcrc data register                           | 73        |
| 13.4 DEMO CODE                                                  | 73        |
| <b>14 PULSE COUNTER</b>                                         | <b>75</b> |
| 14.1 概述                                                         | 75        |
| 14.2 寄存器说明                                                      | 75        |
| 14.2.1 PL_CNT_CON                                               | 75        |
| 14.2.2 PL_CNT_VAL                                               | 75        |
| 14.3 示例代码                                                       | 76        |
| <b>15 RAND64</b>                                                | <b>77</b> |
| 15.1 概述                                                         | 77        |
| 15.2 控制寄存器                                                      | 77        |
| <b>16 SDC</b>                                                   | <b>78</b> |
| 16.1 概述                                                         | 78        |
| 16.2 数字模块控制寄存器                                                  | 78        |
| 16.2.1 JL_SD0->CON0: sd0 control register 0                     | 78        |
| 16.2.2 JL_SD0->CON1: sd0 control register 1                     | 79        |
| 16.2.3 JL_SD0->CON2: sd0 control register 2                     | 80        |
| 16.2.4 JL_SD0->CTU_CON: sd0 continue control register           | 80        |
| 16.2.5 JL_SD0->CTU_CNT: sd0 continual transfer counter register | 81        |
| 16.2.6 JL_SD0->CPTR: sd0 command pointer register               | 81        |
| 16.2.7 JL_SD0->DPTR: sd0 data pointer register                  | 82        |
| 16.3 配置流程示例                                                     | 82        |

|                                                  |           |
|--------------------------------------------------|-----------|
| 16.3.1 发送命令、接收响应流程                               | 82        |
| 16.3.2 发送数据流程（单块写CMD24）                          | 82        |
| 16.3.3 接收数据流程（单块读CMD17）                          | 82        |
| 16.3.4 发送数据流程（多块写CMD25）                          | 83        |
| 16.3.5 接收数据流程（多块写CMD18）                          | 83        |
| 16.4 DEMO CODE                                   | 83        |
| <b>17 SPI</b>                                    | <b>85</b> |
| 17.1 概述                                          | 85        |
| 17.2 特殊注意点                                       | 85        |
| 17.3 数字模块控制寄存器                                   | 86        |
| 17.3.1 SPIx_CON: SPIx control register 0         | 86        |
| 17.3.2 SPIx_BAUD: SPIx baudrate setting register | 87        |
| 17.3.3 SPIx_BUF: SPIx buffer register            | 87        |
| 17.3.4 SPIx_ADR: SPIx DMA address register       | 87        |
| 17.3.5 SPIx_CNT: SPIx DMA counter register       | 88        |
| 17.3.6 SPIx_CON1: SPIx control1 register         | 88        |
| 17.4 传输波形                                        | 88        |
| <b>18 16位GPTIMER</b>                             | <b>90</b> |
| 18.1 概述                                          | 90        |
| 18.2 控制寄存器                                       | 90        |
| 18.3 寄存器说明                                       | 90        |
| 18.3.1 Tx_CON: timer x control register          | 90        |
| 18.3.2 Tx_CNT: timer x counter register          | 91        |
| 18.3.3 Tx_PR: timer x period register            | 91        |
| 18.3.4 Tx_PWM: timer x PWM register              | 92        |
| 18.3.5 Tx_IRFLT: timer x PWM register            | 92        |
| <b>19 UART</b>                                   | <b>94</b> |
| 19.1 概述                                          | 94        |
| 19.2 特殊注意点                                       | 94        |
| 19.3 数字模块控制寄存器                                   | 94        |
| 19.3.1 TX_CON0: uart tx control register 0       | 94        |
| 19.3.2 RX_CON0: uart rx control register 0       | 94        |
| 19.3.3 TX_CON1: uart tx control register 1       | 95        |
| 19.3.4 RX_CON1: uart rx control register 1       | 96        |
| 19.3.5 UTx_CON2: uart x control register 2       | 97        |



|                                                          |            |
|----------------------------------------------------------|------------|
| 19.3.6 UTx_BAUD: uart x baudrate register                | 97         |
| 19.3.7 UTx_BUF: uart x data buffer register              | 98         |
| 19.3.8 UTx_TXADR: uart x TX DMA buffer register          | 98         |
| 19.3.9 UTx_TXCNT: uart x TX DMA counter register         | 98         |
| 19.3.10 UTx_RXCNT: uart x RX DMA counter register        | 98         |
| 19.3.11 UTx_RXSADR: uart x RX DMA start address register | 99         |
| 19.3.12 UTx_RXEADR: uart x RX DMA end address register   | 99         |
| 19.3.13 UTx_HRXCNT: uart x have RX DMA counter register  | 99         |
| 19.3.14 UTx_OTCNT: uart x Over Timer counter register    | 99         |
| 19.3.15 UTx_ERR_CNT: uart x error byte counter register  | 99         |
| <b>20 UART_LITE</b>                                      | <b>101</b> |
| 20.1 概述                                                  | 101        |
| 20.2 数字模块控制寄存器                                           | 101        |
| 20.2.1 TX_CON0: uart tx control register 0               | 101        |
| 20.2.2 RX_CON0: uart rx control register 0               | 101        |
| 20.2.3 TX_CON1: uart tx control register 1               | 102        |
| 20.2.4 RX_CON1: uart rx control register 1               | 102        |
| 20.2.5 UTx_CON2: uart x control register 2               | 102        |
| 20.2.6 UTx_BAUD: uart x baudrate register                | 103        |
| 20.2.7 UTx_BUF: uart x data buffer register              | 103        |
| 20.2.8 UTx_ERR_CNT: uart x error byte counter register   | 103        |
| <b>21 AUDIO LINK</b>                                     | <b>105</b> |
| 21.1 概述                                                  | 105        |
| 21.2 特殊注意点                                               | 105        |
| 21.3 ALNK支持的采样率配置                                        | 105        |
| 21.4 数字模块控制寄存器                                           | 106        |
| 21.4.1 ALNK_CON0: control register 0                     | 106        |
| 21.4.2 ALNK_CON1: control register 1                     | 107        |
| 21.4.3 ALNK_CON2: control register 2                     | 108        |
| 21.4.4 ALNK_CON3: control register 3                     | 108        |
| 21.4.5 ALNK_ADRX: alnk dma start address register        | 109        |
| 21.4.6 ALNK_LEN: alnk dma sample length register         | 109        |
| 21.5 数据通路                                                | 109        |
| 21.5.1 乒乓缓存                                              | 109        |



# 1 序言

AD24N系列芯片是低成本低功耗高性能通用语音MCU，本芯片主要面向智能语音玩具、智能家居设备控制以及通用型智能控制等场景。芯片集成了32位CPU自带ICACHE，内置52k byte数据空间，并可外接FLASH扩展存储空间；同时芯片内建低温票高精度时钟振荡器，无须外接晶振；集成高性能音频ADC与DAC，并自带音频功放模块可直接驱动小功率喇叭发声；PMU电源管理模块可灵活提供多种低功耗工作模式，支持各种超低功耗工作场景；此外还提供了FUSB、ADC、IIC、SPI、UART、SD、IIS等丰富的外设接口。

珠海市杰理科技股份有限公司

## 2 总体介绍

### 2.1 概述

#### SYSTEM

- 32bit Dual-Issue DSP 240MHz
- I-cache
- Support SDTAP/EMU
- On-chip SRAM 52kbyte(share cache ram 20k)
- Built-In Flash
- Internal RC oscillator,PLL

#### Audio

- 1 x 16bit DAC  
SNR 96dB  
Noise 9uVrms  
Sampling rate 8~96kHz
- 1 x 16bit Class-D Speaker Driver  
SNR 97dB  
Sampling rate 8~96kHz  
Drive speaker directly 500mW@4Ω
- 1 x 16bit ADC  
SNR 97dB  
Sampling rate 8~48kHz  
Support Speaker for microphone
- I<sup>2</sup>S AUDIO Master/Slave interface

#### Peripherals

- 1 x Full speed USB
- 1 x SD host controller
- 3 x Multi-function 16bit timer
- 2 x UART interface
- 1 x I<sup>2</sup>C Master/Slave interface
- 3 x SPI Master/Slave interface
- 4 x MCPWM
- 1 x GPCRC
- 1 x 10bit ADC(16 Channels)
- 22 x GPIO Support function remapping

#### PMU

- VPWR range 1.8V to 5.5V
- IOVDD range 2.7V to 3.6V

#### Packages

- QSOP24

#### Temperature

- Operating temperature  
TC = -20°C to +85°C(standard range)

TC = -40°C to +105°C(extended range)

- Storage temperature -65°C to +150°C

### Applications

- Sound Toy
- Audio player

## 2.2 总体框图

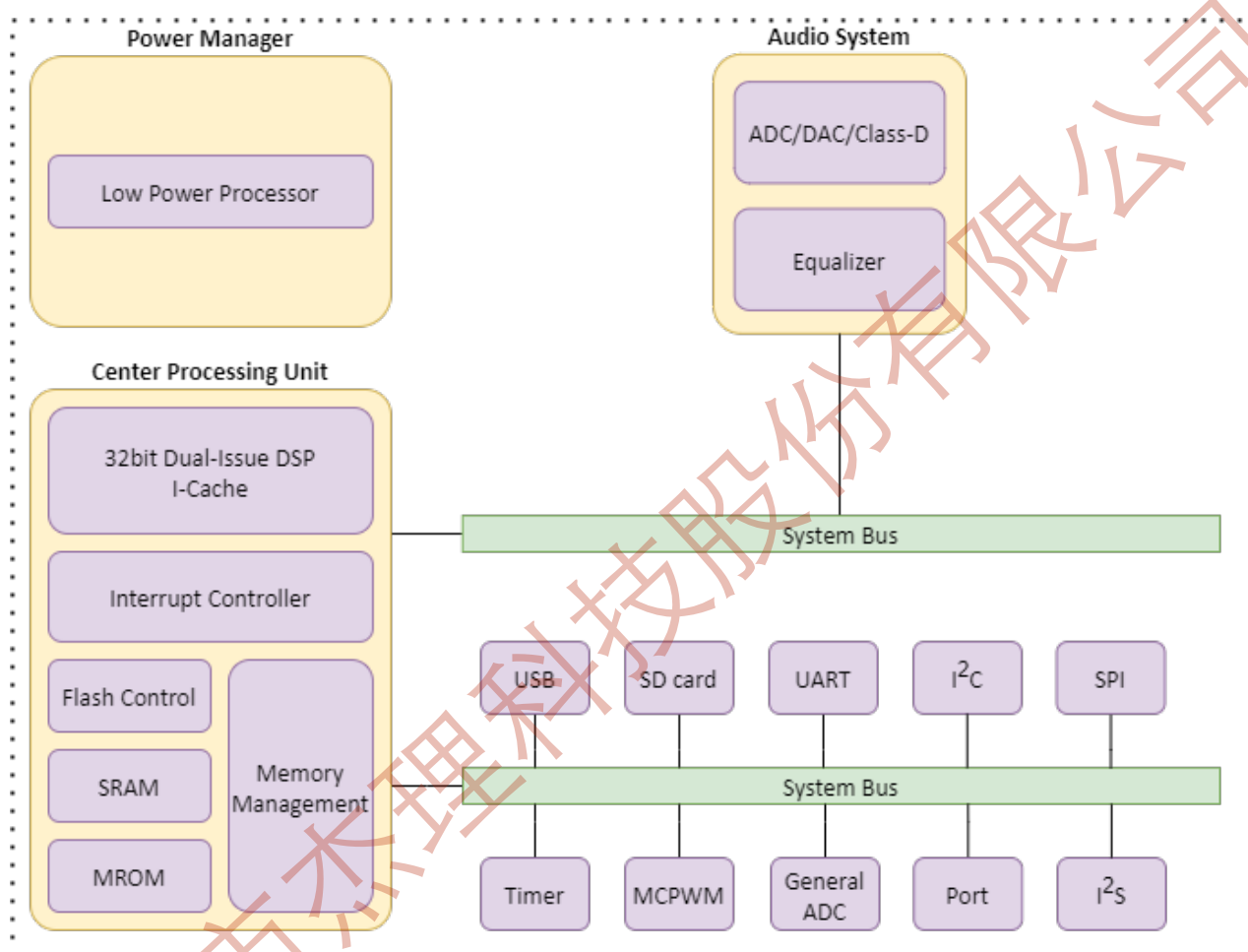


图1 AD24N总体框图

## 3 IO\_CONTROL

### 3.1 概述

IO状态控制寄存器

### 3.2 数字模块控制寄存器

#### 3.2.1 PORTX\_IN: portx input control register:

| Bit   | Name     | RW | Default | RV | Description                                                           |
|-------|----------|----|---------|----|-----------------------------------------------------------------------|
| 31-16 | RESERVED | -  | 0x0     | -  | 预留                                                                    |
| 15-0  | PORTX_IN | r  | -       | -  | 获取portx组的输入值，只有在对应的GPIO设置为输入使能（DIE）<br>有效时数值才正确<br>0: 输入为0<br>1: 输入为1 |

#### 3.2.2 PORTX\_OUT: portx output control register:

| Bit   | Name      | RW | Default | RV | Description                                                             |
|-------|-----------|----|---------|----|-------------------------------------------------------------------------|
| 31-16 | RESERVED  | -  | 0x0     | -  | 预留                                                                      |
| 15-0  | PORTX_OUT | RW | 0x0     | -  | 控制portx组的输出值，只有在对应的GPIO设置为输出方向（DIR）<br>有效时输出数值才正确<br>0: 输出为0<br>1: 输出为1 |

#### 3.2.3 PORTX\_DIR: portx direction control register:

| Bit   | Name      | RW | Default | RV | Description                                 |
|-------|-----------|----|---------|----|---------------------------------------------|
| 31-16 | RESERVED  | -  | 0x0     | -  | 预留                                          |
| 15-0  | PORTX_DIR | RW | 0x0     | -  | 控制portx组的输入输出方向<br>0: 设置为输出方向<br>1: 设置为输入方向 |

#### 3.2.4 PORTX\_DIE: portx input enable control register:

| Bit | Name | RW | Default | RV | Description |
|-----|------|----|---------|----|-------------|
|-----|------|----|---------|----|-------------|

| Bit   | Name      | RW | Default | RV | Description                             |
|-------|-----------|----|---------|----|-----------------------------------------|
| 31-16 | RESERVED  | -  | 0x0     | -  | 预留                                      |
| 15-0  | PORTX_DIE | RW | 0x0     | -  | 控制portx组的输入使能<br>0: 禁止输入使能<br>1: 允许输入使能 |

### 3.2.5 PORTX\_DIEH: portx input enable p33 control register:

| Bit   | Name       | RW | Default | RV | Description                                |
|-------|------------|----|---------|----|--------------------------------------------|
| 31-16 | RESERVED   | -  | 0x0     | -  | 预留                                         |
| 15-0  | PORTX_DIEH | RW | 0x0     | -  | 控制portx组的P33输入使能<br>0: 禁止输入使能<br>1: 允许输入使能 |

### 3.2.6 PORTX\_PU0: portx pull up0 control register:

| Bit   | Name      | RW | Default | RV | Description                     |
|-------|-----------|----|---------|----|---------------------------------|
| 31-16 | RESERVED  | -  | 0x0     | -  | 预留                              |
| 15-0  | PORTX_PU0 | RW | 0x0     | -  | 控制portx组的上拉电阻值，与PORTX_PU1组成4个档位 |

### 3.2.7 PORTX\_PU1: portx pull up1 control register:

| Bit   | Name      | RW | Default | RV | Description                                                                                                        |
|-------|-----------|----|---------|----|--------------------------------------------------------------------------------------------------------------------|
| 31-16 | RESERVED  | -  | 0x0     | -  | 预留                                                                                                                 |
| 15-0  | PORTX_PU0 | RW | 0x0     | -  | 控制portx组的上拉电阻值，与PORTX_PU0组成4个档位 {pu1,pu0}<br>00: 无上拉电阻<br>01: 10KΩ<br>10: 100KΩ<br>11: 1MΩ (mclr上拉电阻不受该位控制，由p33控制) |

### 3.2.8 PORTX\_PD0: portx pull down0 control register:

| Bit   | Name     | RW | Default | RV | Description |
|-------|----------|----|---------|----|-------------|
| 31-16 | RESERVED | -  | 0x0     | -  | 预留          |

| Bit  | Name      | RW | Default | RV | Description                     |
|------|-----------|----|---------|----|---------------------------------|
| 15-0 | PORTX_PD0 | RW | 0x0     | -  | 控制portx组的下拉电阻值，与PORTX_PD1组成4个档位 |

### 3.2.9 PORTX\_PD1: portx pull down1 control register:

| Bit   | Name      | RW | Default | RV | Description                                                                                |
|-------|-----------|----|---------|----|--------------------------------------------------------------------------------------------|
| 31-16 | RESERVED  | -  | 0x0     | -  | 预留                                                                                         |
| 15-0  | PORTX_PD1 | RW | 0x0     | -  | 控制portx组的下拉电阻值，与PORTX_PD0组成4个档位 {pd1,pd0}<br>00: 无下拉电阻<br>01: 10KΩ<br>10: 100KΩ<br>11: 1MΩ |

### 3.2.10 PORTX\_HD0: portx high driver0 control register:

| Bit   | Name      | RW | Default | RV | Description                       |
|-------|-----------|----|---------|----|-----------------------------------|
| 31-16 | RESERVED  | -  | 0x0     | -  | 预留                                |
| 15-0  | PORTX_HD0 | RW | 0x0     | -  | 控制portx组的强驱动电流档位，与PORTX_HD1组成4个档位 |

### 3.2.11 PORTX\_HD1: portx high driver1 control register:

| Bit   | Name      | RW | Default | RV | Description                                 |
|-------|-----------|----|---------|----|---------------------------------------------|
| 31-16 | RESERVED  | -  | 0x0     | -  | 预留                                          |
| 15-0  | PORTX_HD1 | RW | 0x0     | -  | 控制portx组的强驱动电流档位，与PORTX_HD0组成4个档位 {hd1,hd0} |

### 3.2.12 PORTX\_SPL: portx pull-down resistor enable in output control register:

| Bit  | Name     | RW | Default | RV | Description                                                                             |
|------|----------|----|---------|----|-----------------------------------------------------------------------------------------|
| 31-1 | RESERVED | -  | 0x0     | -  | 预留                                                                                      |
| 0    | SPL      | RW | 0x0     | -  | 0: 在IO为输出状态时，内部上下拉电阻无效，读取IO输入寄存器值总为0;<br>1: 在IO为输出状态时，内部上下拉电阻功能有效，读取IO输入寄存器值跟随IO实际电平逻辑; |

### 3.2.13 PORTX\_OUT\_BSR: portx bit set resistor in output control register:

| Bit   | Name    | RW | Default | RV | Description                                                     |
|-------|---------|----|---------|----|-----------------------------------------------------------------|
| 31-16 | OUT_BSR | W  | 0x0     | -  | 设置portx组的输出0，只有在对应的GPIO设置为输出方向（DIR）有效时输出数值才正确<br>0: 无效<br>1: 有效 |
| 15-0  | OUT_BSR | W  | 0x0     | -  | 设置portx组的输出1，只有在对应的GPIO设置为输出方向（DIR）有效时输出数值才正确<br>0: 无效<br>1: 有效 |

### 3.2.14 PORTX\_DIR\_BSR: portx bit set resistor in dir control register:

| Bit   | Name    | RW | Default | RV | Description                            |
|-------|---------|----|---------|----|----------------------------------------|
| 31-16 | DIR_BSR | W  | 0x0     | -  | 设置portx组对应GPIO的方向为输出<br>0: 无效<br>1: 有效 |
| 15-0  | DIR_BSR | W  | 0x0     | -  | 设置portx组对应GPIO的方向为输入<br>0: 无效<br>1: 有效 |

### 3.2.15 PORTX\_DIE\_BSR: portx bit set resistor in die control register:

| Bit   | Name    | RW | Default | RV | Description                            |
|-------|---------|----|---------|----|----------------------------------------|
| 31-16 | DIE_BSR | W  | 0x0     | -  | 设置portx组对应GPIO的die为1<br>0: 无效<br>1: 有效 |
| 15-0  | DIE_BSR | W  | 0x0     | -  | 设置portx组对应GPIO的die为0<br>0: 无效<br>1: 有效 |

### 3.2.16 PORTX\_DIEH\_BSR: portx bit set resistor in dieh control register:

| Bit   | Name     | RW | Default | RV | Description                             |
|-------|----------|----|---------|----|-----------------------------------------|
| 31-16 | DIEH_BSR | W  | 0x0     | -  | 设置portx组对应GPIO的dieh为1<br>0: 无效<br>1: 有效 |
| 15-0  | DIEH_BSR | W  | 0x0     | -  | 设置portx组对应GPIO的dieh为0<br>0: 无效<br>1: 有效 |

### 3.2.17 PORTX\_PU0\_BSR: portx bit set resistor in pu0 control register:

| Bit | Name | RW | Default | RV | Description |
|-----|------|----|---------|----|-------------|
|-----|------|----|---------|----|-------------|



| Bit   | Name    | RW | Default | RV | Description                            |
|-------|---------|----|---------|----|----------------------------------------|
| 31-16 | PU0_BSR | W  | 0x0     | -  | 设置portx组对应GPIO的pu0为1<br>0: 无效<br>1: 有效 |
| 15-0  | PU0_BSR | W  | 0x0     | -  | 设置portx组对应GPIO的pu0为0<br>0: 无效<br>1: 有效 |

### 3.2.18 PORTX\_PU1\_BSR: portx bit set resistor in pu1 control register:

| Bit   | Name    | RW | Default | RV | Description                            |
|-------|---------|----|---------|----|----------------------------------------|
| 31-16 | PU1_BSR | W  | 0x0     | -  | 设置portx组对应GPIO的pu1为1<br>0: 无效<br>1: 有效 |
| 15-0  | PU1_BSR | W  | 0x0     | -  | 设置portx组对应GPIO的pu1为0<br>0: 无效<br>1: 有效 |

### 3.2.19 PORTX\_PD0\_BSR: portx bit set resistor in pd0 control register:

| Bit   | Name    | RW | Default | RV | Description                            |
|-------|---------|----|---------|----|----------------------------------------|
| 31-16 | PD0_BSR | W  | 0x0     | -  | 设置portx组对应GPIO的pd0为1<br>0: 无效<br>1: 有效 |
| 15-0  | PD0_BSR | W  | 0x0     | -  | 设置portx组对应GPIO的pd0为0<br>0: 无效<br>1: 有效 |

### 3.2.20 PORTX\_PD1\_BSR: portx bit set resistor in pd1 control register:

| Bit   | Name    | RW | Default | RV | Description                            |
|-------|---------|----|---------|----|----------------------------------------|
| 31-16 | PD0_BSR | W  | 0x0     | -  | 设置portx组对应GPIO的pd0为1<br>0: 无效<br>1: 有效 |
| 15-0  | PD0_BSR | W  | 0x0     | -  | 设置portx组对应GPIO的pd0为0<br>0: 无效<br>1: 有效 |

### 3.2.21 PORTX\_HD0\_BSR: portx bit set resistor in hd0 control register:

| Bit | Name | RW | Default | RV | Description |
|-----|------|----|---------|----|-------------|
|-----|------|----|---------|----|-------------|

| Bit   | Name    | RW | Default | RV | Description                            |
|-------|---------|----|---------|----|----------------------------------------|
| 31-16 | HD0_BSR | W  | 0x0     | -  | 设置portx组对应GPIO的hd0为1<br>0: 无效<br>1: 有效 |
| 15-0  | HD0_BSR | W  | 0x0     | -  | 设置portx组对应GPIO的hd0为0<br>0: 无效<br>1: 有效 |

### 3.2.22 PORTX\_HD1\_BSR: portx bit set resistor in hd0 control register:

| Bit   | Name    | RW | Default | RV | Description                            |
|-------|---------|----|---------|----|----------------------------------------|
| 31-16 | HD1_BSR | W  | 0x0     | -  | 设置portx组对应GPIO的hd1为1<br>0: 无效<br>1: 有效 |
| 15-0  | HD1_BSR | W  | 0x0     | -  | 设置portx组对应GPIO的hd1为0<br>0: 无效<br>1: 有效 |

### 3.2.23 PORTX\_SPL\_BSR: portx bit set resistor in spl control register:

| Bit   | Name    | RW | Default | RV | Description                            |
|-------|---------|----|---------|----|----------------------------------------|
| 31-16 | SPL_BSR | W  | 0x0     | -  | 设置portx组对应GPIO的spl为1<br>0: 无效<br>1: 有效 |
| 15-0  | SPL_BSR | W  | 0x0     | -  | 设置portx组对应GPIO的spl为0<br>0: 无效<br>1: 有效 |

### 3.2.24 PORTX\_CON\_BSR: portx bit set resistor in con control register(USB\_IO only):

| Bit   | Name    | RW | Default | RV | Description                            |
|-------|---------|----|---------|----|----------------------------------------|
| 31-16 | SPL_CON | W  | 0x0     | -  | 设置USB组IO的CON对应的BIT为1<br>0: 无效<br>1: 有效 |
| 15-0  | SPL_CON | W  | 0x0     | -  | 设置USB组IO的CON对应的BIT为0<br>0: 无效<br>1: 有效 |

## 3.3 USB\_IO CONTROL

### 1. JL\_PORTUSB->DIE:

2. JL\_PORTUSB->DIEH:
3. JL\_PORTUSB->DIR:
4. JL\_PORTUSB->PU0: DP上拉1.5K, DM上拉180K(PU1没有用到)
5. JL\_PORTUSB->PD0: DP 下拉15K, DM下拉15K(PD1没有用到)
6. JL\_PORTUSB->IN:
7. JL\_PORTUSB->OUT:
8. JL\_PORTUSB->SPL:  
上述USB IO寄存器跟普通IO寄存器功能一样, 不过USB 只有两个IO,BIT(0)设置DP, BIT(1)设置DM
9. JL\_PORTUSB->CON: usb io control register 0

| Bit   | Name        | RW  | Default | RV | Description                                                      |
|-------|-------------|-----|---------|----|------------------------------------------------------------------|
| 31-13 | RESERVED    | -   | -       | -  | 预留                                                               |
| 12-11 | DBG_SEL     | rw  | 0x0     | -  | 测试信号选择:<br>00: 1'b0 0<br>1: usb_diff<br>10: usb_dp<br>11: usb_dm |
| 10    | USB_CP_MODE | r'w | -       | -  | 测试信号使能                                                           |
| 9     | USB_DF_MODE | rw  | -       | -  | 测试信号使能                                                           |
| 8     | IO_MODE     | rw  | 0x1     | -  | IO模式使能<br>0: USB模式<br>1: 普通IO模式                                  |
| 7     | CHKDPO      | r   | 0x1     | -  | CHKDPO经系统时钟采样后的输入                                                |
| 6     | DIDF        | r   | 0       | -  | 差分输入DIDF经lsb_clk采样后的输入                                           |
| 5     | HDEN        | -   | -       | -  | 驱动能力选择<br>0:low drive mode<br>1:high drive mode                  |
| 4     | PDCHKDP     | rw  | 0       | -  | DP 外接下拉检查使能<br>0: disable<br>1: enable                           |
| 3     | DM_ADCEN    | -   | -       | -  | dm to adc en                                                     |
| 2     | SR0         | rw  | 0x0     | -  | 输出驱动能力选择                                                         |
| 1     | DP_ADCEN    | rw  | 0       | -  | dp to adc en                                                     |
| 0     | RCVEN       | rw  | 0       | -  | USB_PHY使能 (差分输入使能, 使用USB功能时需打开这一BIT)                             |

## 3.4 IO模拟状态设置

当IO用于模拟功能时，需要正确设置IO寄存器，确保模拟功能不受IO影响。正确配置如下：

如将PA0设置为模拟功能。

```
JL_PORTA->DIR |= BIT(0);  
JL_PORTA->DIE &= ~BIT(0);  
JL_PORTA->DIEH &= ~BIT(0);  
JL_PORTA->PU0 &= ~BIT(0);  
JL_PORTA->PU1 &= ~BIT(0);  
JL_PORTA->PD0 &= ~BIT(0);  
JL_PORTA->PD1 &= ~BIT(0);
```

珠海市杰理科技股份有限公司

## 4 IO\_WAKEUP

### 4.1 概述

IO Wakeup是一个异步事件唤醒模块，它可以捕获IO 的上升沿/下降沿，将处于standby/sleep状态下的系统唤醒，进入正常工作模式。当系统处于正常模式时，Wakeup唤醒源可作为中断触发源，最多可有32个唤醒来源(具体数目以下述Wakeup唤醒源为准)；

### 4.2 Wakeup唤醒源

- 事件0: PA0

事件1: PA1

事件2: PA2

事件3: PA3

事件4: PA4

事件5: PA5

事件6: PA6

事件7: PA7

事件8: PA8

事件9: PA9

事件10: PA10

事件11: PA11

事件12: PA12

事件13: PA13

事件14: PA14

事件15: PA15

事件16: PB0

事件17: PB1

事件18: PB2

事件19: USBDP

事件20: USBDM

事件21: 无

事件22: FSPG(PP1)

事件23: GP\_ICH0

事件24: GP\_ICH1

事件25: GP\_ICH2

事件26: GP\_ICH3

事件27: 无

事件28: 无

事件29: 无

事件30: 无

事件31: 无

唤醒源数量: 32

## 4.3 寄存器

### 4.3.1 WKUP\_CON0: wakeup enable control register

偏移地址: 0x00

| Bit  | Name     | RW | Default | Description                       |
|------|----------|----|---------|-----------------------------------|
| 31-n | RESERVED | -  | -       | -                                 |
| n-0  | WKUP_EN  | rw | 0x0     | 对应唤醒源使能<br>0: 关闭唤醒功能<br>1: 打开唤醒功能 |

n 为唤醒源数量

### 4.3.2 WKUP\_CON1: wakeup edge control register

偏移地址: 0x04

| Bit  | Name      | RW | Default | Description                        |
|------|-----------|----|---------|------------------------------------|
| 31-n | RESERVED  | -  | -       | -                                  |
| n-0  | WKUP_EDGE | rw | 0x0     | 对应唤醒源边沿选择<br>0: 上升沿唤醒;<br>1: 下降沿唤醒 |

n 为唤醒源数量

### 4.3.3 WKUP\_CON2: wakeup pending clear control

偏移地址: 0x08

| Bit | Name | RW | Default | Description |
|-----|------|----|---------|-------------|
|-----|------|----|---------|-------------|

| Bit  | Name      | RW | Default | Description       |
|------|-----------|----|---------|-------------------|
| 31-n | RESERVED  | -  | -       | -                 |
| n-0  | WKUP_CPND | w  | 0x0     | 写1清除对应唤醒源 pending |

n 为唤醒源数量

#### 4.3.4 WKUP\_CON3: wakeup pending register

偏移地址: 0x0c

| Bit  | Name     | RW | Default | Description   |
|------|----------|----|---------|---------------|
| 31-n | RESERVED | -  | -       | -             |
| n-0  | WKUP_PND | r  | 0x0     | 对应唤醒源 pending |

n 为唤醒源数量

## 5 IOMC

### 5.1 概述

IOMC (IO remapping control) 控制寄存器主要用于控制IO除crossbar之外的一些拓展功能的配置，每个外设类型都具有相应的独立IOMC控制寄存器；

项目存在两个基地址不同的IOMC，分别为JL\_IOMC和JL\_SFC\_IOMC，前者用于处理通用外设的拓展功能，后者只用于SFC/spi0处于flash相关的功能。

### 5.2 JL\_IOMC控制寄存器

#### 5.2.1 SPI\_IOMC0: spi iomc control register 0

偏移地址: 0x14

| Bit   | Name         | RW | Default | Description                                    |
|-------|--------------|----|---------|------------------------------------------------|
| 31    | Reserved     | -  | -       | Reserved                                       |
| 30-27 | Reserved     | -  | -       | Reserved                                       |
| 26-24 | Reserved     | -  | -       | Reserved                                       |
| 23    | SPI2_DUPLEX  | rw | 0       | SPI2从机输入时钟选择<br>0: spi2_clki;<br>1: spi1_clkco |
| 22-20 | SPI2_ICK_SEL | rw | 0       | SPI2输入时钟延迟级数选择                                 |
| 18-16 | Reserved     | -  | -       | Reserved                                       |
| 15-13 | Reserved     | -  | -       | Reserved                                       |
| 12    | Reserved     | -  | -       | Reserved                                       |
| 11-8  | Reserved     | -  | -       | Reserved                                       |
| 7     | Reserved     | -  | -       | Reserved                                       |
| 6-3   | Reserved     | -  | -       | Reserved                                       |
| 2-0   | SPI0_IOS     | rw | 0       | SPI0 固定IO 组选择                                  |

#### 5.2.2 SDC\_IOMC0: sdc iomc control register 0

偏移地址: 0x18



| Bit   | Name     | RW | Default | Description |
|-------|----------|----|---------|-------------|
| 31    | Reserved | -  | -       | Reserved    |
| 30-27 | Reserved | -  | -       | Reserved    |
| 26-24 | Reserved | -  | -       | Reserved    |
| 23    | Reserved | -  | -       | Reserved    |
| 22-19 | Reserved | -  | -       | Reserved    |
| 18-16 | Reserved | -  | -       | Reserved    |
| 15    | Reserved | -  | -       | Reserved    |
| 14-10 | Reserved | -  | -       | Reserved    |
| 11    | Reserved | -  | -       | Reserved    |
| 10-8  | Reserved | -  | -       | Reserved    |
| 7-2   | Reserved | -  | -       | Reserved    |
| 3     | Reserved | -  | -       | Reserved    |
| 2-0   | Reserved | -  | -       | Reserved    |

### 5.2.3 sys iomc control register 0

偏移地址: 0x1c

| Bit   | Name     | RW | Default | Description |
|-------|----------|----|---------|-------------|
| 31-12 | Reserved | -  | -       | Reserved    |
| 11    | Reserved | -  | -       | Reserved    |
| 10    | Reserved | -  | -       | Reserved    |
| 9     | Reserved | -  | -       | Reserved    |
| 8     | Reserved | -  | -       | Reserved    |
| 7     | Reserved | -  | -       | Reserved    |
| 6     | Reserved | -  | -       | Reserved    |
| 5     | Reserved | -  | -       | Reserved    |
| 4     | Reserved | -  | -       | Reserved    |
| 3     | Reserved | -  | -       | Reserved    |

| Bit | Name     | RW | Default | Description |
|-----|----------|----|---------|-------------|
| 2   | Reserved | -  | -       | Reserved    |
| 1-0 | Reserved | -  | -       | Reserved    |

## 5.2.4 sys iomc control register 1

偏移地址: 0x20

| Bit   | Name     | RW | Default | Description   |
|-------|----------|----|---------|---------------|
| 31    | Reserved | -  | -       | Reserved      |
| 30-27 | Reserved | -  | -       | Reserved      |
| 26    | Reserved | -  | -       | Reserved      |
| 25-24 | Reserved | -  | -       | Reserved      |
| 23-1  | Reserved | -  | -       | Reserved      |
| 0     | SFC_IOS  | rw | 0       | SFC 固定IO 组别选择 |

## 5.2.5 ASS\_IOMC0: audio iomc control register 0

偏移地址: 0x38

| Bit   | Name          | RW | Default | Description |
|-------|---------------|----|---------|-------------|
| 31    | Reserved      | -  | -       | Reserved    |
| 31-26 | ASS_OCH1_sSEL | rw | 0       |             |
| 25    | ASS_OCH1_REG  | rw | 0       |             |
| 24    | ASS_OCH1_EN   | rw | 0       |             |
| 23-18 | ASS_OCH0_SEL  | rw | 0       |             |
| 17    | ASS_OCH0_REG  | rw | 0       |             |
| 16    | ASS_OCH0_EN   | rw | 0       |             |
| 15    | Reserved      | -  | -       | Reserved    |
| 14-11 | Reserved      | -  | -       | Reserved    |
| 10-8  | Reserved      | -  | -       | Reserved    |
| 7     | Reserved      | -  | -       | Reserved    |

| Bit | Name     | RW | Default | Description |
|-----|----------|----|---------|-------------|
| 6-3 | Reserved | -  | -       | Reserved    |
| 2-0 | Reserved | -  | -       | Reserved    |

### 5.2.6 WL\_IOMC0: wireless iomc control register 0

偏移地址: 0x3c

| Bit   | Name     | RW | Default | Description |
|-------|----------|----|---------|-------------|
| 31    | Reserved | -  | -       | Reserved    |
| 30-27 | Reserved | -  | -       | Reserved    |
| 26-24 | Reserved | -  | -       | Reserved    |
| 23    | Reserved | -  | -       | Reserved    |
| 22-19 | Reserved | -  | -       | Reserved    |
| 18-16 | Reserved | -  | -       | Reserved    |
| 15    | Reserved | -  | -       | Reserved    |
| 14-11 | Reserved | -  | -       | Reserved    |
| 10-8  | Reserved | -  | -       | Reserved    |
| 7-6   | Reserved | -  | -       | Reserved    |
| 5     | Reserved | -  | -       | Reserved    |
| 4     | Reserved | -  | -       | Reserved    |
| 3     | Reserved | -  | -       | Reserved    |
| 2-0   | Reserved | -  | -       | Reserved    |

### 5.2.7 ICH\_IOMC0: input channel control register0

偏移地址: 0x40

| Bit   | Name                | RW | Default | Description |
|-------|---------------------|----|---------|-------------|
| 31-30 | Reserved            | -  | -       |             |
| 29-25 | ICH5_SEL (TMR2_CAP) | rw | 0       | 同ICH0选项     |

| Bit   | Name                | RW | Default | Description                                                    |
|-------|---------------------|----|---------|----------------------------------------------------------------|
| 24-20 | ICH4_SEL (TMR1_CAP) | rw | 0       | 同ICH0选项                                                        |
| 19-15 | ICH3_SEL (TMR0_CAP) | rw | 0       | 同ICH0选项                                                        |
| 14-10 | ICH2_SEL (TMR2_CIN) | rw | 0       | 同ICH0选项                                                        |
| 9-5   | ICH1_SEL (TMR1_CIN) | rw | 0       | 同ICH0选项                                                        |
| 4-0   | ICH0_SEL (TMR0_CIN) | rw | 0       | 功能输入通道选择<br>0-4: gp_ich0-gp_ich3<br>6: tmr1_pwm<br>7: tmr2_pwm |

### 5.2.8 ICH\_IOMC1: input channel control register1

偏移地址: 0x44

| Bit   | Name                   | RW | Default | Description |
|-------|------------------------|----|---------|-------------|
| 31-30 | Reserved               | -  | -       |             |
| 29-25 | ICH11_SEL (PLNK_IDAT0) | rw | 0       | 同ICH0选项     |
| 24-20 | ICH10_SEL (UART1_CTS)  | rw | 0       | 同ICH0选项     |
| 19-15 | ICH9_SEL (MC_PWM1_FP)  | rw | 0       | 同ICH0选项     |
| 14-10 | ICH8_SEL (MC_PWM0_FP)  | rw | 0       | 同ICH0选项     |
| 9-5   | ICH7_SEL (MC_PWM1_CK)  | rw | 0       | 同ICH0选项     |
| 4-0   | ICH6_SEL (MC_PWM0_CK)  | rw | 0       | 同ICH0选项     |

### 5.2.9 ICH\_IOMC2: input channel control register2

偏移地址: 0x48

| Bit   | Name                     | RW | Default | Description |
|-------|--------------------------|----|---------|-------------|
| 31-30 | Reserved                 | -  | -       |             |
| 29-25 | ICH17_SEL (SPI0_CSI)     | rw | 0       | 同ICH0选项     |
| 24-20 | ICH16_SEL (AUD_DBG_DATI) | rw | 0       | 同ICH0选项     |
| 19-15 | ICH15_SEL (EXT_CLK)      | rw | 0       | 同ICH0选项     |

| Bit   | Name                    | RW | Default | Description |
|-------|-------------------------|----|---------|-------------|
| 14-10 | ICH14_SEL (CLK_MUX_IN)  | rw | 0       | 同ICH0选项     |
| 9-5   | ICH13_SEL (CAP_MUX_OUT) | rw | 0       | 同ICH0选项     |
| 4-0   | ICH12_SEL (PLNK_IDAT1)  | rw | 0       | 同ICH0选项     |

### 5.2.10 ICH\_IOMC3: input channel control register3

偏移地址: 0x4c

| Bit   | Name                 | RW | Default | Description |
|-------|----------------------|----|---------|-------------|
| 31-30 | Reserved             | -  | -       |             |
| 29-25 | ICH23_SEL (Reserve)  | rw | 0       | 同ICH0选项     |
| 24-20 | ICH22_SEL (Reserve)  | rw | 0       | 同ICH0选项     |
| 19-15 | ICH21_SEL (Reserve)  | rw | 0       | 同ICH0选项     |
| 14-10 | ICH20_SEL (Reserve)  | rw | 0       | 同ICH0选项     |
| 9-5   | ICH19_SEL (SPI2_CSI) | rw | 0       | 同ICH0选项     |
| 4-0   | ICH18_SEL (SPI1_CSI) | rw | 0       | 同ICH0选项     |

### 5.2.11 ICH\_IOMC4: input channel control register4

偏移地址: 0x50

| Bit   | Name                | RW | Default | Description |
|-------|---------------------|----|---------|-------------|
| 31-30 | Reserve             | -  | -       |             |
| 29-25 | ICH29_SEL (Reserve) | rw | 0       | 同ICH0选项     |
| 24-20 | ICH28_SEL (Reserve) | rw | 0       | 同ICH0选项     |
| 19-15 | ICH27_SEL (Reserve) | rw | 0       | 同ICH0选项     |
| 14-10 | ICH26_SEL (Reserve) | rw | 0       | 同ICH0选项     |
| 9-5   | ICH25_SEL (Reserve) | rw | 0       | 同ICH0选项     |
| 4-0   | ICH24_SEL (Reserve) | rw | 0       | 同ICH0选项     |

### 5.2.12 ICH\_IOMC5: input channel control register5

偏移地址: 0x54

| Bit   | Name                 | RW | Default | Description |
|-------|----------------------|----|---------|-------------|
| 31-20 | Reserved             | -  | -       |             |
| 19-15 | ICH33_SEL (Reserved) | rw | 0       | 同ICH0选项     |
| 14-10 | ICH32_SEL (Reserved) | rw | 0       | 同ICH0选项     |
| 9-5   | ICH31_SEL (Reserved) | rw | 0       | 同ICH0选项     |
| 4-0   | ICH30_SEL (Reserved) | rw | 0       | 同ICH0选项     |

### 5.2.13 OCH\_IOMC0: output channel control register0

偏移地址: 0x58

| Bit   | Name     | RW | Default | Description                                                                                                                                                                                                                                                                                                                                                                                              |
|-------|----------|----|---------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 31-24 | OCH3_SEL | rw | 0       | 同OCH0选项                                                                                                                                                                                                                                                                                                                                                                                                  |
| 23-16 | OCH2_SEL | rw | 0       | 同OCH0选项                                                                                                                                                                                                                                                                                                                                                                                                  |
| 15-8  | OCH1_SEL | rw | 0       | 同OCH0选项                                                                                                                                                                                                                                                                                                                                                                                                  |
| 7-0   | OCH0_SEL | rw | 0       | 通用输出通道选择<br>0: tmr0_pwm<br>1: tmr1_pwm<br>2: tmr2_pwm<br>3: gp_ich0<br>4: gp_ich1<br>5: uart1_rts<br>6: plnk_clk<br>7: aud_dbg_clko<br>8: aud_dbg_dato[0]<br>9: aud_dbg_dato[1]<br>10: aud_dbg_dato[2]<br>11: aud_dbg_dato[3]<br>12: aud_dbg_dato[4]<br>13: clk_out2io[0]<br>14: 1'b0<br>15: clk_out2io[2]<br>16: p33_clk_dbg<br>17: p33_sig_dbg[0]<br>18: p33_sig_dbg[1]<br>19: usb_dbg_out<br>20: 1'b0 |

### 5.2.14 OCH\_IOMC1: output channel control register1

偏移地址: 0x5c

| Bit   | Name     | RW | Default | Description |
|-------|----------|----|---------|-------------|
| 31-24 | OCH7_SEL | -  | -       | Reserved    |
| 23-16 | OCH6_SEL | -  | -       | Reserved    |
| 15-8  | OCH5_SEL | -  | -       | Reserved    |
| 7-0   | OCH4_SEL | -  | -       | Reserved    |

### 5.2.15 OCH\_IOMC2/3/4/5: output channel control register2/3/4/5

偏移地址: 0x60/0x64/0x68/0x6c

| Bit   | Name                                    | RW | Default | Description |
|-------|-----------------------------------------|----|---------|-------------|
| 31-24 | OCH11_SEL/OCH15_SEL/OCH19_SEL/OCH23_SEL | -  | -       | Reserved    |
| 23-16 | OCH10_SEL/OCH14_SEL/OCH18_SEL/OCH22_SEL | -  | -       | Reserved    |
| 15-8  | OCH9_SEL/OCH13_SEL/OCH17_SEL/OCH21_SEL  | -  | -       | Reserved    |
| 7-0   | OCH8_SEL/OCH12_SEL/OCH16_SEL/OCH20_SEL  | -  | -       | Reserved    |

### 5.2.16 FSPG\_CON: FSPG control register

偏移地址: 0x00

| Bit  | Name     | RW | Default | Description                                                                                    |
|------|----------|----|---------|------------------------------------------------------------------------------------------------|
| 31-3 | Reserved | -  | -       | Reserved                                                                                       |
| 2    | SOFT     | rw | 0       | FSPG 驱动强度选择, 为flash供电时, 上下电速度的控制<br>00: PAD高阻<br>0: 快上/下电 (tr/tf≤2us)<br>1: 慢上/下电 (tr/tf≥20us) |
| 1    | CS1_EN   | rw | 0       | 是否短接CS1到FSPG<br>该功能用于FLASH上/下电时避免电流通过CS PIN倒灌                                                  |

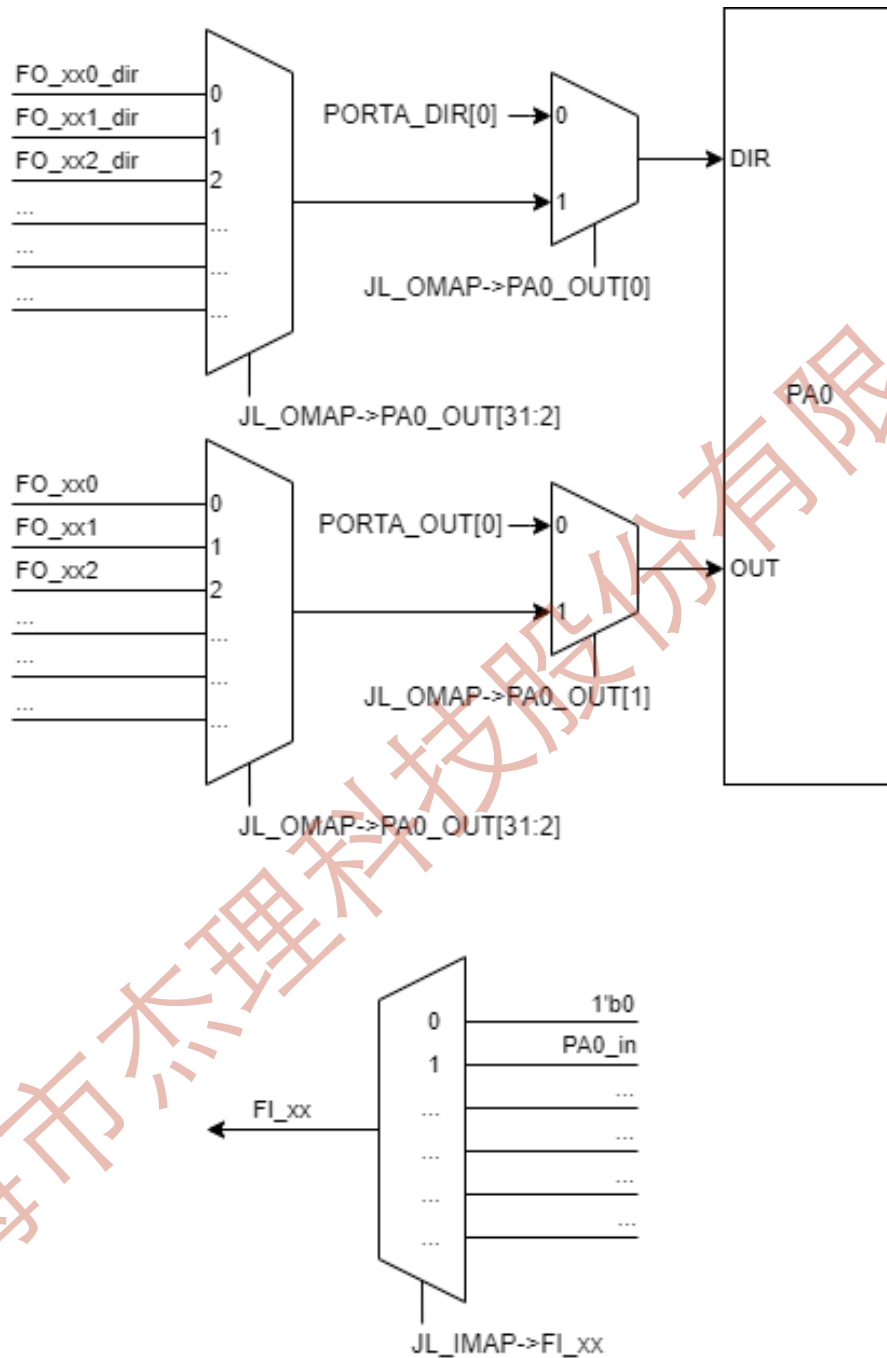
| Bit | Name   | RW | Default | Description                                   |
|-----|--------|----|---------|-----------------------------------------------|
| 0   | CS0_EN | rw | 0       | 是否短接CS0到FSPG<br>该功能用于FLASH上/下电时避免电流通过CS PIN倒灌 |

珠海市杰理科技股份有限公司



## 6 PORT\_Crossbar

### 6.1 Crossbar功能结构图



### 6.2 寄存器功能

#### 6.2.1 JL\_OMAP-> POUT

| Bit | Name | RW | Description |
|-----|------|----|-------------|
|-----|------|----|-------------|

| Bit    | Name       | RW | Description                                                               |
|--------|------------|----|---------------------------------------------------------------------------|
| [0]    | FUN_DIR_EN | rw | Crossbar 方向控制使能:<br>0: CPU 控制<br>1: 外设控制<br>(注: 若所选外设无方向控制功能, 则都由 CPU 控制) |
| [1]    | FUN_OUT_EN | rw | Crossbar 数据控制使能:<br>0: CPU 数据<br>1: 外设数据                                  |
| [31:2] | FUN_SELECT | rw | Crossbar 输出功能选择:<br>(见功能表或头文件)                                            |

### 6.2.2 JL\_IMAP-> FUN\_IN

| Bit    | Name         | RW | Description                    |
|--------|--------------|----|--------------------------------|
| [31:0] | INPUT_SELECT | rw | Crossbar 输出引脚选择:<br>(见功能表或头文件) |

## 7 CLOCK\_SYSTEM

### 7.1 原生时钟源

本芯片具备的原生时钟源如下表所示：

| 时钟       | 概述                              |
|----------|---------------------------------|
| rc_250k  | 内置RC振荡器，用于复位系统和watch dog功能时钟    |
| rc_16m   | 内置RC振荡器，用于系统启动的初始时钟。            |
| lrc_200k | 内置低温度电压漂移RC振荡器，用于低功耗计数和PLL参考时钟。 |
|          |                                 |

### 7.2 衍生时钟源

衍生时钟源于芯片内部的PLL

#### 7.2.1 内部PLL 参考时钟源

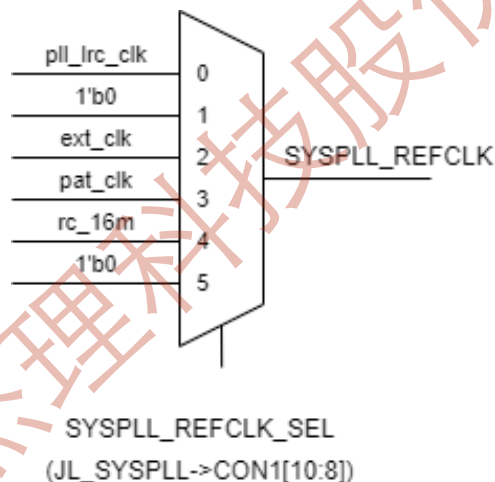
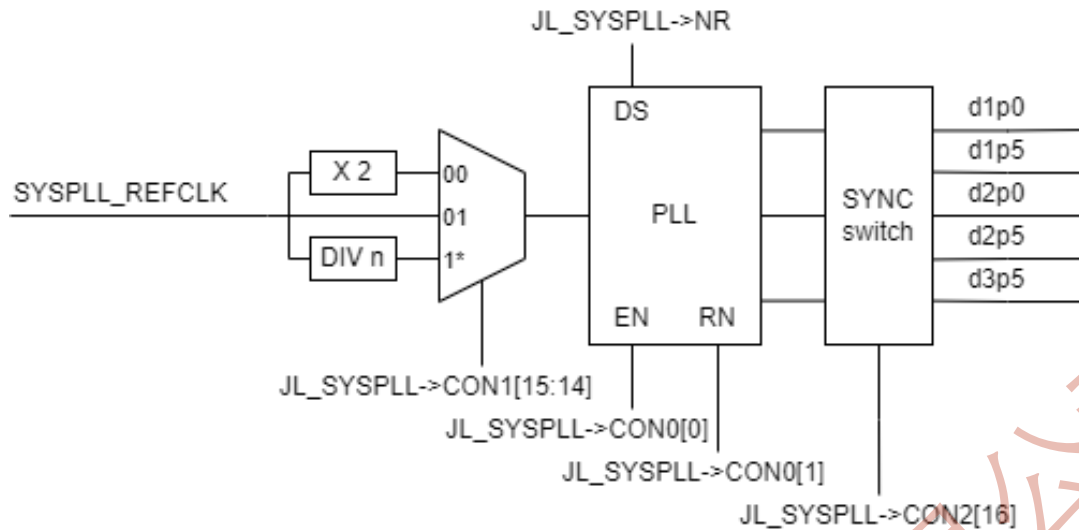


图1 内部PLL 参考时钟源

#### 7.2.2 SYSPLL

SYSPLL工作范围覆盖160-240MHz，同时两者均输出

PLL\_D1P0, PLL\_D1P5, PLL\_D2P0, PLL\_D2P5, PLL\_D3P5的时钟，如果SYSPLL配置为240MHz，则能输出240MHz、160MHz、120MHz、96MHz和68.5MHz；每个时钟都有独立的使能端。在不使用该时钟时，软件应关闭其使能端以减少额外电源消耗。



$$F_{out} = F_{ref} / n * m; \quad (n = SYSPLL\_CON1[6:0] + 2, \quad m = SYSPLL\_NR[11:0] + 2)$$

Fout 可以为 160-240 MHz, 建议值为192MHz

图2 SYSPLL时钟结构框图

### 7.3 标准时钟

除上述的原生时钟外, 芯片内还有频率固定的标准时钟, 分别是wclk、pll\_96m、std\_48m、std\_24m和std\_12m, 其时钟源头选择如下:

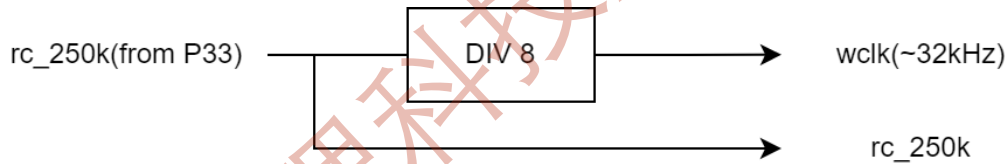


图3 系统 rc\_250k 和 wclk 时钟结构

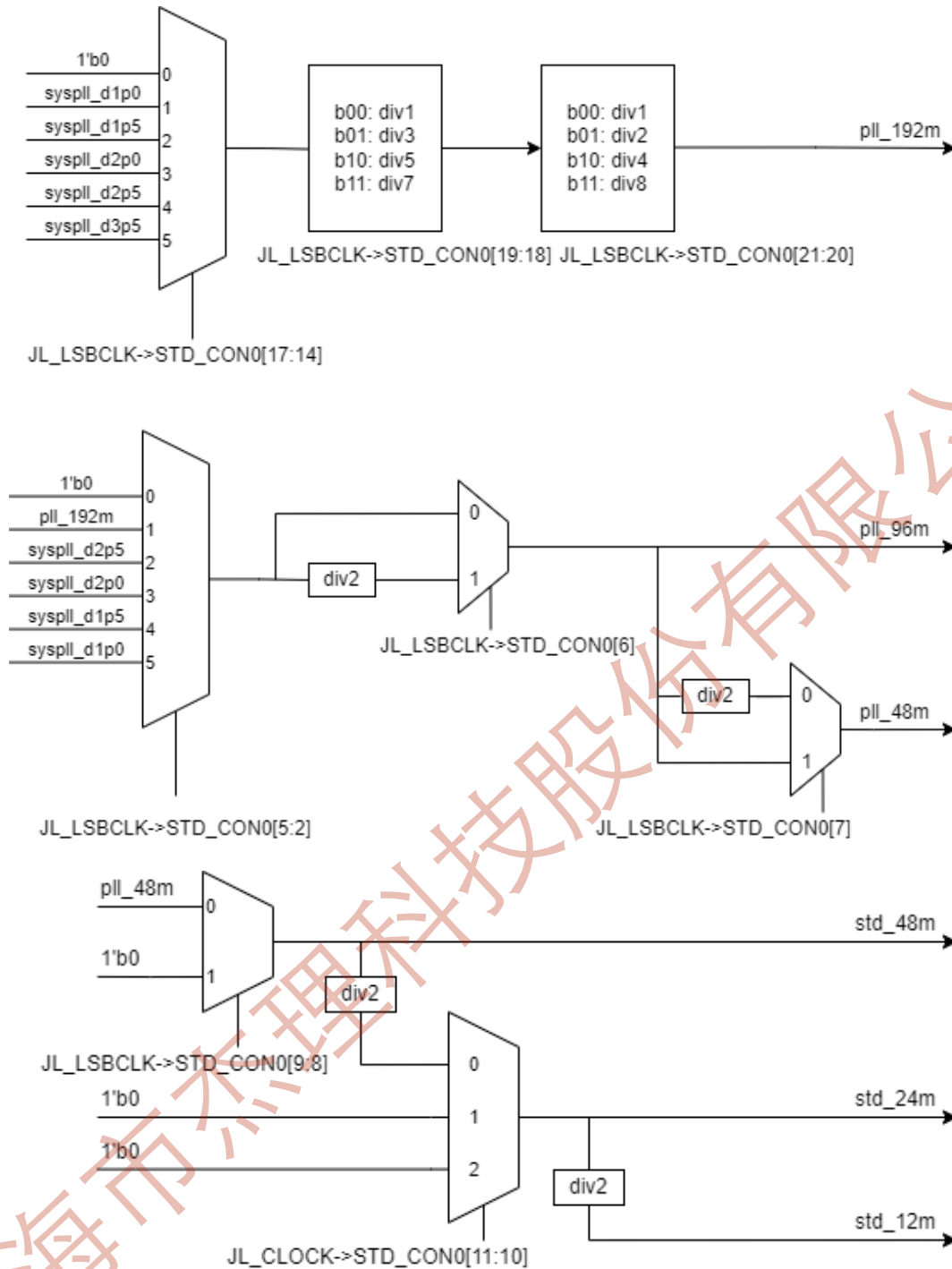


图4 STD\_CLK时钟结构图

## 7.4 系统时钟

系统时钟部分电路框图如下，系统运行过程中可随时更改hsb\_clk, lsb\_clk的分频值，但需确保在任何时刻，各分频时钟都不会超过其允许的最高运行频率。

### 7.4.1 HSB时钟源

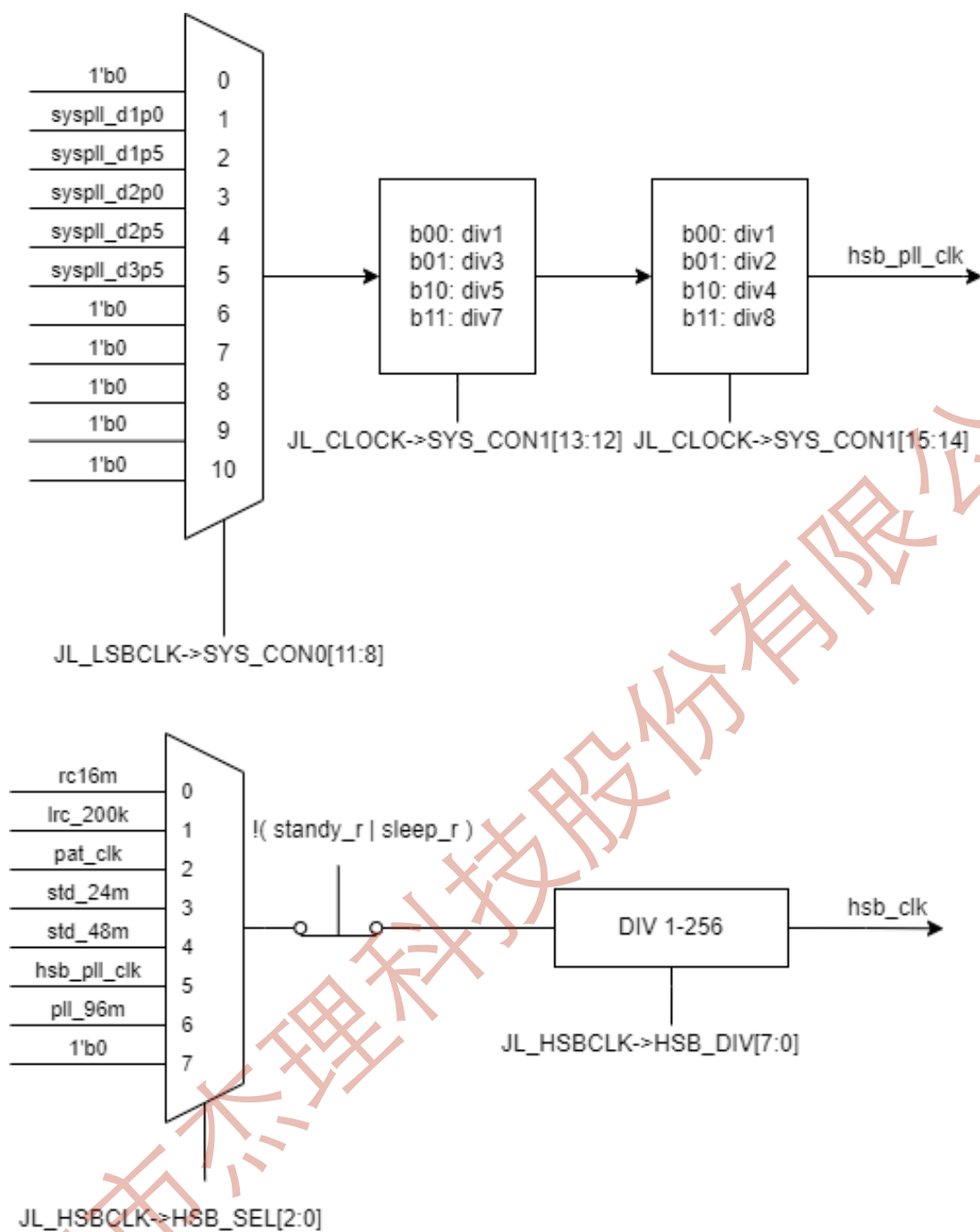


图5 hsb\_clk时钟结构图

## 7.4.2 LSB时钟源

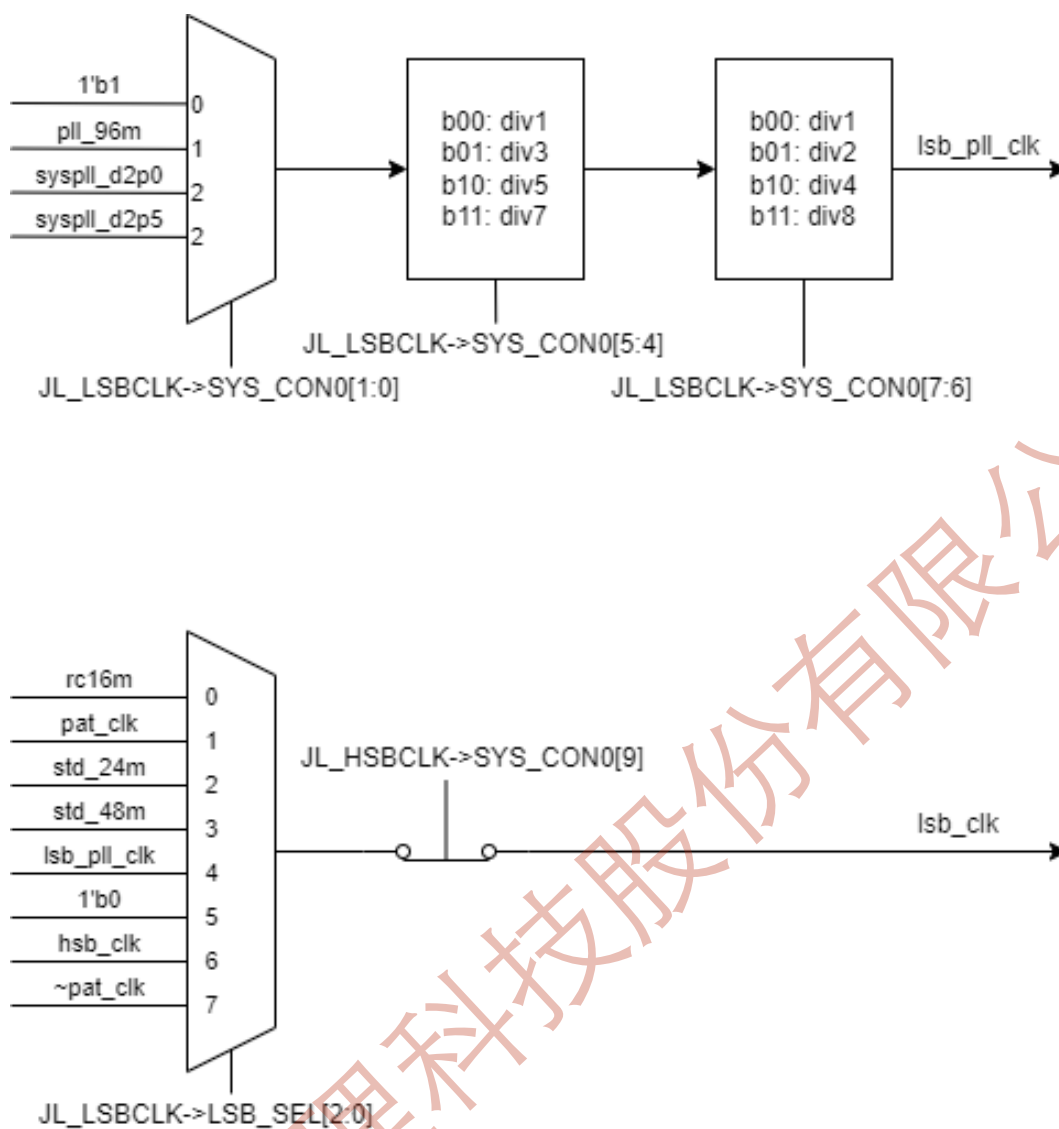


图6 lsb\_clk时钟电路

【注意】：LSB\_SEL 不能连续切换，两次切换中间需要等待5个LSB\_CLK;

## 7.5 外设时钟

### 7.5.1 SFC 时钟源

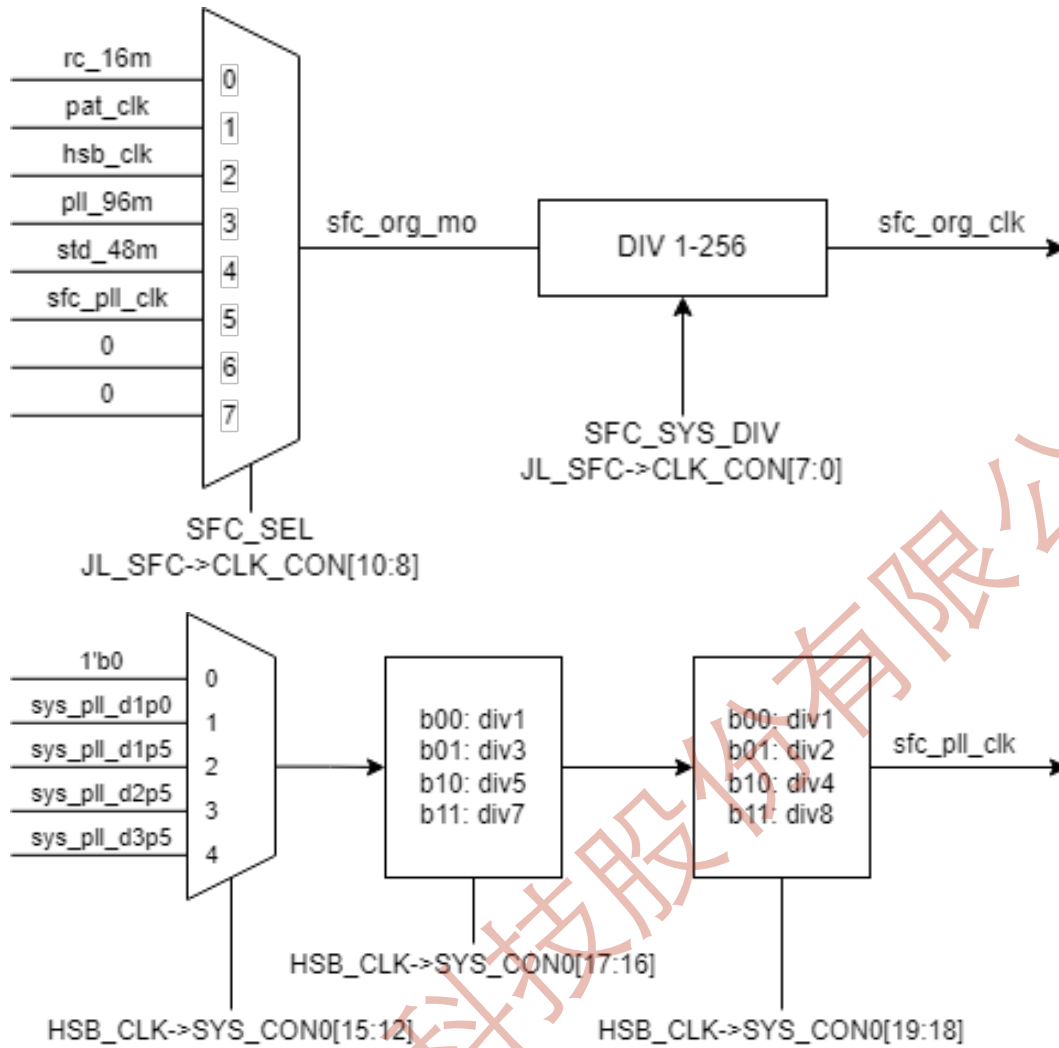
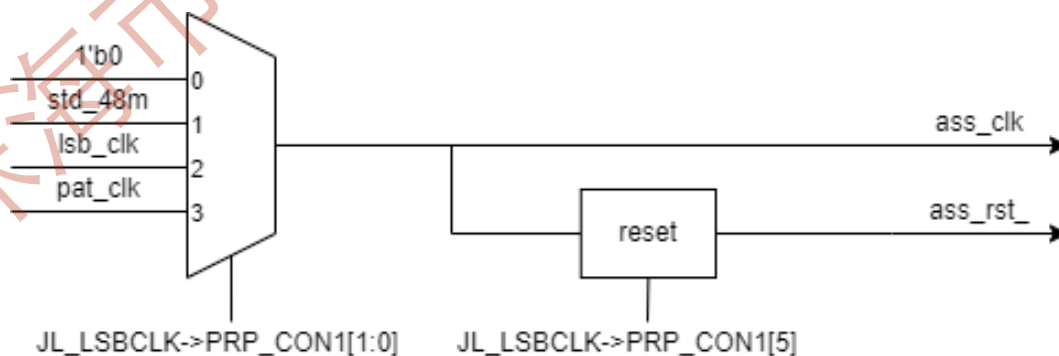


图7 SFC时钟结构图

【注意】：SFC\_CLK可动态切换，但不能连续切换，两次切换中间需要等待大于5个SFC\_CLK；  
【注意2】：sfc\_pll\_clk的时钟与sfc\_org\_clk的控制寄存器的基地址不同

### 7.5.2 音频时钟源





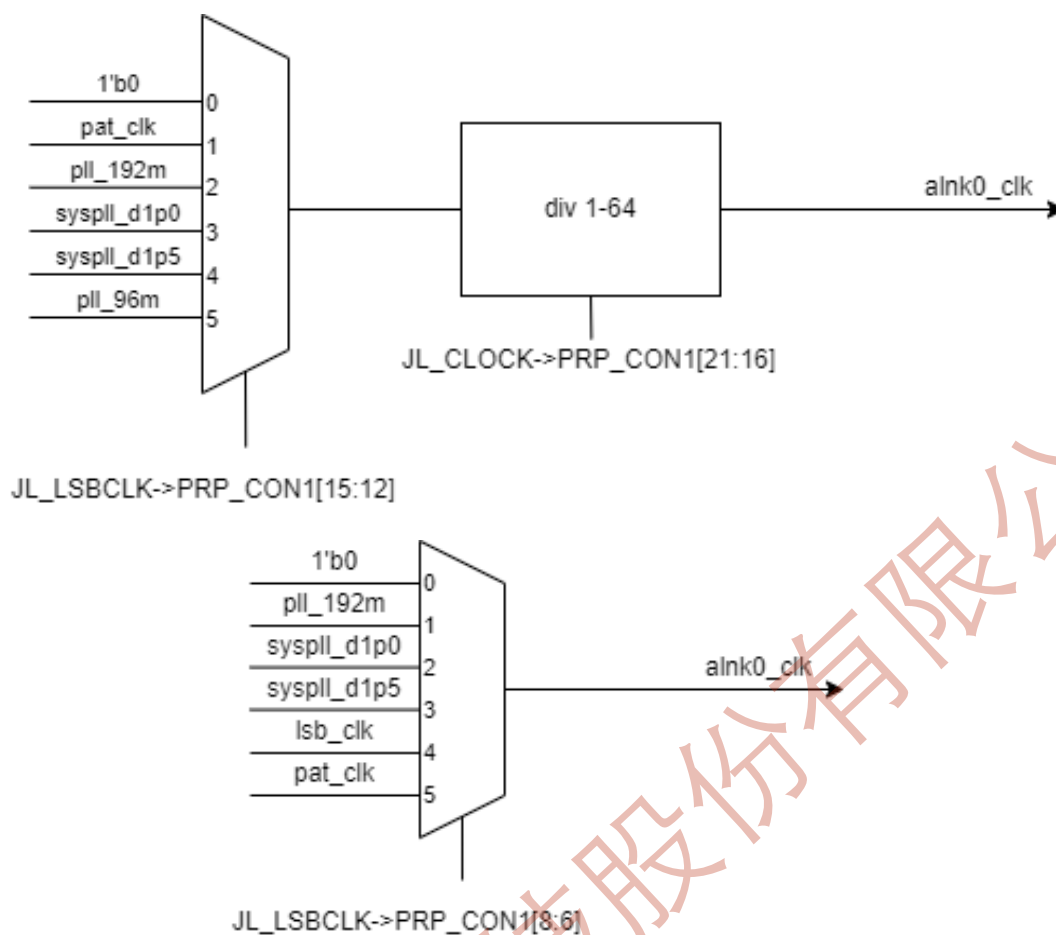
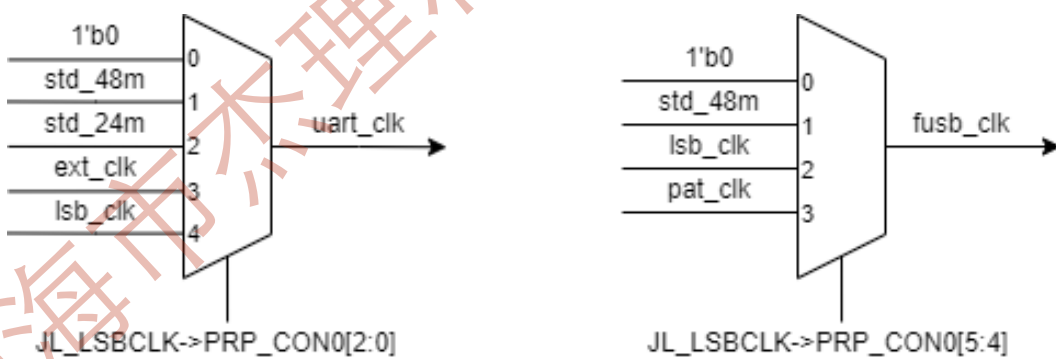


图8 音频时钟结构图

### 7.5.3 外设时钟源



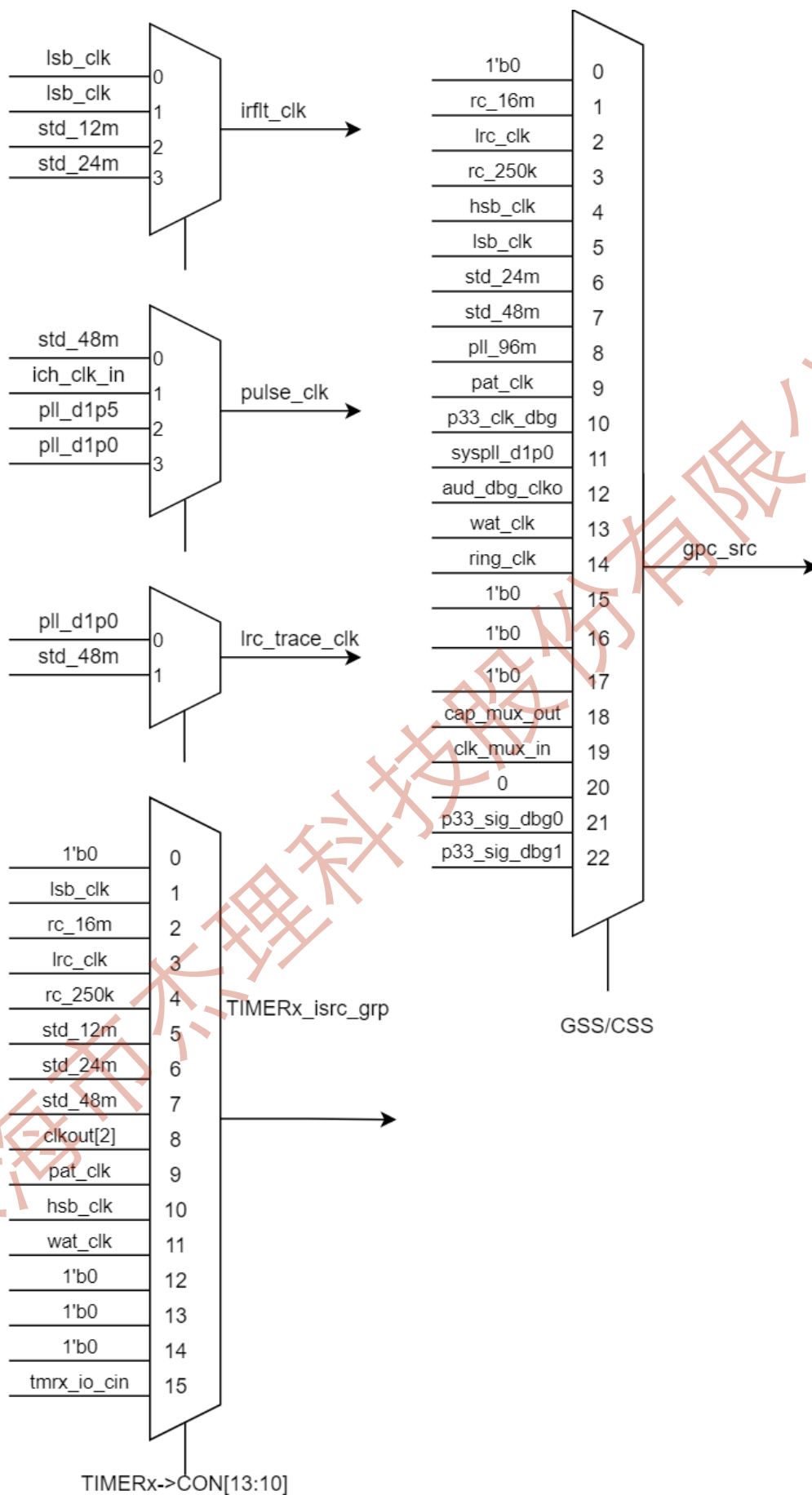


图10 低速外设时钟结构图

- 该芯片有2组时钟输出，可以通过crossbar将内部的时钟信号驱动至片外，用于测试或特殊用途。

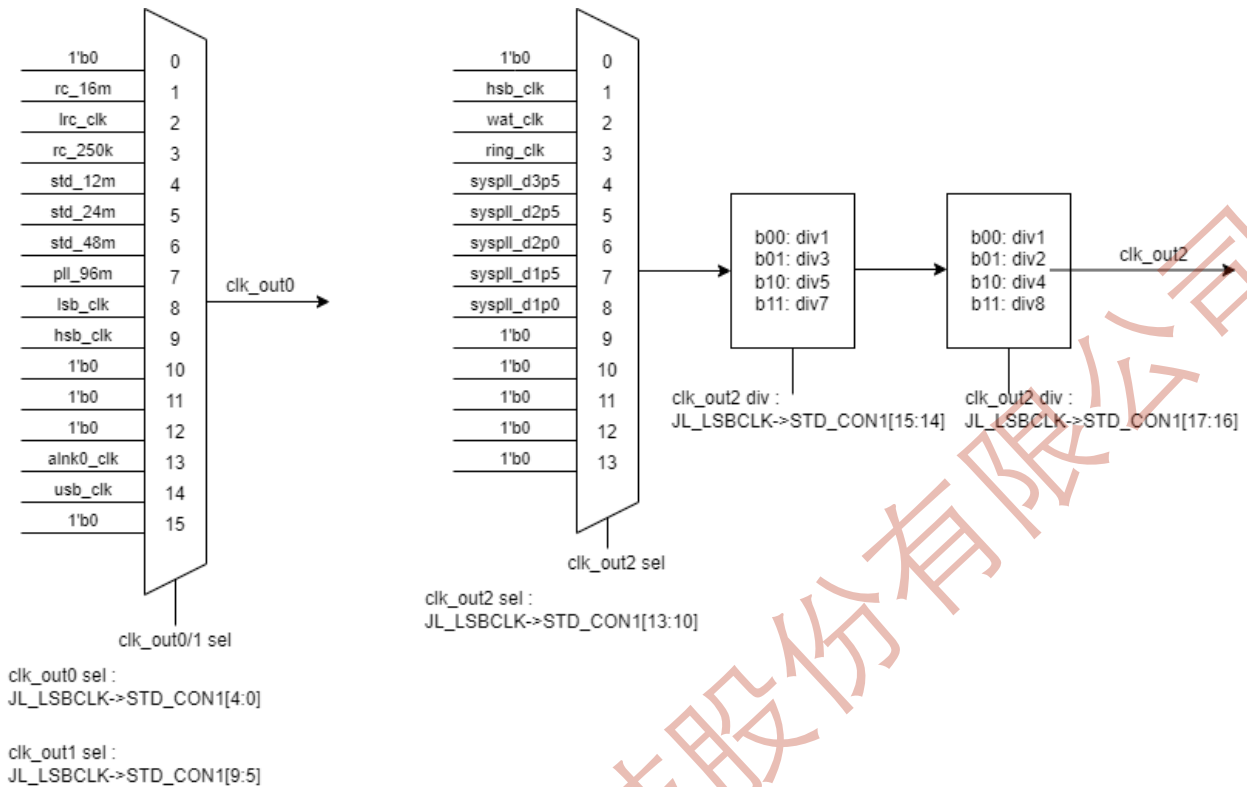


图11 IO输出时钟结构图

## 7.6 数字模块控制寄存器

### 7.6.1 JL\_LSBCK->SYS\_CON0: lsbck system config register 0

| Bit   | Name        | RW | Default | RV | Description     |
|-------|-------------|----|---------|----|-----------------|
| 31-11 | RESERVED    | -  | 0       | -  | 预留              |
| 15-12 | HSB_PLL_DIV | rw | 0       | -  | HSB_PLL时钟分频，见图5 |
| 11-8  | HSB_PLL_SEL | rw | 0       | -  | HSB_PLL时钟选择，见图5 |
| 7-4   | LSB_PLL_DIV | rw | 0       | -  | LSB_PLL时钟分频，见图6 |
| 3-0   | LSB_PLL_SEL | rw | 0       | -  | LSB_PLL时钟选择，见图6 |

### 7.6.2 LSB\_SEL: lsb clock sel register

| Bit   | Name     | RW | Default | RV | Description |
|-------|----------|----|---------|----|-------------|
| 31-20 | RESERVED | -  | 0       | -  | 预留          |
| 2-0   | LSB_SEL  | rw | 0       | -  | LSB时钟选择，见图6 |

### 7.6.3 JL\_LSBCK->STD\_CON0: standard clock register 0

| Bit   | Name         | RW | Default | RV | Description                                                    |
|-------|--------------|----|---------|----|----------------------------------------------------------------|
| 31-9  | RESERVED     | -  | 0       | -  | 预留                                                             |
| 21-18 | PLL_192M_DIV | rw | 0       | -  | PLL_192M分频系数, 见图4                                              |
| 17-14 | PLL_192M_SEL | rw | 0       | -  | PLL_192M输入时钟源选择, 见图4                                           |
| 13-12 | RESERVED     | -  | 0       | -  | 预留                                                             |
| 11-10 | STD_24M_SEL  | rw | 0       | -  | STD_24M来源选择, 见图4                                               |
| 9-8   | STD_48M_SEL  | rw | 0       | -  | STD_48M来源选择, 见图4                                               |
| 7     | PLL_48M_DS   | rw | 0       | -  | PLL_48M DIV2使能选择, 见图4                                          |
| 6     | PLL_96M_DS   | rw | 0       | -  | PLL_96M DIV2使能选择, 见图4                                          |
| 5-2   | PLL_96M_SEL  | rw | 0       | -  | PLL_96M输入时钟源选择, 见图4                                            |
| 1     | RC_250K_EN   | rw | 1       | -  | 片内RC (250k) 振荡器使能 (PMU低功耗模式无效)<br>0: 关闭片内RC振荡器<br>1: 打开片内RC振荡器 |
| 0     | RC_16M_EN    | rw | 1       | -  | 片内RC (16M) 振荡器使能 (PMU低功耗模式无效)<br>0: 关闭片内RC振荡器<br>1: 打开片内RC振荡器  |

### 7.6.4 JL\_LSBCK->STD\_CON1: standard clock register 1

| Bit   | Name        | RW | Default | RV | Description            |
|-------|-------------|----|---------|----|------------------------|
| 31-20 | RESERVED    | -  | -       | -  | 预留                     |
| 17-14 | CLKOUT2_DIV | rw | 1       | -  | CLK_OUT2输出时钟分频选择, 见图11 |
| 13-10 | CLKOUT2_SEL | rw | 0       | -  | CLK_OUT2输出时钟源选择, 见图11  |
| 9-5   | RESERVED    | -  | -       | -  | 预留                     |
| 4-0   | CLKOUT0_SEL | rw | 0       | -  | CLK_OUT0输出时钟选择, 见图11   |

### 7.6.5 JL\_LSBCK->STD\_CON2: standard clock register 2

| Bit | Name | RW | Default | RV | Description |
|-----|------|----|---------|----|-------------|
|-----|------|----|---------|----|-------------|

| Bit  | Name     | RW | Default | RV | Description |
|------|----------|----|---------|----|-------------|
| 31-0 | RESERVED | -  | -       | -  | 预留          |

#### 7.6.6 JL\_LSBCK->PRP\_CON0: peripheral clock config register 0

| Bit   | Name         | RW | Default | RV | Description         |
|-------|--------------|----|---------|----|---------------------|
| 31-10 | RESERVED     | -  | 0       | -  | 预留                  |
| 9     | RINGOSC_CKEN | rw | 0       | -  | ring_clk使能          |
| 8-7   | MBIST_CKSEL  | rw | 0       | -  | IC内部测试功能, MBIST时钟源  |
| 6     | USB_CKPOL    | rw | 0       | 0  | IC内部测试功能, fusb时钟反相  |
| 5-4   | USB_CKSEL    | rw | 0       | -  | fusb时钟源选择, 见图10     |
| 3     | UART_CKPOL   | rw | 0       | 0  | IC内部测试功能, uart 时钟反相 |
| 2-0   | UART_CKSEL   | rw | 0       | -  | uart 时钟源选择, 见图10    |

#### 7.6.7 JL\_LSBCK->PRP\_CON1: peripheral clock config register 1

| Bit   | Name         | RW | Default | RV | Description        |
|-------|--------------|----|---------|----|--------------------|
| 31-22 | RESERVED     | -  | 0       | -  | 预留                 |
| 21-16 | ALNK0_DIV    | rw | 0       | -  | alnk0_clk分频选择, 见图8 |
| 15-12 | ALNK0_SEL    | rw | 0       | -  | alnk0_clk来源选择, 见图8 |
| 11-10 | RESERVED     | -  | 0       | -  | 预留                 |
| 9     | APA_TEST     | rw | 0       | 0  | IC内部测试功能, 应保持为0    |
| 8-6   | APA_CKSEL    | rw | 0       | -  | apa_clk来源选择, 见图8   |
| 5     | ASS_REG_RSTN | rw | 0       | -  | ass复位              |
| 4-2   | RESERVED     | -  | 0       | -  | 预留                 |
| 1-0   | ASS_CKSEL    | rw | 0       | -  | ass_clk来源选择, 见图8   |

#### 7.6.8 JL\_HSBCK->SYS\_CON0: hsbck system config register 0

| Bit   | Name        | RW | Default | RV | Description     |
|-------|-------------|----|---------|----|-----------------|
| 31-11 | RESERVED    | -  | -       | -  | 预留              |
| 19-16 | SFC_PLL_DIV | rw | 0       | -  | sfc_pll_mo的时钟分频 |

| Bit   | Name         | RW | Default | RV | Description                                                                                          |
|-------|--------------|----|---------|----|------------------------------------------------------------------------------------------------------|
| 15-12 | SFC_PLL_SEL  | rw | 0       | -  | sfc_pll_mo的来源选择:<br>0: 空<br>1: sys_pll_d1p0<br>2: sys_pll_d1p5<br>3: sys_pll_d2p5<br>4: sys_pll_d3p5 |
| 11    | RESERVED     | -  | -       | -  | 预留                                                                                                   |
| 10    | LSB_SFR_CKMD | rw | 0       | -  | lsb_sfr_clk时钟模式选择:<br>0: 有SFR写的时候, lsb_sfr_clk才使能<br>1: 无论有无SFR写, lsb_sfr_clk都使能                     |
| 9     | LSB_CKEN     | rw | 1       | -  | LSB时钟使能, 默认开                                                                                         |
| 8     | AXI_EN       | rw | 1       | -  | axi_clk使能:<br>0: 不使能<br>1: 使能                                                                        |
| 7-1   | RESERVED     | -  | 0       | -  | 预留                                                                                                   |
| 0     | HSB_SFR_CKMD | rw | 0       | -  | hsb_sfr_clk时钟模式选择:<br>0: 有SFR写的时候, hsb_sfr_clk才使能<br>1: 无论有无SFR写, hsb_sfr_clk都使能                     |

#### 7.6.9 HSB\_DIV: hsb clock div register

| Bit   | Name     | RW | Default | RV | Description  |
|-------|----------|----|---------|----|--------------|
| 31-20 | RESERVED | -  | 0       | -  | 预留           |
| 7-0   | HSB_DIV  | rw | 0       | -  | HSB时钟分频, 见图5 |

#### 7.6.10 HSB\_SEL: hsb clock sel register

| Bit   | Name     | RW | Default | RV | Description  |
|-------|----------|----|---------|----|--------------|
| 31-20 | RESERVED | -  | 0       | -  | 预留           |
| 2-0   | HSB_SEL  | rw | 0       | -  | HSB时钟选择, 见图5 |

#### 7.6.11 SYSPLL\_CON0: pll control register 0

| Bit   | Name         | RW | Default | RV | Description |
|-------|--------------|----|---------|----|-------------|
| 31-30 | RESERVED     | -  | -       | -  | 预留          |
| 29    | PLL_VCTRL_OE | rw | 0       | -  | VBG TRIM使能位 |

| Bit   | Name         | RW | Default | RV | Description                                    |
|-------|--------------|----|---------|----|------------------------------------------------|
| 28    | PLL_TEST_EN  | rw | 0       | -  | PLL模拟控制位                                       |
| 27    | RESERVED     | -  | -       | -  | 预留                                             |
| 26-25 | PLL_TEST_S   | rw | 0       | -  | PLL测试使能                                        |
| 24    | PLL_LDO_BP   | rw | 0       | -  | PLL测试功能选择                                      |
| 23-20 | RESERVED     | -  | -       | -  | 预留                                             |
| 19-16 | PLL_LDO11A_S | rw | 0       | -  | PLL模拟控制位                                       |
| 15-13 | PLL_IVCOS    | rw | 0       | -  | PLL模拟控制位                                       |
| 12-10 | PLL_LPFR2    | rw | 0       | -  | PLL模拟控制位                                       |
| 9     | RESERVED     | -  | -       | -  | 预留                                             |
| 8-7   | PLL_ICPS     | rw | 0       | -  | PLL模拟控制位                                       |
| 5-4   | PLL_PFDS     | rw | 0       | -  | PLL模拟控制位                                       |
| 2     | PLL_MODE     | rw | 0       | -  | PLL工作模式<br>0: 整数模式<br>1: 小数模式                  |
| 1     | PLL_RST      | rw | 0       | 1  | PLL模块复位，需在PLL EN使能10uS之后才能释放<br>0: 复位<br>1: 释放 |
| 0     | PLL_EN       | rw | 0       | 1  | PLL模块使能<br>0: 关闭<br>1: 打开                      |

#### 7.6.12 SYSPLL\_CON1: pll control register 1

| Bit   | Name         | RW | Default | RV | Description                                                                                       |
|-------|--------------|----|---------|----|---------------------------------------------------------------------------------------------------|
| 31-16 | RESERVED     | -  | -       | -  | 预留                                                                                                |
| 15-14 | PLL_REFDSSEN | rw | 0       | -  | PLL参考时钟分频使能（见图1）<br>00: PLL内参考时钟分频器X2；<br>01: PLL内参考时钟分频器关闭（DIV1）；<br>1*: PLL内参考时钟分频器打开（DIV2-129） |

| Bit  | Name        | RW | Default | RV | Description                                   |
|------|-------------|----|---------|----|-----------------------------------------------|
| 12   | PLL_REFSEL  | rw | 0       | -  | 0:PLL_REFCLKP0_20v<br>1:PLL_REFCLK_08v        |
| 11   | RESERVED    | -  | -       | -  | 预留                                            |
| 10-8 | PLL_REF_SEL | rw | 0       | -  | PLL模块参考时钟选择（见图1）                              |
| 6-0  | PLL_REFDS   | rw | 0       | -  | PLL参考时钟分频值， $n = \text{PLL\_REFDS} + 2$ （见图1） |

#### 7.6.13 SYSPLL\_NR: pll config register nr

| Bit   | Name     | RW | Default | RV | Description                                                         |
|-------|----------|----|---------|----|---------------------------------------------------------------------|
| 31-12 | RESERVED | -  | -       | -  | 预留                                                                  |
| 11-0  | PLL_NR   | rw | 0       | -  | PLL参考时钟反馈分频(相当倍频),<br>$m = \text{PLL\_NR} + 2$ 。见图1<br>注意：读出值为写入值+2 |

#### 7.6.14 SYSPLL\_CON2: pll control register 2

| Bit   | Name              | RW | Default | RV | Description |
|-------|-------------------|----|---------|----|-------------|
| 31-16 | RESERVED          | -  | -       | -  | 预留          |
| 16    | PLL_CKEN          | rw | 0       | 0  | 输出时钟使能      |
| 15-7  | RESERVED          | -  | -       | -  | 预留          |
| 6     | PLL_CKSYN_CORE_OE | rw | 0       | 0  | 同步时钟使能      |
| 5     | RESERVED          | -  | -       | -  | 预留          |
| 4     | PLL_CKOUT_D3P5_OE | rw | 0       | -  | 3.5分频时钟输出使能 |
| 3     | PLL_CKOUT_D2P5_OE | rw | 0       | -  | 2.5分频时钟输出使能 |
| 2     | PLL_CKOUT_D2P0_OE | rw | 0       | -  | 2分频时钟输出使能   |
| 1     | PLL_CKOUT_D1P5_OE | rw | 0       | -  | 1.5分频时钟输出使能 |
| 0     | PLL_CKOUT_D1P0_OE | rw | 0       | -  | 1分频时钟输出使能   |



## 8 ADC

### 8.1 概述

10Bit ADC(A/D转换器)，其时钟最大不可超过500KHz。

### 8.2 寄存器说明

#### 8.2.1 ADC\_CON: ADC control register

| Bit   | Name         | RW | Default | RV | Description                                                                                                    |
|-------|--------------|----|---------|----|----------------------------------------------------------------------------------------------------------------|
| 31    | DONE_PND     | r  | 0       | -  | 只读，采样完成pnd; kst后完成一次该位置1                                                                                       |
| 30    | DONE_PND_CLR | w  | -       | -  | 只写，写‘1’清除DONE_PND位，写‘0’无效                                                                                      |
| 29    | DONE_PND_IE  | rw | 0       | -  | DONE_PND中断允许                                                                                                   |
| 28-24 | RESERVED     | -  | -       | -  | 预留                                                                                                             |
| 23-21 | ADC_MUX_SEL  | rw | 0x0     | 0  | ADC模式选择：<br>001: ADC采集IO通道电压<br>010: ADC采集内部模拟通道电压<br>111: 将选通的内部模拟电压通过IO口输出                                   |
| 20-18 | ADC_ASEL     | rw | 0x0     | -  | 内部模拟通道选择：<br>000: 无<br>001: AUDIO<br>010: PMU<br>011: LRC200K<br>100: 无<br>101:SYSPLL<br>110:WAT<br>111:CLASSD |
| 17    | ADC_CLKEN    | rw | 0       | 0  | ADC 时钟使能                                                                                                       |
| 16    | ADCISEL      | rw | 0       | -  | ADC CMP power select                                                                                           |
| 15-12 | WAIT_TIME    | rw | x       | -  | 启动延时控制，实际启动延时为此数值乘8个ADC时钟                                                                                      |

| Bit  | Name     | RW | Default | RV | Description                                                                                                                                                                                                                                                 |
|------|----------|----|---------|----|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 11-8 | CH_SEL   | rw | x       | -  | IO通道选择:<br>0000: 选择PA0<br>0001: 选择PA1<br>0010: 选择PA2<br>0011: 选择PA3<br>0100: 选择DP<br>0101: 选择DM<br>0110: 选择PA4<br>0111: 选择PA5<br>1000: 选择PA6<br>1001: 选择PA10<br>1010: 选择PA11<br>1011: 选择PA12<br>1100: 选择PA13<br>1101: 选择PA14<br>1110: 选择PA15<br>1111: 选择PB0 |
| 7    | PND      | r  | 1       | -  | PND: 只读, 中断请求位, 当ADC完成一次转换后, 此位会被设置为‘1’, 需由软件清‘0’                                                                                                                                                                                                           |
| 6    | CPND     | w  | x       | -  | CPND: 只写, 写‘1’清除中断请求位, 写‘0’无效                                                                                                                                                                                                                               |
| 5    | ADC_IE   | rw | 0       | -  | ADC_IE: ADC中断允许                                                                                                                                                                                                                                             |
| 4    | ADC_EN   | rw | 0       | -  | ADC_EN: ADC控制器使能                                                                                                                                                                                                                                            |
| 3    | ADC_AE   | rw | 0       | -  | ADC_AE: ADC 模拟模块常使能                                                                                                                                                                                                                                         |
| 2-0  | ADC_BAUD | rw | 0x0     | -  | ADC_BAUD: ADC时钟频率选择<br>000: LSB时钟1分频<br>001: LSB时钟6分频<br>010: LSB时钟12分频<br>011: LSB时钟24分频<br>100: LSB时钟48分频<br>101: LSB时钟72分频<br>110: LSB时钟96分频<br>111: LSB时钟128分频                                                                                          |

## 8.2.2 ADC\_RES: ADC result register

| Bit   | Name     | RW | Default | RV | Description |
|-------|----------|----|---------|----|-------------|
| 31-10 | RESERVED | -  | -       | -  | 预留          |
| 9-0   | RES      | r  | x       | -  | ADC结果       |

## 9 CRC

### 9.1 概述

CRC16计算器，每次运算8bit，多项式为： $X^{16} + X^{12} + X^5 + X^1$ 。

CRC模块的原理框图如下所示：

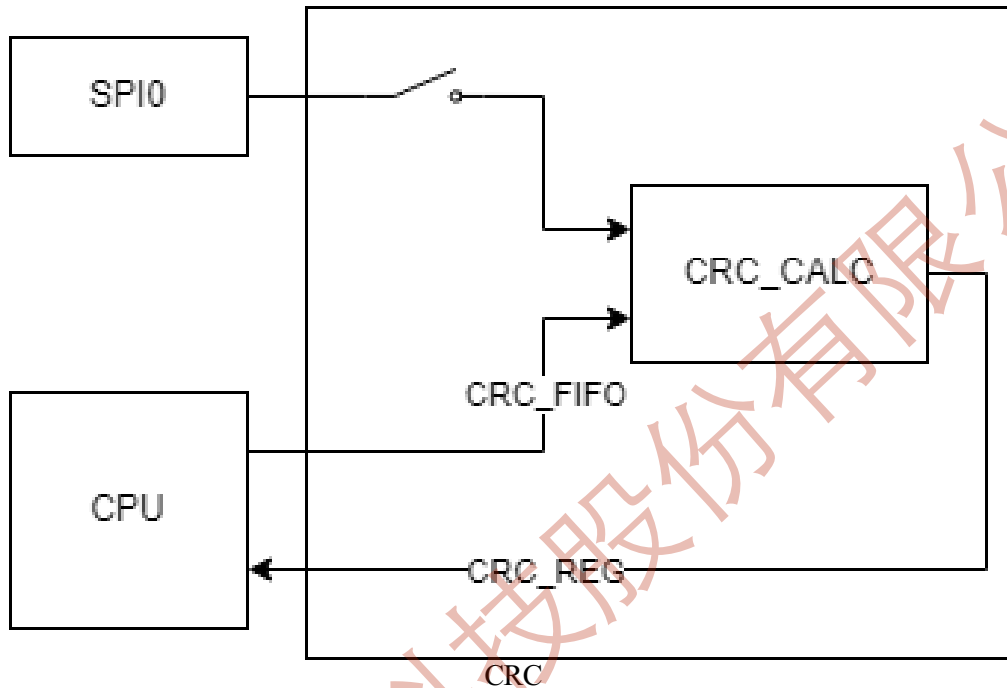


图1、CRC原理框图

当SPI0在DMA状态下计算CRC时，应避免使用CPU操作CRC\_FIFO

注意：该项目CRC模块被集成到SPI0中，与SPI0共用基地址

### 9.2 寄存器说明

#### 9.2.1 REG: CRC16 校验码

| Bit   | Name     | RW | Default | RV | Description         |
|-------|----------|----|---------|----|---------------------|
| 31-16 | RESERVED | -  | -       | -  | 预留                  |
| 15-0  | CRC_REG  | rw | x       | -  | 写入初始值，CRC计算完毕，读取校验码 |

#### 9.2.2 FIFO: 运算数据输入

| Bit  | Name     | RW | Default | RV | Description |
|------|----------|----|---------|----|-------------|
| 31-8 | RESERVED | -  | -       | -  | 预留          |

| Bit | Name     | RW | Default | RV | Description          |
|-----|----------|----|---------|----|----------------------|
| 7-0 | CRC_FIFO | w  | x       | -  | 运算数据输入，HSB先运算，LSB后运算 |

珠海市杰理科技股份有限公司

## 10 GPCNT

### 10.1 概述

GPCNT为时钟脉冲计数器，用于计算两个时钟周期的比例，即用一个已知时钟（主时钟）计算另一个时钟（次时钟）的周期。

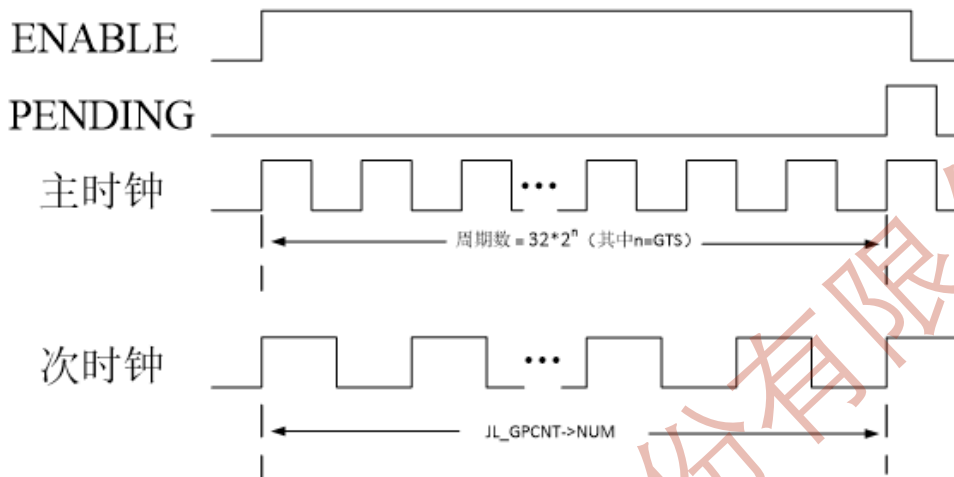


图 1 GPCNT时钟计算示意图

### 10.2 寄存器说明

#### 10.2.1 JL\_GPCNT->CON: GPCNT configuration register

| Bit   | Name     | RW | Default | RV | Description                                                                                                  |
|-------|----------|----|---------|----|--------------------------------------------------------------------------------------------------------------|
| 31    | PND      | r  | 0       | -  | 中断请求标志（当JL_GPCNT->CON[0]置1后，主时钟达到JL_GPCNT->CON[11:8]设定的周期数后，PENDING置1，并请求中断）：<br>0: 无PENDING;<br>1: 有PENDING |
| 30    | CLR_PND  | w  | x       | -  | 清除中断请求标志位：<br>0: 无效;<br>1: 清PENDING                                                                          |
| 29-20 | RESERVED | -  | -       | -  | 预留                                                                                                           |
| 19-16 | GTS      | rw | 0x0     | -  | 主时钟周期数选择：主时钟周期数 = $32 * 2^n$ （其中n=GTS）                                                                       |
| 15-13 | RESERVED | -  | -       | -  | 预留                                                                                                           |
| 12-8  | GSS      | rw | 0x0     | -  | 主时钟选择（计数时钟）：参见Clock_System文档                                                                                 |
| 7-6   | RESERVED | -  | -       | -  | 预留                                                                                                           |

| Bit | Name   | RW | Default | RV | Description                     |
|-----|--------|----|---------|----|---------------------------------|
| 5-1 | CSS    | rw | 0x0     | -  | 次时钟选择（被计数时钟）：参见Clock_System 文档  |
| 0   | ENABLE | rw | 0       | -  | GPCNT模块使能位：<br>0: 不使能；<br>1: 使能 |

【注意】：需配置好其他位，才将ENABLE置1。

### 10.2.2 JL\_GPCNT->NUM: The number of GPCNT clock cycle register

| Bit  | Name | RW | Default | RV | Description                                      |
|------|------|----|---------|----|--------------------------------------------------|
| 31-0 | NUM  | r  | 0x0     | -  | 在JL_GPCNT->CON[0]置1到 PENDING置1（中断到来）之间，次时钟跑的周期数。 |

## 11 IIC

### 11.1 概述

IIC接口是一个可兼容大部分IIC总线标准的串行通讯接口。在上面传输的数据以Byte（8bit）为最小单位，且永远是MSB在前。

- IIC接口主要支持特性：(1). 主模式和从模式，不支持多机功能；(2). 支持标准速度模式（Standard-mode, Sm, 高达100kHz），快速模式（Fast-mode, Fm, 高达400kHz），快速增强模式（Fast-mode Plus, Fm+, 高达1MHz）；(3). 7bit主从机寻址模式，可配置从机地址应答，广播地址响应；(4). 总线时钟延展功能可配置；(5). 总线数据建立时间和保持时间可配置；(6). 基于便捷的事务操作驱动总线；(7). 数据接收发送各独立具有1字节缓冲；(8). 可配置总线信号数字滤波器；
- 主机：(1). IIC接口SCL时钟由本机产生，提供给片外IIC设备使用；(2). IIC接口SCL时钟可配置，SCL时钟周期为：

$$TIIC\_BUS\_SCL = TIIC\_clk / (2 * BUAD\_CNT) + T_{电阻上拉}$$

- 从机：(1). IIC接口SCL时钟由片外IIC设备产生，经内部IIC\_clk滤波采样后给本机使用；(2). IIC总线上每一个start/restart位后接从机地址，此时当外部IIC设备所发的地址与我们匹配时（开启广播地址响应，在接收到广播地址时也会响应），可配置硬件自动回应（ack位为“0”）；

### 11.2 特殊注意点

**时钟要求：**作为主机时，BAUD、TSU、THD、FLT\_SEL值设置应当满足以下关系

$$BAUD\_CNT > (SETUP\_CNT + HOLD\_CNT + FLT\_SEL + 5)$$

作为从机时，SCL时钟低电平时间与TSU、THD、FLT\_SEL值设置应当满足以下关系

$$TSCL\_L > (SETUP\_CNT + HOLD\_CNT + FLT\_SEL + 5) * TI2C\_clk$$

**IO设置：**在选择IO作为IIC接口的SCL、SDA输入输出时，必须配置该两个IO的SPL寄存器位为1；

**事务寄存器：**写入事务后需等待硬件将写入的事务加载后（tx\_task\_load/rx\_task\_load置位即表示对应事务被硬件加载，task\_done pnd置位即表示事务已被硬件加载且执行完成）才可写入新的事务；

**【注意】：**IIC的通信频率不仅与所配置的波特率有关，还会受到IIC总线上拉时间的影响，请尽可能符合IIC总线硬件电气要求。

### 11.3 数字模块控制寄存器

#### 11.3.1 IIC\_CON0: iic control register 0

| Bit   | Name          | RW | Default | RV | Description                                 |
|-------|---------------|----|---------|----|---------------------------------------------|
| 31-10 | RESERVED      | -  | -       | -  | 预留                                          |
| 10    | BADDR_RESP_EN | rw | 0       | -  | 广播地址响应使能：<br>1：响应总线上的广播地址；<br>0：不响应总线上的广播地址 |

| Bit | Name          | RW | Default | RV | Description                                                                                                                                                                                      |
|-----|---------------|----|---------|----|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 9   | AUTO_SLV_TX   | rw | 0       | -  | 从机自动发送数据事务使能（主机模式下无效）：<br>0：不使能；<br>1：使能 从机接收匹配的地址且为从机接收数据时，接下来硬件会自动填充SEND_DATA事务操作，使得硬件能不间断的发送数据；<br>（作为从机与不支持时钟延展的主机通信时必须使能，其他工作场景依据需求使能）                                                      |
| 8   | AUTO_SLV_RX   | rw | 0       | -  | 从机自动接收数据事务使能（主机模式下无效）：<br>0：不使能； 1：使能 从机接收匹配的地址且为从机接收数据时，接下来硬件会自动填充RECV_DATA事务操作，使得在软件读取数据时，硬件接口能继续不间断接收数据；（作为从机与不支持时钟延展的主机通信时必须使能，其他工作场景依据需求使能）                                                 |
| 7   | AUTO_ADR_RESP | rw | 0       | -  | 从机接收地址硬件自动响应使能：作为从机时，在接收到地址数据后（接收的第一字节数据）<br>0：不自动进行响应，即不对接收到的地址数据进行NACK/ACK，需cpu读取接收数据后，再执行NACK/ACK事务；<br>1：自动进行响应，接收到匹配的地址（含广播地址）自动响应ACK；接收到不匹配的地址，自动响应NACK（从机与不支持时钟延展的主机通信时必须使能，其他工作场景依据需求使能） |
| 6   | IGNORE_NACK   | rw | 0       | -  | NACK调试模式使能<br>0：在发送数据中遇到NACK后会停止接收发送；<br>1：在发送数据中遇到NACK后不会中断接收发送事务（仅用于调试）；                                                                                                                       |
| 5   | NO_STRETCH    | rw | 0       | -  | IIC主从时钟延展功能选择<br>0：支持时钟延展；<br>1：不支持时钟延展                                                                                                                                                          |
| 4-3 | FLT_SEL       | rw | 0       | -  | IIC接口数字滤波器选择，<br>无特殊情况软件配置值为2'b10<br>2'b11：滤除3 * Tiic_baud_clk 以下尖峰脉宽；<br>2'b10：滤除2 * Tiic_baud_clk 以下尖峰脉宽；<br>2'b01：滤除Tiic_baud_clk 以下尖峰脉宽；<br>2'b00：关闭数字滤波器                                    |
| 2   | SLAVE MODE    | rw | 0       | -  | IIC接口主从机模式：<br>0：主机模式；<br>1：从机模式                                                                                                                                                                 |
| 1   | I2C_RST       | rw | 0       | -  | IIC接口复位（使能后再释放复位）<br>0：IIC接口复位；<br>1：IIC接口释放                                                                                                                                                     |



| Bit | Name | RW | Default | RV | Description                          |
|-----|------|----|---------|----|--------------------------------------|
| 0   | EN   | rw | 0       | -  | IIC接口使能。<br>0: 关闭IIC接口<br>1: 打开IIC接口 |

### 11.3.2 IIC\_TASK: iic task register

| Bit  | Name          | RW | Default | RV | Description                                                                                                                                                                                                                                          |
|------|---------------|----|---------|----|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 31-4 | RESERVED      | -  | -       | -  | 预留                                                                                                                                                                                                                                                   |
| 12   | SLAVE_CALL    | r  | -       | -  | 从机地址匹配（仅debug，用户正常流程不可使用）                                                                                                                                                                                                                            |
| 11   | BC_CALL       | r  | -       | -  | 广播地址匹配（仅debug，用户正常流程不可使用）                                                                                                                                                                                                                            |
| 10   | SLAVE_RW      | r  | -       | -  | 从机读写状态（仅debug，用户正常流程不可使用）                                                                                                                                                                                                                            |
| 9-6  | RUNNING_TASK  | r  | -       | -  | 正在运行事务（仅debug，用户正常流程不可使用）                                                                                                                                                                                                                            |
| 5    | TASK_RUN_RDY  | r  | -       | -  | 事务运行状态（仅debug，用户正常流程不可使用）                                                                                                                                                                                                                            |
| 4    | TASK_LOAD_RDY | r  | -       | -  | 事务加载状态（仅debug，用户正常流程不可使用）                                                                                                                                                                                                                            |
| 3-0  | RUN_TASK      | w  | 0x0     | -  | 事务操作：以下9种事务操作涵盖了主从机接收发送时的总线行为序列（具体解析见下节）<br>0x0: SEND_RESET<br>0x1: SEND_ADDR<br>0x2: SEND_DATA<br>0x3: SEND_ACK<br>0x4: SEND_NACK<br>0x5: SEND_STOP<br>0x6: SEND_NACK_STOP<br>0x7: RECV_DATA<br>0x8: RECV_DATA_WITH_ACK<br>0x9: RECV_DATA_WITH_NACK |

### 11.3.3 IIC\_PND: iic pending register

| Bit   | Name            | RW | Default | RV | Description           |
|-------|-----------------|----|---------|----|-----------------------|
| 31-29 | RESERVED        | -  | -       | -  | 预留                    |
| 28    | RXTASK_LOAD_PND | r  | 0x0     | -  | 硬件将当前的接收相关的task事务加载完成 |
| 27    | TXTASK_LOAD_PND | r  | 0x0     | -  | 硬件将当前的发送相关的task事务加载完成 |
| 26    | RXBYTE_DONE_PND | r  | 0x0     | -  | 接收到1byte的数据           |

| Bit | Name            | RW | Default | RV | Description                                          |
|-----|-----------------|----|---------|----|------------------------------------------------------|
| 25  | ADR_MATCH_PND   | r  | 0x0     | -  | 作为从机接收到与ADDR相符的地址<br>(开始广播地址响应使能后,<br>匹配广播地址也会起该PND) |
| 24  | RXNACK_PND      | r  | 0x0     | -  | 发送地址/数据后接收到NACK                                      |
| 23  | RXACK_PND       | r  | 0x0     | -  | 发送地址/数据后接收到ACK                                       |
| 22  | STOP_PND        | r  | 0x0     | -  | 接收到总线STOP信号序列                                        |
| 21  | RESTART_PND     | r  | 0x0     | -  | 接收到总线RESTART信号序列                                     |
| 20  | TASK_PND        | r  | 0x0     | -  | 事务执行完成PND                                            |
| 19  | RESERVED        | -  | -       | -  | 预留                                                   |
| 18  | RXTASK_LOAD_CLR | w  | -       | -  | 写“1”清除对应的PND                                         |
| 17  | TXTASK_LOAD_CLR | w  | -       | -  | 写“1”清除对应的PND                                         |
| 16  | RXDATA_DONE_CLR | w  | -       | -  | 写“1”清除对应的PND                                         |
| 15  | ADR_MATCH_CLR   | w  | -       | -  | 写“1”清除对应的PND                                         |
| 14  | RXNACK_CLR      | w  | -       | -  | 写“1”清除对应的PND                                         |
| 13  | RXACK_CLR       | w  | -       | -  | 写“1”清除对应的PND                                         |
| 12  | STOP_CLR        | w  | -       | -  | 写“1”清除对应的PND                                         |
| 11  | RESTART_CLR     | w  | -       | -  | 写“1”清除对应的PND                                         |
| 10  | TASK_CLR        | w  | -       | -  | 写“1”清除对应的PND                                         |
| 9   | RESERVED        | -  | -       | -  | 预留                                                   |
| 8   | RXTASK_LOAD_IE  | rw | 0       |    | 对应PND的中断使能                                           |
| 7   | TXTASK_LOAD_IE  | rw | 0       |    | 对应PND的中断使能                                           |
| 6   | RXDATA_DONE_IE  | rw | 0       |    | 对应PND的中断使能                                           |
| 5   | ADR_MATCH_IE    | rw | 0       |    | 对应PND的中断使能                                           |
| 4   | RXNACK_IE       | rw | 0       |    | 对应PND的中断使能                                           |
| 3   | RXACK_IE        | rw | 0       | -  | 对应PND的中断使能                                           |
| 2   | STOP_IE         | rw | 0       | -  | 对应PND的中断使能                                           |
| 1   | RESTART_IE      | rw | 0       | -  | 对应PND的中断使能                                           |
| 0   | TASK_IE         | rw | 0       | -  | 对应PND的中断使能                                           |

#### 11.3.4 IIC\_TXBUF: iic tx buff register

| Bit  | Name     | RW | Default | RV | Description |
|------|----------|----|---------|----|-------------|
| 31-8 | RESERVED | -  | -       | -  | -           |
| 7-0  | TX_BUF   | rw | -       | -  | 待发送数据       |

#### 11.3.5 IIC\_RXBUF: iic rx buff register

| Bit  | Name     | RW | Default | RV | Description |
|------|----------|----|---------|----|-------------|
| 31-8 | RESERVED | -  | -       | -  | -           |
| 7-0  | RX_BUF   | r  | -       | -  | 已接收数据       |

#### 11.3.6 IIC\_ADDR: iic device address register

| Bit  | Name            | RW | Default | RV | Description                                 |
|------|-----------------|----|---------|----|---------------------------------------------|
| 31-8 | RESERVED        | -  | -       | -  | -                                           |
| 7-1  | IIC DEVICE ADDR | rw | 0x00    | -  | IIC 设备地址（仅支持7bit模式）                         |
| 0    | W/R OPTION      | rw | 0x0     | -  | 作为主机发送地址时的读写标志：<br>0：发送数据到从机；<br>1：读取从机的数据； |

#### 11.3.7 IIC\_BAUD: iic baud clk register

| Bit   | Name     | RW | Default | RV | Description                                                                 |
|-------|----------|----|---------|----|-----------------------------------------------------------------------------|
| 31-12 | RESERVED | -  | -       | -  | -                                                                           |
| 11-0  | BAUD_CNT | rw | -       | -  | IIC 总线传输时钟SCL速率配置：<br>$FIIC\_BUS\_SCL = FIIC\_clk / BUAD\_CNT$<br>注意：不可设置为0 |

#### 11.3.8 IIC\_TSU: iic setup time register

| Bit  | Name     | RW | Default | RV | Description |
|------|----------|----|---------|----|-------------|
| 31-7 | RESERVED | -  | -       | -  | -           |

| Bit | Name      | RW | Default | RV | Description                                                                                   |
|-----|-----------|----|---------|----|-----------------------------------------------------------------------------------------------|
| 5-0 | SETUP_CNT | rw | -       | -  | IIC<br>总线SDA信号更新后到SCL时钟上升的最少时间配置：<br>$T_{setup} = T_{IIC\_clk} * SETUP\_CNT$<br>无特殊情况软件配置值为2； |

### 11.3.9 IIC\_THD: iic hold time register

| Bit  | Name     | RW | Default | RV | Description                                                                              |
|------|----------|----|---------|----|------------------------------------------------------------------------------------------|
| 31-7 | RESERVED | -  | -       | -  | -                                                                                        |
| 5-0  | HOLD_CNT | rw | -       | -  | IIC 总线SCL时钟下降到SDA信号更新前的最少时间配置：<br>$T_{hold} = T_{IIC\_clk} * HOLD\_CNT$<br>无特殊情况软件配置值为2； |

### 11.3.10 IIC\_DBG: iic debug register

| Bit   | Name    | RW | Default | RV | Description |
|-------|---------|----|---------|----|-------------|
| 31-16 | SDA_DBG | r  | -       | -  | SDA相关调试信号   |
| 15-0  | SCL_DBG | r  | -       | -  | SCL相关调试信号   |

## 11.4 基本事务操作

IIC\_TASK 事务寄存器允许写入以下9种事务操作值来驱动IIC接口做出相应总线行为序列：

#### 1. 0x0: SEND\_RESET

工作在主机模式下，向IIC总线发送START 和 STOP；

#### 2. 0x1: SEND\_ADDR

工作在主机模式下，向IIC总线发送START 和 ADDR的数据，并接收从机回复的ACK/NACK；

#### 3. 0x2: SEND\_DATA

工作在主机模式下，向IIC总线发送TX BUFF的数据，并接收从机回复的ACK/NACK；

#### 4. 0x3: SEND\_ACK

工作在主/从机模式下，向IIC总线发送ACK；

#### 5. 0x4: SEND\_NACK

工作在主/从机模式下，向IIC总线发送NACK；

#### 6. 0x5: SEND\_STOP

工作在主机模式下，向IIC总线发送STOP；

#### 7. 0x6: SEND\_NACK\_STOP

工作在主机模式下，向IIC总线发送NACK和STOP；

#### 8. 0x7: RECV\_DATA

工作在主机/从机模式下，从IIC总线上接收一个字节数据，但不进行ACK/NACK（持续拉低SCL线，总线上的设备需支持时钟延展），直到SEND\_ACK/SEND\_NACK的事务被填充，SCL才会撤销拉低；

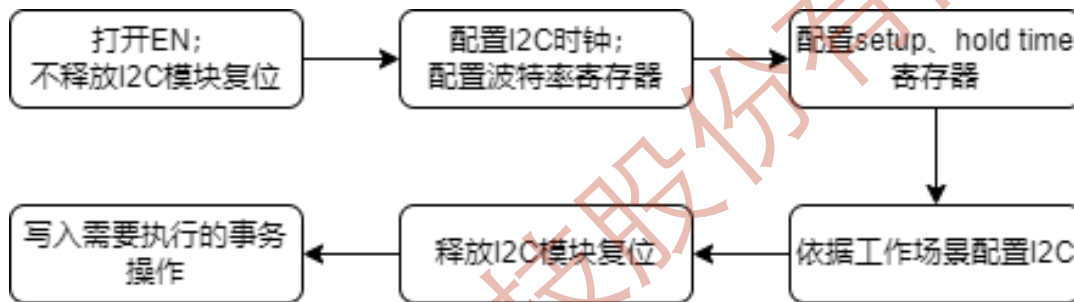
#### 9. 0x8: RECV\_DATA\_WITH\_ACK

工作在主机/从机模式下，从IIC总线上接收一个字节数据，且随后向IIC总线发送ACK；

#### 10. 0x9: RECV\_DATA\_WITH\_NACK

工作在主机/从机模式下，从IIC总线上接收一个字节数据，且随后向IIC总线发送NACK；

### 11.5 基本工作场景使用流程



#### 1. 主机发送

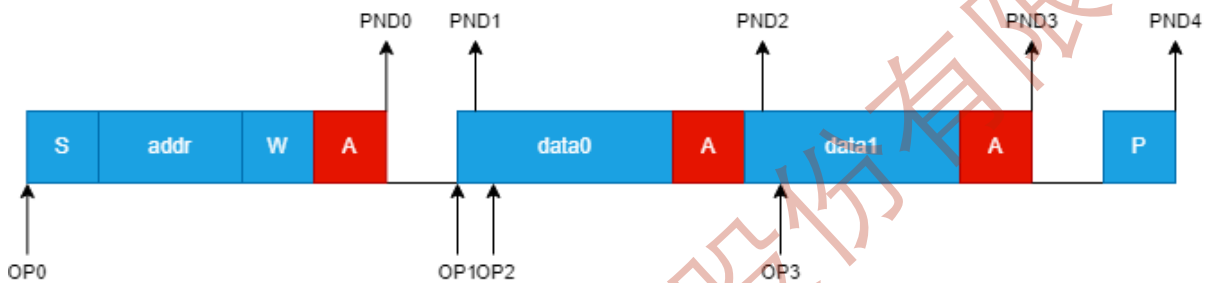
主机发送具有字节断续和字节连续发送两种工作场景。

字节连续发送：用于传输数据量较大的（中途无需进行数据处理）场景，主机在发送地址并且匹配后通过txtask\_pnd来预先（提前约8个I2C\_SCL时钟周期）填充下一事务达到连续发送。



OP0: ADDR = addr + w  
TASK = SEND\_ADDR  
OP1: TX\_BUF = data0  
TASK = SEND\_DATA  
OP2: TX\_BUF = data1  
TASK = SEND\_DATA  
OP3: TASK = SEND\_STOP

PND0: task\_done(发送 addr, 接收 ack)  
PND1: task\_done(发送 data0, 接收 ack)  
PND2: task\_done(发送 data1, 接收 ack)  
PND3: task\_done(发送 stop)

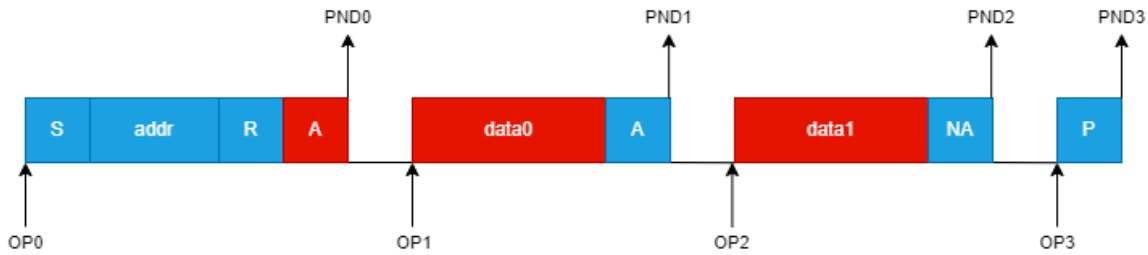


OP0: ADDR = addr + w  
TASK = SEND\_ADDR  
OP1: TX\_BUF = data0  
TASK = SEND\_DATA  
OP2: TX\_BUF = data1  
TASK = SEND\_DATA  
OP3: TASK = SEND\_STOP

PND0: task\_done(发送 addr, 接收 ack)  
PND1: txtask\_load(SEND\_DATA task loaded)  
PND2: txtask\_load(SEND\_DATA task loaded)  
PND3: task\_done(send data1 done, 接收到ack / nack)  
PND3: task\_done(send stop done)

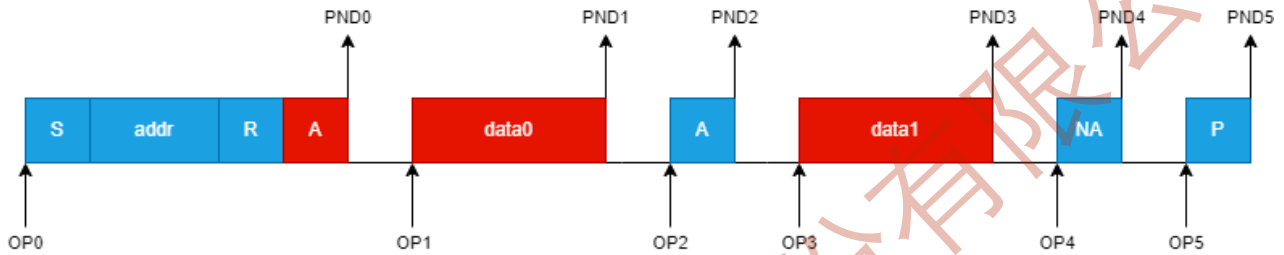
## 2. 主机接收

与主机发送相似，主机发送同样具有字节断续和字节连续发送两种工作场景；



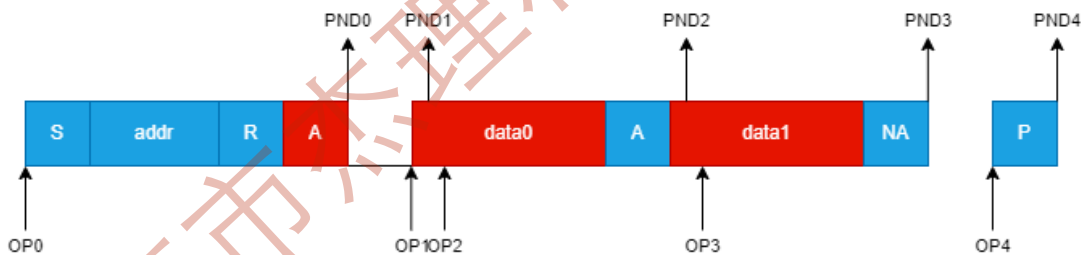
OP0: ADDR = addr + r  
TASK = SEND\_ADDR  
OP1: TASK = RECV\_DATA\_WITH\_ACK  
OP2: buf[0] = RX\_BUF  
TASK = RECV\_DATA\_WITH\_NAK  
OP3: buf[1] = RX\_BUF  
TASK = SEND\_STOP

PND0: task\_done(发送 addr, 接收 ack)  
PND1: task\_done(发送 ack, 接收 data0)  
PND2: task\_done(发送 nack, 接收 data1)  
PND3: task\_done(发送 stop)



OP0: ADDR = addr + r  
TASK = SEND\_ADDR  
OP1: TASK = RECV\_DATA  
OP2: buf[0] = RX\_BUF  
TASK = SEND\_ACK  
OP3: TASK = RECV\_DATA  
OP4: buf[1] = RX\_BUF  
TASK = SEND\_NACK  
OP5: TASK = SEND\_STOP

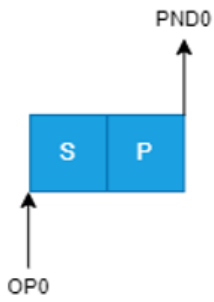
PND0: task\_done(发送 addr, 接收 ack)  
PND1: task\_done(接收 data0)  
PND2: task\_done(发送 ack)  
PND3: task\_done(接收 data1)  
PND4: task\_done(发送 nack)  
PND5: task\_done(发送 stop)



OP0: ADDR = addr + r  
TASK = SEND\_ADDR  
OP1: TASK = RECV\_DATA\_WITH\_ACK  
OP2: TASK = RECV\_DATA\_WITH\_NACK  
OP3: buf[0] = RX\_BUF  
TASK = SEND\_STOP  
OP4: buf[1] = RX\_BUF

PND0: task\_done(发送地址, 接收ack)  
PND1: rxtask\_load(RECV\_DATA task loaded)  
PND2: rxtask\_load(RECV\_DATA task loaded)  
PND3: task\_done(recv data1 done, 发送NACK/ACK)  
PND4: task\_done(发送 stop)

### 3. 主机其他操作（用于复位总线上其他从设备）



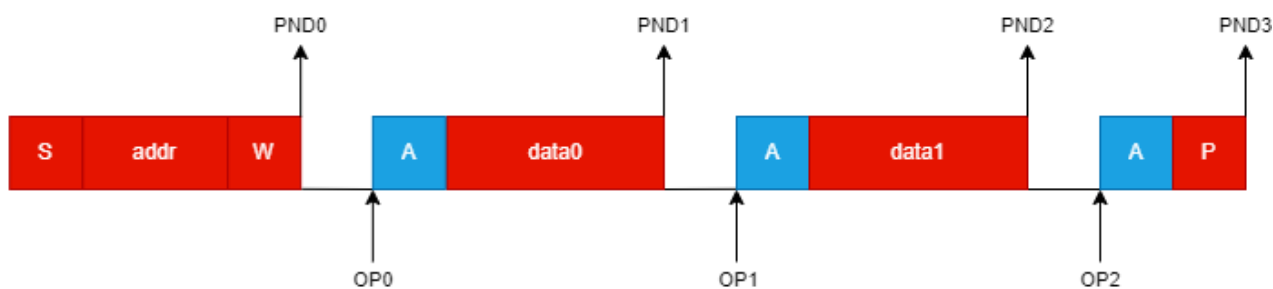
OP0 : **TASK = SEND\_RESET**

PND0 : "start" & "stop" send done

#### 4. 从机接收

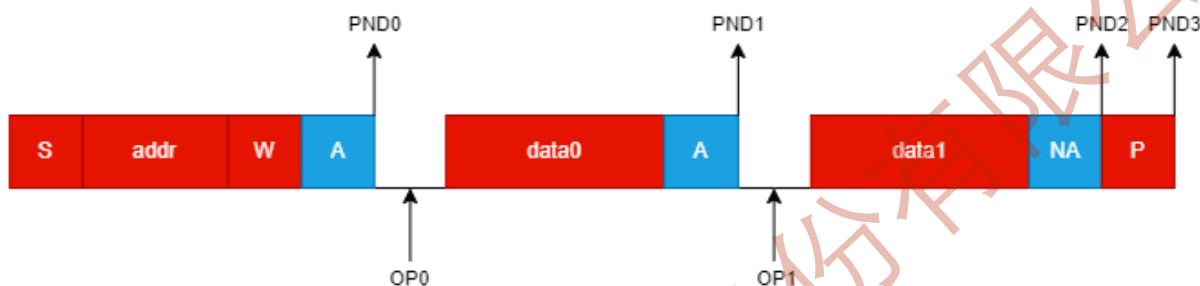
珠海市杰理科技股份有限公司





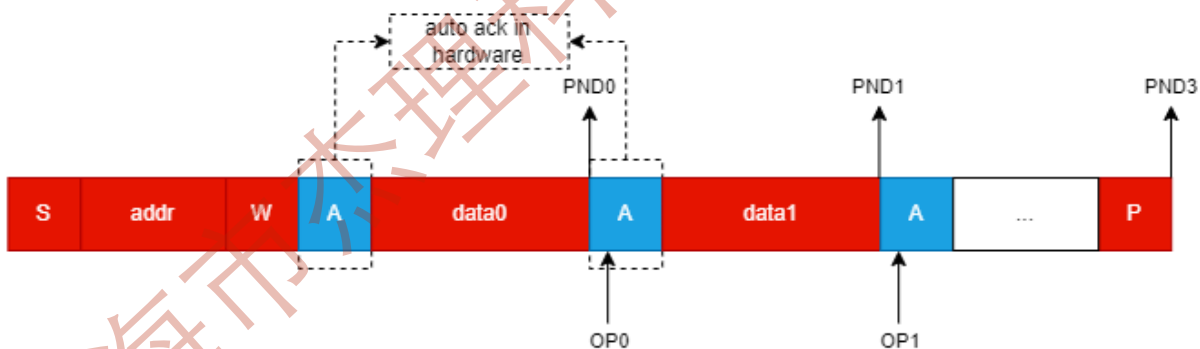
STRETCH\_EN = 1  
AUTO\_ADR\_RESP = 0

OP0 : TASK = RECV\_DATA\_WITH\_ACK PND0 : task\_done("addr" match)  
OP1 : TASK = RECV\_DATA\_WITH\_ACK PND1 : task\_done("data0" receive done)  
OP2 : TASK = RECV\_DATA\_WITH\_ACK / SEND\_NACK PND2 : task\_done("data1" receive done)  
PND3 : task\_done("stop" receive)



STRETCH\_EN = 1  
AUTO\_ADR\_RESP = 1

OP0 : TASK = RECV\_DATA\_WITH\_ACK PND0 : task\_done("addr" match)  
OP1 : TASK = RECV\_DATA\_WITH\_ACK / RECV\_DATA\_WITH\_NACK PND1 : task\_done("data0" receive done)  
PND2 : task\_done("data1" receive done)  
PND3 : task\_done("stop" receive)

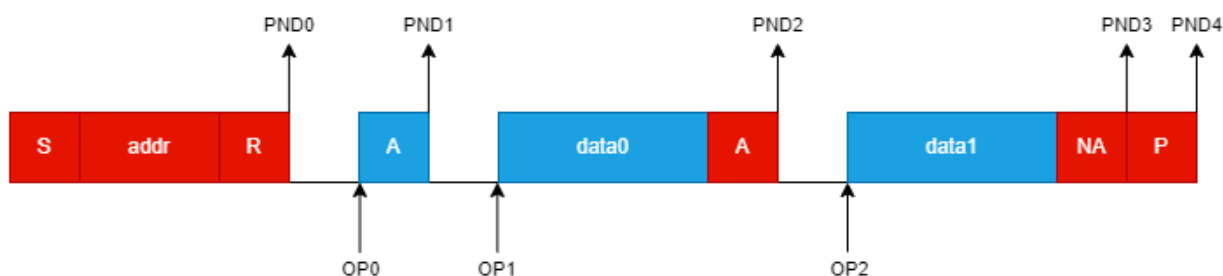


STRETCH\_EN = 0  
AUTO\_ADR\_RESP = 1  
AUTO\_SLV\_RX = 1

OP0 : TASK = SEND\_ACK(for data1)  
OP1 : TASK = SEND\_ACK(for data2)  
...

PND0 : rxbyte\_done("addr" match, "data0" receive)  
PND1 : rxdata\_done("data1" receive)  
...  
PND3 : stop("stop" receive)

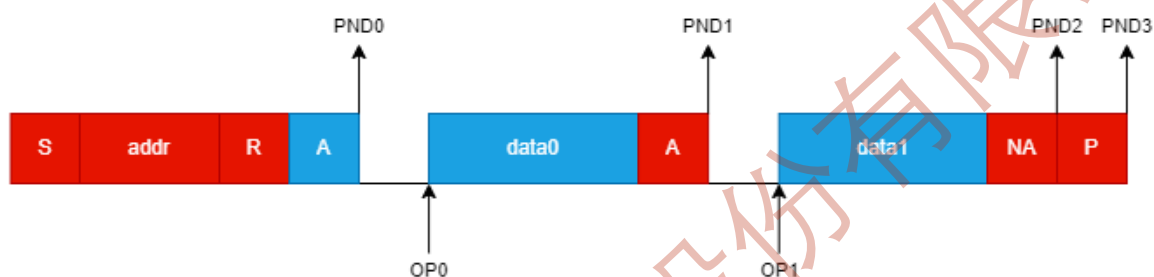
## 5. 从机发送



STRETCH\_EN = 1  
AUTO\_ADR\_RESP = 0

OP0 : TASK = SEND\_ACK  
OP1 : TX\_BUF = data0  
OP2 : TX\_BUF = data1  
TASK = SEND\_DATA  
TASK = SEND\_DATA

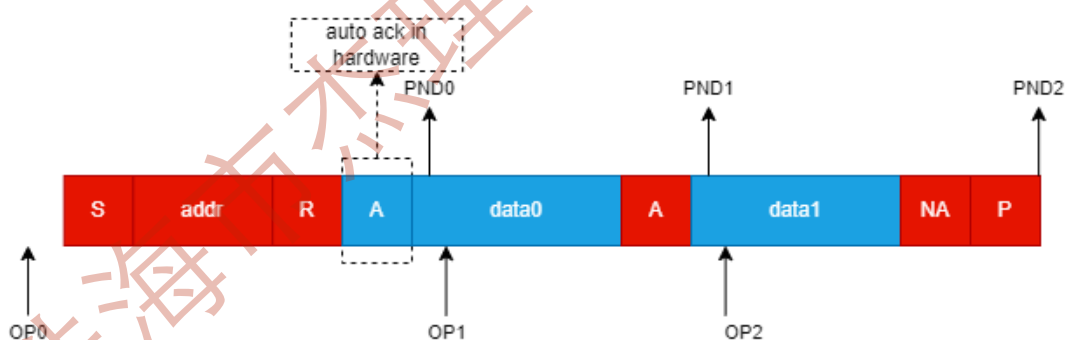
PND0 : task\_done("addr" match)  
PND1 : task\_done("ack" send done)  
PND2 : task\_done("data0" send done , "ack" receive)  
PND3 : task\_done("data1" send done , "nack" receive)  
PND4 : task\_done("stop" receive)



STRETCH\_EN = 1  
AUTO\_ADR\_RESP = 1

OP0 : TX\_BUF = data0  
OP1 : TX\_BUF = data1  
TASK = SEND\_DATA  
TASK = SEND\_DATA

PND0 : task\_done("addr" match)  
PND1 : task\_done("data0" send done , "ack" receive)  
PND2 : task\_done("data1" send done , "nack" receive)  
PND3 : task\_done("stop" receive)



STRETCH\_EN = 0  
AUTO\_ADR\_RESP = 1  
AUTO\_SLV\_TX = 1

OP0 : TX\_BUF = data0  
OP1 : TX\_BUF = data1  
OP2 : TX\_BUF = data2

PND0 : tx load done("addr" match , "data0" tx load done)  
PND1 : tx load done("data1" tx load done , "ack" receive)  
PND2 : stop ("nack" receive, "stop" receive)

## 12 MCPWM

### 12.1 概述

MCPWM功能模块包括：n个timer时基模块，n对独立的PWM通道，n路故障保护输入。框图如下：

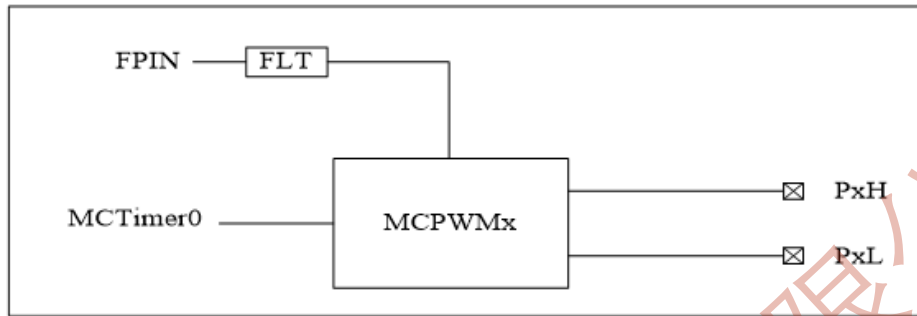


图1-1 MCPWM结构图

### 12.2 定时器 MCTIMER

#### 1. 概述

mctimer是16位定时器，可作为pwm的时基控制

2. 模块特性 (1). 16位定时功能 (2). 带缓冲的周期寄存器 (3). 支持多种工作模式：递增、递增-递减、外部引脚控制递增或递减 (4). 多种中断模式：上溢出中断、下溢出中断、上下溢出中断

3. 注意：MCPWM增加了一BIT (MCPWM CON BIT (16))用于开关MCPWM时钟，防止MCPWM模块不用时还有漏电；使用MCPWM之前需要打开这一BIT。

### 12.3 TIMER模块寄存器

#### 12.3.1 TMRx\_CON: Timer contrl register

| Bit   | Name     | RW | Default | RV | Description                                       |
|-------|----------|----|---------|----|---------------------------------------------------|
| 31-16 | RESERVED | -  | -       | -  | 预留                                                |
| 15    | INCF     | rw | 0       | -  | 递增递减标志位<br>0: 递减<br>1: 递增                         |
| 13    | UFPND    | r  | 0       | -  | 递减借位标志<br>0: 没借位<br>1: 已发生借位                      |
| 12    | OFPN     | r  | 0       | -  | 计数溢出 (TMRXCNT == TMRXPR) 标志<br>0: 没溢出<br>1: 已发生溢出 |

| Bit | Name     | RW | Default | RV | Description                                                   |
|-----|----------|----|---------|----|---------------------------------------------------------------|
| 11  | UFCLR    | w  | -       | 1  | 清除UFPND标志位<br>0: 无效<br>1: 清除UFPND                             |
| 10  | OFCLR    | w  | -       | 1  | 清除OFPND标志位<br>0: 无效<br>1: 清除OFPND                             |
| 9   | UFIE     | rw | 0       | 1  | 定时递减借位中断使能<br>0: 禁止<br>1: 允许                                  |
| 8   | OFIE     | rw | 0       | 1  | 计数溢出中断使能<br>0: 禁止<br>1: 允许                                    |
| 7   | CKSRC    | rw | 0       | -  | 定时器时钟源<br>0: 内部时钟<br>1: 外部引脚时钟                                |
| 6-3 | CKPS     | rw | 0       | -  | 时钟预分频设置, $TCK/(2^{CKPS})$                                     |
| 2   | RESERVED | -  | -       | -  | 预留                                                            |
| 1-0 | MODE     | rw | 0       | -  | 定时器工作模式<br>00: 保持计数值<br>01: 递增模式<br>10: 递增-递减循环模式<br>11: 递减模式 |

### 12.3.2 TMRx\_CNT: counter initial value register

| Bit  | Name    | RW | Default | RV | Description |
|------|---------|----|---------|----|-------------|
| 15-0 | TMR_CNT | rw | x       | -  | 计数/定时初值     |

### 12.3.3 TMR\_PR: counter target value register

| Bit  | Name   | RW | Default | RV | Description |
|------|--------|----|---------|----|-------------|
| 15-0 | TMR_PR | rw | x       | -  | 计数/定值目标值    |

## 12.4 模块引脚

PWM0: PWMCH0H、PWMCH0L

PWM1: PWMCH1H、PWMCH1L

PWM2: PWMCH2H、PWMCH2L

.....

故障输入引脚：FPIN

## 12.5 模块特性

1. 边沿对齐和中心对齐输出模式
2. 可运行过程中更改PWM频率、占空比
3. 多种更新频率/占空比模式：上溢出重载、下溢出重载、上下溢出重载
4. 灵活配置每对通道的有效电平状态
5. 可编程死区控制
6. 硬件故障输入引脚

## 12.6 PWM模块控制寄存器

### 12.6.1 CHx\_CON0: pwm control register0

| Bit   | Name   | RW | Default | RV | Description                                                                                                             |
|-------|--------|----|---------|----|-------------------------------------------------------------------------------------------------------------------------|
| 15-12 | DTCKPS | rw | 0       | -  | 死区时钟预分频, $T_{sys}/(2^{\wedge}DTCKPS)$                                                                                   |
| 11-7  | DTPR   | rw | 0       | -  | 死区时间控制<br>死区时间: $T_{sys}/(2^{\wedge}DTCKPS) \times (DTPR+1)$                                                            |
| 6     | DTEN   | rw | 0       | -  | 死区允许控制, 对应PWMCHxH、PWMCHxL<br>0: 禁止<br>1: 允许                                                                             |
| 5     | L_INV  | rw | 0       | -  | 对应PWMCHXL输出反向控制<br>0: 反向禁止<br>1: 反向允许                                                                                   |
| 4     | H_INV  | rw | 0       | -  | 对应PWMCHXH输出反向控制<br>0: 反向禁止<br>1: 反向允许                                                                                   |
| 3     | L_EN   | rw | 0       | -  | 对应PWMCHXL输出允许控制<br>0: 禁止PWMCHXL<br>1: 允许PWMCHXL                                                                         |
| 2     | H_EN   | rw | 0       | -  | 对应PWMCHxH输出允许控制<br>0: 禁止PWMCHxH<br>1: 允许PWMCHxH                                                                         |
| 1-0   | CMP_LD | rw | 0       | -  | CHx_CMP重新载入控制<br>00: 时基TMRX_CNT等于“0”载入<br>01: 时基TMRX_CNT等于“0”或者等于TMRX_PR时候载入<br>10: 时基TMRX_CNT等于TMRX_PR时候载入<br>11: 立即载入 |

### 12.6.2 CHX\_CON1: pwm control register1

| Bit   | Name     | RW | Default | RV | Description                                                                      |
|-------|----------|----|---------|----|----------------------------------------------------------------------------------|
| 15    | FPND     | r  | 0       | -  | 故障保护输入标志，只读，写无效<br>读0：未发生保护<br>读1：已发生保护，模块的PWM引脚会变成高阻态                           |
| 14    | FCLR     | w  | -       | -  | 清除FPND标志位，只写，读为“0”<br>写0：无效<br>写1：清除FPND                                         |
| 13-12 | RESERVED | -  | -       | -  | 预留                                                                               |
| 11    | INTEN    | rw | 0       | 1  | FPND中断允许<br>0：禁止 1：允许                                                            |
| 10-8  | TMRSEL   | rw | 0       | -  | 选择TMR0-7作为PWM时基<br>0-7：选择时基                                                      |
| 7-5   | RESERVED | -  | -       | -  | 预留                                                                               |
| 4     | FPINEN   | rw | 0       | 1  | 故障保护输入允许控制<br>0：禁止保护<br>1：允许保护                                                   |
| 3     | FPINAUTO | rw | 0       | 1  | 故障自动保护控制<br>0：禁止自动保护<br>1：允许自动保护，<br>当检测到故障引脚有效信号时，<br>自动把PWM引脚设成高阻态，直到软件清除FPND。 |
| 2-0   | FPINSEL  | rw | 0       | -  | 故障保护输入引脚选择<br>0-1：选择FPIN作为故障保护输入<br>(0:选择FPIN0 1:选择FPIN1)                        |

### 12.6.3 CHX\_CMPH: pwm high port compare register

| Bit  | Name     | RW | Default | RV | Description                    |
|------|----------|----|---------|----|--------------------------------|
| 15-0 | MMRX_PRH | rw | -       | -  | 带缓冲的16位比较寄存器，对应PWMCHXH引脚的占空比控制 |

### 12.6.4 CHX\_CMPL: pwm low port compare register

| Bit  | Name     | RW | Default | RV | Description                    |
|------|----------|----|---------|----|--------------------------------|
| 15-0 | TMRX_PRL | rw | -       | -  | 带缓冲的16位比较寄存器，对应PWMCHXL引脚的占空比控制 |

### 12.6.5 FPIN\_CON: input filter control register

| Bit   | Name     | RW | Default | RV | Description                                  |
|-------|----------|----|---------|----|----------------------------------------------|
| 23-16 | EDGE     | rw | 0       | -  | FPINx边沿选择<br>0: 下降沿<br>1: 上升沿                |
| 15-8  | FLT_EN   | rw | 0       | -  | FLT_EN: FPINx滤波使能开关<br>0: 滤波关闭<br>1: 滤波开启    |
| 7-6   | RESERVED | -  | -       | -  | 预留                                           |
| 5-0   | FLT_PR   | rw | 0       | -  | FLT_PR: 滤波宽度选择<br>滤波宽度=16 × FLT_PR × lsb_clk |

#### 12.6.6 MCPWM\_CON: mcpwm control register

| Bit  | Name   | RW | Default | RV | Description                                   |
|------|--------|----|---------|----|-----------------------------------------------|
| 16   | CLK_EN | rw | 0       | -  | mcpwm 模块时钟使能，<br>配置mcpwm模块之前先配该位为1            |
| 15-8 | TMR_EN | rw | 0       | -  | 定时器计数开关控制(tmr7-0)<br>0: 定时器计数关闭<br>1: 定时器计数打开 |
| 7-0  | PWM_EN | rw | 0       | -  | 模块开关控制 (pwm7-0)<br>0: 模块关闭<br>1: 模块开启         |

## 13 GPCRC

### 13.1 概述

GPCRC（General Purpose Cyclic Redundancy Calculation Unit，即通用循环冗余检验计算单元）使用多项式计算一个8位/16位/32位的数据字产生的CRC码。

支持多项式位数：7位、8位、16位、32位。

支持多项式系数编程，或使用默认多项式（CRC-32/MPEG-2）。

支持输入数据位反转、输出数据位反转、输入数据端序翻转。

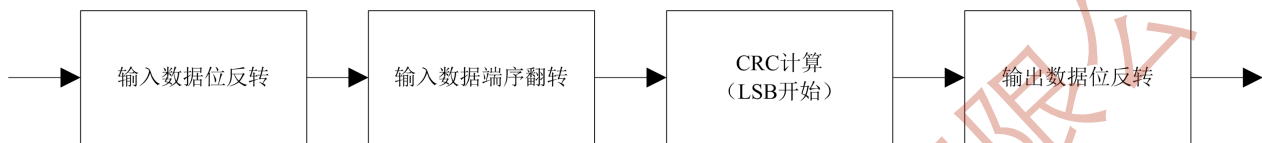


图1-1 GPCRC原理图

### 13.2 功能描述

#### 13.2.1 数据处理

输入数据端序翻转：

指大小端变换，如数据为{Byte3, Byte2, Byte1, Byte0}，半字翻转后为{Byte2, Byte3, Byte0, Byte1}，字翻转后为{Byte0, Byte1, Byte2, Byte3}。

输入数据反转：

指字节按位反转，如数据为{0x01, 0x02, 0x03, 0x04}，反转为{0x80, 0x40, 0xc0, 0x20}。

输出数据反转：

指对应7位、8位、16位、32位多项式按位反转。

#### 13.2.2 多项式配置

| 多项式名称       | 多项式公式                 | REV_IN | REV_OUT | POL        | INIT       |
|-------------|-----------------------|--------|---------|------------|------------|
| CRC-7/MMC   | $x^7+x^3+1$           | 不反转    | 不反转     | 0x00000009 | 0x00000000 |
| CRC-8       | $x^8+x^2+x+1$         | 不反转    | 不反转     | 0x00000007 | 0x00000000 |
| CRC-8/ITU   | $x^8+x^2+x+1$         | 不反转    | 不反转     | 0x00000007 | 0x00000000 |
| CRC-8/ROHC  | $x^8+x^2+x+1$         | 反转     | 反转      | 0x00000007 | 0x000000ff |
| CRC-8/MAXIM | $x^8+x^5+x^4+1$       | 反转     | 反转      | 0x00000031 | 0x00000000 |
| CRC-16/IBM  | $x^{16}+x^{15}+x^2+1$ | 反转     | 反转      | 0x00008005 | 0x00000000 |



| 多项式名称              | 多项式公式                                                                             | REV_IN | REV_OUT | POL        | INIT       |
|--------------------|-----------------------------------------------------------------------------------|--------|---------|------------|------------|
| CRC-16/MAXIM       | $x^{16}+x^{15}+x^2+1$                                                             | 反转     | 反转      | 0x00008005 | 0x00000000 |
| CRC-16/USB         | $x^{16}+x^{15}+x^2+1$                                                             | 反转     | 反转      | 0x00008005 | 0x0000ffff |
| CRC-16/MODBUS      | $x^{16}+x^{15}+x^2+1$                                                             | 反转     | 反转      | 0x00008005 | 0x0000ffff |
| CRC-16/CCITT       | $x^{16}+x^{12}+x^5+1$                                                             | 反转     | 反转      | 0x00001021 | 0x00000000 |
| CRC-16/CCITT-FALSE | $x^{16}+x^{12}+x^5+1$                                                             | 不反转    | 不反转     | 0x00001021 | 0x0000ffff |
| CRC-16/X25         | $x^{16}+x^{12}+x^5+1$                                                             | 反转     | 反转      | 0x00001021 | 0x0000ffff |
| CRC-16/XMODEM      | $x^{16}+x^{12}+x^5+1$                                                             | 不反转    | 不反转     | 0x00001021 | 0x00000000 |
| CRC-16/DNP         | $x^{16}+x^{13}+x^{12}+x^{11}+x^{10}+x^8+x^6+x^5+x^2+1$                            | 反转     | 反转      | 0x00003d65 | 0x00000000 |
| CRC-32             | $x^{32}+x^{26}+x^{23}+x^{22}+x^{16}+x^{12}+x^{11}+x^{10}+x^8+x^7+x^5+x^4+x^2+x+1$ | 反转     | 反转      | 0x04c11db7 | 0xffffffff |
| CRC-32/MPEG-32     | $x^{32}+x^{26}+x^{23}+x^{22}+x^{16}+x^{12}+x^{11}+x^{10}+x^8+x^7+x^5+x^4+x^2+x+1$ | 不反转    | 不反转     | 0x04c11db7 | 0xffffffff |

## 13.3 数字模块控制寄存器

### 13.3.1 GPCRC\_CON: gpcrc control register

| Bit  | Name     | RW | Default | RV | Description                |
|------|----------|----|---------|----|----------------------------|
| 31:9 | RESERVED | -  | -       | -  | 预留                         |
| 8    | REV_IN   | rw | 1       | -  | 输入数据位反转<br>0: 反转<br>1: 不反转 |
| 7    | REV_OUT  | rw | 0       | -  | 输出数据位反转<br>0: 不反转<br>1: 反转 |

| Bit | Name     | RW | Default | RV | Description                                 |
|-----|----------|----|---------|----|---------------------------------------------|
| 6:5 | SWP_IN   | rw | 0x0     | -  | 输入数据端序翻转<br>0/3: 不翻转<br>1: 半字翻转<br>2: 字翻转   |
| 4:3 | POL_SIZE | rw | 0x0     | -  | 多项式位数<br>0: 32位<br>1: 16位<br>2: 8位<br>3: 7位 |
| 2:1 | DAT_SIZE | rw | 0x0     | -  | 输入数据类型<br>0/3: 32位<br>1: 16位<br>2: 8位       |
| 0   | RESERVED | -  | -       | -  | 预留                                          |

### 13.3.2 GPCRC\_POL: gpcrc poly register

| Bit  | Name | RW | Default    | RV | Description     |
|------|------|----|------------|----|-----------------|
| 31:0 | POL  | rw | 0x04c11db7 | -  | CRC多项式系数，第0位恒为1 |

### 13.3.3 GPCRC\_INIT: gpcrc initial register

| Bit  | Name | RW | Default | RV | Description |
|------|------|----|---------|----|-------------|
| 31:0 | INIT | w  | -       | -  | 写入CRC初始数据   |

### 13.3.4 GPCRC\_REG: gpcrc data register

| Bit  | Name | RW | Default    | RV | Description     |
|------|------|----|------------|----|-----------------|
| 31:0 | REG  | rw | 0xffffffff | -  | 写入CRC数据或读出CRC结果 |

## 13.4 DEMO CODE

```
#define gpcrc_con_set(swp_in, rev_out, rev_in, pol_size, dat_size) \
( \
    (rev_in & 0x1) << 8 | \
    (rev_out & 0x1) << 7 | \
    (swp_in & 0x3) << 5 | \
    (pol_in & 0x3) << 3 | \
    (dat_size & 0x3) << 1 | \
)

typedef enum {
    NO_SWP_IN, HWAD_SWP_IN, WORD_SWP_IN
```

```
} SWP_IN_CFG;

typedef enum {
    NO_REV_OUT, REV_OUT
} REV_OUT_CFG;

typedef enum {
    REV_IN, NO_REV_IN
} REV_IN_CFG;

typedef enum {
    POL32, POL16, POL8, POL7
} POL_SIZE;

typedef enum {
    U32, U16, U8
} DAT_SIZE;

u8 crc8_rohc_u8(u8 *data, u32 length)
{
    JL_GPCRC->INIT = 0x000000ff;
    JL_GPCRC->CON = gpcrc_con_set(NO_SWP_IN, REV_OUT, REV_IN, POL8, U8);
    JL_GPCRC->POL = 0x00000007;
    while(length--)
        JL_GPCRC->REG = *data++;
    return JLGPCRC->REG;
}

u8 crc8_rohc_u16(u16 *data, u32 length)
{
    JL_GPCRC->INIT = 0x000000ff;
    JL_GPCRC->CON = gpcrc_con_set(NO_SWP_IN, REV_OUT, REV_IN, POL8, U16);
    JL_GPCRC->POL = 0x00000007;
    while(length--)
        JL_GPCRC->REG = *data++;
    return JLGPCRC->REG;
}

u8 crc8_rohc_u32(u32 *data, u32 length)
{
    JL_GPCRC->INIT = 0x000000ff;
    JL_GPCRC->CON = gpcrc_con_set(NO_SWP_IN, REV_OUT, REV_IN, POL8, U32);
    JL_GPCRC->POL = 0x00000007;
    while(length--)
        JL_GPCRC->REG = *data++;
    return JLGPCRC->REG;
}
```

## 14 PULSE COUNTER

### 14.1 概述

以下为pulse\_counter/触摸键的寄存器说明及使用方法。其工作原理如下所示：

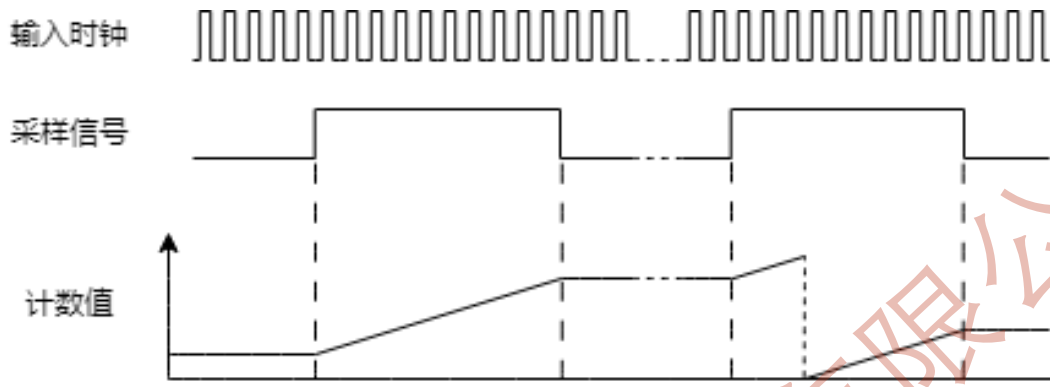


图1、pulse counter计数原理

### 14.2 寄存器说明

#### 14.2.1 PL\_CNT\_CON

| Bit  | Name     | RW | Default | RV | Description                                                                                                                                                                                   |
|------|----------|----|---------|----|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 31-4 | RESERVED | -  | -       | -  | 预留                                                                                                                                                                                            |
| 3-2  | CLK_SEL  | rw | 0x0     | -  | pulse counter 时钟选择<br>00:选择std48m作为计数时钟;<br>01: 选择ich_clk_pin作为计数时钟, 具体可参考IOMC;<br>10: 选择pll_d1p5_mo作为计数时钟;<br>11: 选择pll_d1p0_mo作为计数时钟; 备注: 时钟选择的频率越大, pl_cnt_value计数值会越大, 分辨率会越高, 触摸键的灵敏度也越高 |
| 1    | EN       | rw | 0       | -  | 触摸键使能, 当cap_mux_in为1时, PLCNTVL累加计数, 计满后归0再重新累加<br>0: 触摸键禁止;<br>1: 触摸键使能                                                                                                                       |
| 0    | TEST_EN  | rw | 0       | 0  | 触摸键测试使能 (此位专用于测试)<br>0: 测试模式未使能;<br>1: 测试模式使能;                                                                                                                                                |

【注意】：当cap\_mux\_in选择TMRx\_PWMOUT, pulse counter时钟选择clk\_mux\_in时, 可以用内部固定周期的PWM, 计算芯片IO上不确定的时钟频率。(具体cap\_mux选择参考IOMC文档)

#### 14.2.2 PL\_CNT\_VAL

| Bit | Name | RW | Default | RV | Description |
|-----|------|----|---------|----|-------------|
|-----|------|----|---------|----|-------------|

| Bit  | Name  | RW | Default | RV | Description             |
|------|-------|----|---------|----|-------------------------|
| 31-0 | VALUE | r  | -       | -  | 32位counter递增计数值，默认值为随机数 |

【注意】：芯片上电时Pluse Counter的PL\_CNT\_VAL都会产生一个随机的初始值，因此每次启动Pluse Counter前都需要读取一次

## 14.3 示例代码

以下以PA1为例：

### 1. 变量定义：

```
int Touchkey_value_old, Touchkey_value_new, Touchkey_value_delta; //32bit
```

### 2. 选择检测管脚

```
JL_IMAP->FI_GP_ICH1 = PA1_IN; // PA1选通为ICH1
JL_IOMC->ICH_CONx = (ICH1)<<8; //PLUSE COUNTER 选择使用ICH1作为输入
```

### 3. 初始化计数器配置

```
JL_PCNT->CON = 0; //
JL_PCNT->CON |= (PCCON_CLOCK_3)<<2; //选择触摸键时钟
JL_PCNT->CON |= BIT(1); //使能计数器
```

### 4. 检测电容

```
Touchkey_value_old = JL_PCNT->VAL; //读取旧值
JL_PORTA->DIR |= BIT(1); //对应管脚输入使能
While(PORTA_IN & BIT(1));
Touchkey_value_new = JL_PCNT->VAL;
// 判断是否溢出
if(Touchkey_value_old > Touchkey_value_new)
Touchkey_value_new += 0x10000;
Touchkey_value_delta = Touchkey_value_new - Touchkey_value_old;
```

## 15 RAND64

### 15.1 概述

RAND64是一个产生64位伪随机序列的模块，它由片内RC振荡器驱动，RC振荡器振荡频率的不稳定性进一步确保了生成序列的随机性。

由于片内RC振荡器的振荡频率大约为250KHz，即每个周期4uS，因此RAND64生成的序列约每4uS自动更新一次。若软件有读取RAND64寄存器（RAND64H或RAND64L）的动作，则生成序列将立即更新，以确保每次读取均能获得不同的序列。

### 15.2 控制寄存器

| 寄存器列表   | 宽度    | 读写 | 复位值 |
|---------|-------|----|-----|
| RAND64H | 32bit | 只读 | 不确定 |
| RAND64L | 32bit | 只读 | 不确定 |

## 16 SDC

### 16.1 概述

SD接口是一个标准的遵守SD协议的串行通讯接口，在上面传输的数据一Byte(8bit)为最小单位。

1. SD接口只支持主机模式。
2. SD接口支持1bit data和4bit data模式：
3. 1bit data模式：串行数据通过一根DAT线传输，一个字节数据需8个SD时钟
4. 4bit data模式：串行数据通过四根DAT线传输，一个字节数据需2个SD时钟
5. SD接口支持高速模式和低速模式，最高时钟速度可以达到50MHz。
6. SD接口支持命令DMA和数据DMA，两个DMA时独立的，即可以同时做DMA

### 16.2 数字模块控制寄存器

#### 16.2.1 JL\_SD0->CON0: sd0 control register 0

| Bit  | Name     | RW | Default | RV | Description                                                                                                                          |
|------|----------|----|---------|----|--------------------------------------------------------------------------------------------------------------------------------------|
| 15   | DINT     | r  | 0       | -  | SD数据pending<br>0: 数据未结束<br>1: 数据结束                                                                                                   |
| 14   | CLR_DINT | w  | 0       | 1  | 清空SD数据pending（读为0）<br>0: N/A<br>1: 清空                                                                                                |
| 13   | LINE4    | rw | 0       | -  | 4线模式控制位<br>0: 1线模式<br>1: 4线模式                                                                                                        |
| 12   | DATCLK8  | rw | 0       | -  | SD发送/接收数据时时钟延长8个CLK<br>0: 延长关闭<br>1: 延长打开                                                                                            |
| 11   | DATCRC   | r  | 0       | -  | 接收数据的CRC标志位<br>0: 接收数据的响应CRC正常<br>1: 接收数据出错，CRC错误                                                                                    |
| 10-8 | SDDMODE  | w  | 0x0     | -  | SD数据模式（kickstart）<br>000: no operation<br>100: 发送数据，不检查“busy”位<br>101: 发送数据，检查“busy”位<br>110: 接收数据，不检查“busy”位<br>111: 接收数据，检查“busy”位 |

| Bit | Name     | RW | Default | RV | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|-----|----------|----|---------|----|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 7   | CINT     | r  | 0       | -  | SD命令pending<br>0: 命令未结束<br>1: 命令结束                                                                                                                                                                                                                                                                                                                                                                                                                                             |
| 6   | CLR_CINT | w  | 0       | -  | 清空SD命令pending (读为0)<br>0: N/A<br>1: 清空                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| 5   | TIMEOUT  | r  | 0       | -  | 命令响应超时标志位<br>0: 命令响应正常<br>1: 命令响应超时                                                                                                                                                                                                                                                                                                                                                                                                                                            |
| 4   | CMDCLK8  | rw | 0       | -  | SD发送/接收命令时钟延长8个CLK<br>0: 延长关闭<br>1: 延长打开                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| 3   | CMDCRC   | r  | 0       | -  | CMDCRC: 接收响应的CRC标志位<br>0: 接收响应正确, CRC正确<br>1: 接收响应出错, CRC错误                                                                                                                                                                                                                                                                                                                                                                                                                    |
| 2-0 | SDCMODE  | w  | 0x0     | -  | SD命令模式(kickstart)<br>000: no operation<br>001: send command, receive 6-byte response, with check busy<br>010: send command, receive 17-byte response, with check busy<br>011: send command, without receive response, with check busy<br>101: send command, receive 6-byte response, without check busy<br>110: send command, receive 17-byte response, without check busy<br>111: send command, without receive response, without check busy<br>上面后六种命令模式设置后, sdc控制器会进入命令收发状态 |

### 16.2.2 JL\_SD0->CON1: sd0 control register 1

| Bit  | Name    | RW | Default | RV | Description                                     |
|------|---------|----|---------|----|-------------------------------------------------|
| 15-8 | SD_BAUD | rw | 0x0     | -  | SD_BAUD: SD波特率<br>SD波特率= $F_{sys}/(SD\_BAUD+1)$ |
| 7    | DATIE   | rw | 0       | -  | DATIE: SD数据中断允许<br>0: 不允许<br>1: 允许              |
| 6    | CMDIE   | rw | 0       | -  | CMDIE: SD命令中断允许<br>0: 不允许<br>1: 允许              |



| Bit | Name     | RW | Default | RV | Description                                        |
|-----|----------|----|---------|----|----------------------------------------------------|
| 5   | CLKALL   | rw | 0       | -  | CLKALL: SD时钟开关<br>0: SD时钟在不传送命令和数据时关闭<br>1: SD时钟常开 |
| 4   | RESERVED | -  | -       | -  | 预留                                                 |
| 3-1 | CRCSTA   | r  | 0x0     | -  | CRCSTA: CRS State 详细请查阅SD spec                     |
| 0   | SDCEN    | rw | 0       | -  | SDCEN: SD控制器允许位<br>0: SD控制器关闭<br>1: SD控制器打开        |

### 16.2.3 JL\_SD0->CON2: sd0 control register 2

| Bit  | Name     | RW | Default | RV | Description                                                                     |
|------|----------|----|---------|----|---------------------------------------------------------------------------------|
| 15-9 | RESERVED | -  | -       | -  | 预留                                                                              |
| 8-0  | SDDCNT   | rw | 0x0     | -  | SDDCNT: 数据读写数目<br>000: 1byte<br>001: 2byte<br>002: 3byte<br>...<br>1ff: 512byte |

### 16.2.4 JL\_SD0->CTU\_CON: sd0 continue control register

| Bit   | Name         | RW | Default | RV | Description                                                                                                                                                                                           |
|-------|--------------|----|---------|----|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-11 | RESERVED     | -  | -       | -  | 预留                                                                                                                                                                                                    |
| 10-8  | SDDMODE_KEEP | rw | 0x0     | -  | SDDMODE_KEEP:<br>SD连续数据读写模式<br>(连续模式开始后多次写进SDDMODE, 不是kickstart, kickstart是写SD_CTU_CNT)<br>000: N/A<br>100: 发送数据, 不检查“busy”位<br>101: 发送数据, 检查“busy”位<br>110: 接收数据, 不检查“busy”位<br>111: 接收数据, 检查“busy”位 |
| 7     | CTU_PND      | r  | 0       | -  | CTU_PND:<br>SD连续读写结束标志<br>0: 未结束<br>1: 结束                                                                                                                                                             |
| 6     | CLR_CTU_PND  | w  | 0       | -  | CLR_CTU_PND: 清空SD连续读写结束标志(读为0)【注意】: 先清SD数据pending, 再清这里)<br>0: N/A<br>1: 清空                                                                                                                           |

| Bit | Name        | RW | Default | RV | Description                                                                        |
|-----|-------------|----|---------|----|------------------------------------------------------------------------------------|
| 5   | CTU_ERR     | r  | 0       | -  | CTU_ERR:<br>SD连续读写出错标志<br>0: 正确<br>1: 出错                                           |
| 4   | CLR_CTU_ERR | w  | 0       | -  | CLR_CTU_ERR:<br>清空SD连续读写出错标志（读为0）<br>（【注意】：先清SD数据pending, 再清这里）<br>0: N/A<br>1: 清空 |
| 3   | CTU_BUSY    | r  | 0       | -  | CTU_BUSY: SD数据连续读写状态<br>0: 空闲<br>1: 忙                                              |
| 2   | CTU_PND_IE  | rw | 0       | -  | CTU_PND_IE:<br>SD读写结束中断允许<br>0: 不允许<br>1: 允许                                       |
| 1   | CTU_ERR_IE  | rw | 0       | -  | CTU_ERR_IE:<br>SD读写出错中断允许（出错后会打断数据传输）<br>0: 不允许<br>1: 允许                           |
| 0   | SD_CTU_EN   | rw | 0       | -  | SD_CTU_EN:<br>SD 数据多个BLOCK读写使能<br>0: 关闭<br>1: 打开                                   |

### 16.2.5 JL\_SD0-> CTU\_CNT: sd0 continual transfer counter register

| Bit  | Name    | RW | Default | RV | Description                                                                                                                                                                                                        |
|------|---------|----|---------|----|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-0 | CTU_CNT | rw | 0x0     | -  | SD 数据连续读写模式计数器（写寄存器kictstart数据传输）：<br>0: 开始1次SD数据读写<br>1: 开始2次SD数据读写<br>2: 开始3次SD数据读写<br>.....<br>0xffff: 开始65536次SD数据读写<br>读寄存器可以看到剩下的数据传输次数：<br>0: 不再传输<br>1: 还剩1次传输<br>2: 还剩2次传输<br>.....<br>0xffff: 还剩65535次传输 |

### 16.2.6 JL\_SD0-> CPTR: sd0 command pointer register

| Bit  | Name | RW | Default | RV | Description |
|------|------|----|---------|----|-------------|
| 26-0 | CPTR | w  | 0x0     | -  | SD命令地址      |

### 16.2.7 JL\_SD0-> DPTR: sd0 data pointer register

| Bit  | Name | RW | Default | RV | Description |
|------|------|----|---------|----|-------------|
| 26-0 | DPTR | w  | 0x0     | -  | SD数据地址      |

## 16.3 配置流程示例

### 16.3.1 发送命令、接收响应流程

1. 配置SD\_CON0和SD\_CON1进行初始化;
2. 往memory中写入长度为40bit的命令（实际命令长度为48bit，其中8bit是CRC和结束位由硬件自动进行计算并补齐），然后将数据的首地址配置给SDCPTR。（可将命令写入数组中，然后将数组首地址配置给SDCMDPTR，见2. SDCPTR的CMD responses 排列示例（CMD3））；
3. 配置SDCMODE（SD\_CON0[2:0]）进行kickstart，开始发送命令；
4. 等待PENDING（查询CINT（SD\_CON0[7]）），或等待中断到来；
5. 配置CLR\_CINT（SD\_CON0[6]），清除PENDING；
6. 查询CMDCRC（SD\_CON0[3]），确认接收的响应正确；
7. 查看接收的响应。

### 16.3.2 发送数据流程（单块写CMD24）

1. 配置SD\_CON0和SD\_CON1进行初始化；
2. 配置SD\_CON2进行配置将要发送数据的块长度；
3. 往memory中写入将要发送的数据，然后将数据的首地址配置给SDDPTR；
4. 配置SDDMODE（SD\_CON0[10:8]）进行kickstart，开始发送数据；
5. 等待PENDING（查询DINT（SD\_CON0[15]）），或等待中断到来；
6. 配置CLR\_DINT（SD\_CON0[14]），清除PENDING；
7. 查询CRCSTA（SD\_CON1[3:1]），确认发送的数据正确。

### 16.3.3 接收数据流程（单块读CMD17）

1. 配置SD\_CON0和SD\_CON1进行初始化；
2. 配置SD\_CON2进行配置将要接收数据的块长度；
3. 配置SDDPTR为数据保存在memory中的首地址；
4. 配置SDDMODE（SD\_CON0[10:8]）进行kickstart，开始接收数据；

5. 等待PENDING（查询DINT（SD\_CON0[15]）），或等待中断到来；
6. 配置CLR\_DINT（SD\_CON0[14]），清除PENDING；
7. 查询DATCRC（SD\_CON0[11]），确认接收的数据正确；
8. 查看接收的数据。

**【注意】**：进行单块读CMD17时，需先进行kickstart接收数据，然后发送命令，最后等待数据PENDING，即在上述流程4和5之间进行发送命令。

#### 16.3.4 发送数据流程（多块写CMD25）

1. 配置SD\_CON0、SD\_CON1和SD\_CTU\_CON进行初始化；
2. 配置SD\_CON2为将要发送数据的块长度，配置SD\_CTU\_CNT为将要发送数据的块数量；
3. 往memory中写入将要发送的数据，然后将数据的首地址配置给SDDPTR；
4. 配置SDDMODE\_KEEP（SD\_CTU\_CON[10:8]）进行kickstart，开始发送数据；
5. 等待PENDING（查询CTU\_PND（SD\_CTU\_CON[7]）），或等待中断到来；
6. 配置CLR\_CTU\_PND（SD\_CTU\_CON[6]），清除PENDING；
7. 查询CTU\_ERR（SD\_CTU\_CON[5]），确认发送数据过程没有发生错误（发送各块数据返回的CRC状态有一个错误都会标志在该位）。

#### 16.3.5 接收数据流程（多块写CMD18）

1. 配置SD\_CON0、SD\_CON1和SD\_CTU\_CON进行初始化；
2. 配置SD\_CON2为将要接收数据的块长度，配置SD\_CTU\_CNT为将要接收数据的块数量；
3. 配置SDDPTR为数据保存在memory中的首地址；
4. 配置SDDMODE\_KEEP（SD\_CTU\_CON[10:8]）进行kickstart，开始接收数据；
5. 等待PENDING（查询CTU\_PND（SD\_CTU\_CON[7]）），或等待中断到来；
6. 配置CLR\_CTU\_PND（SD\_CTU\_CON[6]），清除PENDING；
7. 查询CTU\_ERR（SD\_CTU\_CON[5]），确认接收数据过程没有发生错误（接收各块数据的CRC有一个错误都会标志在该位）；
8. 查看接收的数据。

**【注意】**：进行多块读CMD18时，需先进行kickstart接收数据，然后发送命令，最后等待数据PENDING，即在上述流程4和5之间进行发送命令。

### 16.4 DEMO CODE

CMD responses 排列示例（CMD3）：

```
u32 __attribute__((aligned (16))) cmd_str[6];
u32 rca;
//send cmd
```

```
cmd_str[0] = ((0x40 + 3));
cmd_str[1] = 0x0;
JL_SD0->CPTR = (volatile u32)cmd_str;
.....
//obtain respond
sd0_cmd_finish_check();
rca = cmd_str[1] >> 24;
rca |= cmd_str[2] << 8;
rca &= 0x0000ffff;
```

| cmd_str[0] | Cmd send byte 3 | Cmd send byte 2 | Cmd send byte 1 | Cmd send byte 0 |
|------------|-----------------|-----------------|-----------------|-----------------|
| cmd_str[1] | Cmd res byte 1  | Cmd res byte 0  | Cmd send byte 5 | Cmd send byte 4 |
| cmd_str[2] | Cmd res byte 5  | Cmd res byte 4  | Cmd res byte 3  | Cmd res byte 2  |
| cmd_str[3] | Cmd res byte 9  | Cmd res byte 8  | Cmd res byte 7  | Cmd res byte 6  |
| cmd_str[4] | Cmd res byte 13 | Cmd res byte 12 | Cmd res byte 11 | Cmd res byte 10 |
| cmd_str[5] |                 | Cmd res byte 16 | Cmd res byte 15 | Cmd res byte 14 |

(本质是接收响应的memory地址紧接在发送命令的memory地址后面)

## 17 SPI

### 17.1 概述

SPI接口是一个标准的遵守SPI协议的串行通讯接口，在上面传输的数据以Byte（8bit）为最小单位，且永远是MSB在前。

SPI接口支持主机和从机两种模式。

主机：SPI接口时钟由本机产生，提供给片外SPI设备使用。

从机：SPI接口时钟由片外SPI设备产生，提供给本机使用。

工作于主机模式时，SPI接口的驱动时钟可配置，范围为系统时钟~系统时钟/256。

工作于从机模式时，SPI接口的驱动时钟频率无特殊要求，但数据速率需要进行限制，否则易出现接收缓冲覆盖错误，在系统不繁忙情况下可以接近系统时钟速率。

SPI接口可独立地选择在SPI时钟的上升沿或下降沿更新数据，在SPI时钟的上升沿或下降沿采样数据。

SPI接口支持单向（Unidirection）和双向（Bidirection）模式

单向模式：使用SPICK和SPIDAT两组连线，其中SPIDAT为双向信号线，同一时刻数据只能单方向传输。

双向模式：使用SPICK，SPIDI和SPIDO三组连线，同一时刻数据双向传输。但DMA不支持双向数据传输，当在本模式下使能DMA时，也只有一个方向的数据能通过DMA和系统进行传输。

SPI单向模式支持1bit data、2bit data和4bit data模式，即：

1bit data模式：串行数据通过一根DAT线传输，一个字节数据需8个SPI时钟。

2bit data模式：串行数据通过两根DAT线传输，一个字节数据需4个SPI时钟。

4bit data模式：串行数据通过四根DAT线传输，一个字节数据需2个SPI时钟。

SPI双向模式只支持1bit data模式，即：

1bit data模式：串行数据通过两根DAT线进行双向传输，一个字节数据需8个SPI时钟。

SPI接口在发送方向上为单缓冲，在上一次传输未完成之前，不可开始下一次传输。在接收方向上为双缓冲，如果在下一次传输完成时CPU还未取走本次的接收数据，那么本次的接收数据将会丢失。

SPI接口的发送寄存器和接收寄存器在物理上是分开的，但在逻辑上它们一起称为SPIBUF寄存器，使用相同的SFR地址。当写这个SFR地址时，写入至发送寄存器。当读这个SFR地址时，从接收寄存器读出。

SPI传输支持由CPU直接驱动，写SPIBUF的动作将启动一次Byte传输。

SPI传输也支持DMA操作，但DMA操作永远是单方向的，即一次DMA要么是发送一包数据，要么是接收一包数据，不能同时发送并且接收一包数据，即使在双向模式下也是这样。每次DMA操作支持的数据量为 $1-(2^{32}-1)$ Byte。写SPIADR的动作将启动一次DMA传输。

### 17.2 特殊注意点

【注意】：该模块所有说明及配置仅作为SPIx模块使用，不适用到SFC中

## 17.3 数字模块控制寄存器

### 17.3.1 SPIx\_CON: SPIx control register 0

| Bit   | Name       | RW | Default | RV  | Description                                                                                        |
|-------|------------|----|---------|-----|----------------------------------------------------------------------------------------------------|
| 31-22 | RESERVED   | -  | -       | -   | 预留                                                                                                 |
| 21    | OF_PND     | r  | 0       | -   | dma fifo overflow中断，当spi做从机dma接收时，fifo满且dma响应慢，而新接收的数据到来时导致fifo发出溢出就会产生of_pnd<br>【注意】：仅做debug使用!!! |
| 20    | OF_PND_CLR | w  | 0       | -   | 清除of_pnd，写1清零                                                                                      |
| 19    | OF_IE      | rw | 0       | 1   | of_pnd使能位                                                                                          |
| 18    | UF_PND     | r  | 0       | -   | dma fifo underflow中断，当spi做从机dma发送时，fifo空且dma响应慢，而主机端请求新数据时导致fifo读空就会产生uf_pnd<br>【注意】：仅做debug使用     |
| 17    | UF_PND_CLR | w  | 0       | -   | 清除uf_pnd，写1清零                                                                                      |
| 16    | UF_IE      | rw | 0       | 1   | uf_pnd使能位                                                                                          |
| 15    | PND        | r  | 0       | 0   | 中断请求标志，当1Byte传输完成或DMA传输完成时会被硬件置1。<br>有3种方法清除此标志 向PCLR写入‘1’ 写SPIBUF寄存器来启动一次传输<br>写SPICNT寄存器来启动一次DMA |
| 14    | CPND       | w  | 0       | 1   | 软件在此位写入‘1’将清除PND中断请求标志。                                                                            |
| 13    | IE         | rw | 0       | 1   | SPI 中断使能<br>0: 禁止SPI中断<br>1: 允许中断允许                                                                |
| 12    | DIR        | rw | 0       | 0   | 在单向模式或DMA操作时设置传输的方向<br>0: 发送数据<br>1: 接收数据                                                          |
| 11-10 | DATW       | r  | 0x0     | 0x0 | SPI数据宽度设置<br>00: 1bit数据宽度<br>01: 2bit 数据宽度<br>10: 4bit数据宽度<br>11: NA，不可设置为此项                       |
| 9-8   | RESERVED   | -  | -       | -   | 预留                                                                                                 |



| Bit | Name  | RW | Default | RV | Description                                                                                                                                                      |
|-----|-------|----|---------|----|------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 7   | CSID  | rw | 0       | 0  | SPICS信号极性选择<br>0: SPICS空闲时为0电平<br>1: SPICS空闲时为1电平                                                                                                                |
| 6   | CKID  | rw | 0       | 0  | SPICK信号极性选择<br>0: SPICK空闲时为0电平<br>1: SPICK空闲时为1电平                                                                                                                |
| 5   | UE    | rw | 0       | 0  | 更新数据边沿选择<br>0: 在SPICK的上升沿更新数据<br>1: 在SPICK的下降沿更新数据                                                                                                               |
| 4   | SE    | rw | 0       | 0  | 采样数据边沿选择<br>0: 在SPICK的上升沿采样数据<br>1: 在SPICK的下降沿采样数据                                                                                                               |
| 3   | BDIR  | rw | 0       | 0  | 单向/双向模式选择<br>0: 单向模式，数据单向传输，同一时刻只能发送或者接收数据。<br>数据传输方向因收发而改变，所以由硬件控制，不受写IO口DIR影响。<br>1: 双向模式，数据双向传输，同时收发数据，但DMA只支持一个方向的数据传输。<br>数据传输方向设置后不改变，所以由软件控制，通过写IO口DIR控制。 |
| 2   | CSE   | rw | 0       | 1  | SPICS信号使能，always set为1                                                                                                                                           |
| 1   | SLAVE | rw | 0       | 0  | 从机模式                                                                                                                                                             |
| 0   | SPIEN | rw | 0       | 0  | SPI接口使能<br>0: 关闭SPI接口<br>1: 打开SPI接口                                                                                                                              |

### 17.3.2 SPIx\_BAUD: SPIx baudrate setting register

| Bit | Name     | RW | Default | RV  | Description                                                          |
|-----|----------|----|---------|-----|----------------------------------------------------------------------|
| 7-0 | SPI_BAUD | rw | 0x0     | 0x0 | SPI主机时钟设置寄存器<br>$SPICK = \text{system clock} / (\text{SPIBAUD} + 1)$ |

### 17.3.3 SPIx\_BUF: SPIx buffer register

| Bit | Name    | RW | Default | RV  | Description                                |
|-----|---------|----|---------|-----|--------------------------------------------|
| 7-0 | SPI_BUF | rw | 0x0     | 0x0 | 发送寄存器和接收寄存器共用此SFR地址。<br>写入至发送寄存器，从接收寄存器读出。 |

### 17.3.4 SPIx\_ADR: SPIx DMA address register



| Bit  | Name    | RW | Default | RV  | Description               |
|------|---------|----|---------|-----|---------------------------|
| 31-0 | SPI_ADR | rw | 0x0     | 0x0 | SPI DMA起始地址寄存器，只写，读出为不确定值 |

### 17.3.5 SPIx\_CNT: SPIx DMA counter register

| Bit  | Name    | RW | Default | RV  | Description                                                                                               |
|------|---------|----|---------|-----|-----------------------------------------------------------------------------------------------------------|
| 31-0 | SPI_CNT | rw | 0x0     | 0x0 | SPI DMA计数寄存器，读出为当下剩余读写数量，此寄存器用于设置DMA操作的数目（按Byte计）并启动DMA传输。<br>如，需启动一次512Byte的DMA传输，写入0x0200，此写入动作将启动本次传输。 |

### 17.3.6 SPIx\_CON1: SPIx control1 register

| Bit  | Name         | RW | Default | RV | Description                                                                                                            |
|------|--------------|----|---------|----|------------------------------------------------------------------------------------------------------------------------|
| 31-2 | RESERVED     | -  | -       | -  | 预留                                                                                                                     |
| 4    | SPI_9B_DATA  | rw | 0       | -  | 在9bit模式下，发送命令时，为0；<br>发送参数/数据时，为1。                                                                                     |
| 3    | SPI_9B_MODE  | rw | 0       | -  | 0: 关闭9bit模式<br>1: 打开9bit模式                                                                                             |
| 2    | MIX_MODE     | rw | 0       | -  | 3wire模式混合接线                                                                                                            |
| 1-0  | SPI_BIT_MODE | rw | 0       | 0  | 设置spi输入输出位流模式，默认0为标准格式<br>0: 【7,6,5,4,3,2,1,0】<br>1: 【0,1,2,3,4,5,6,7】<br>2: 【3,2,1,0,7,6,5,4】<br>3: 【4,5,6,7,0,1,2,3】 |

## 17.4 传输波形

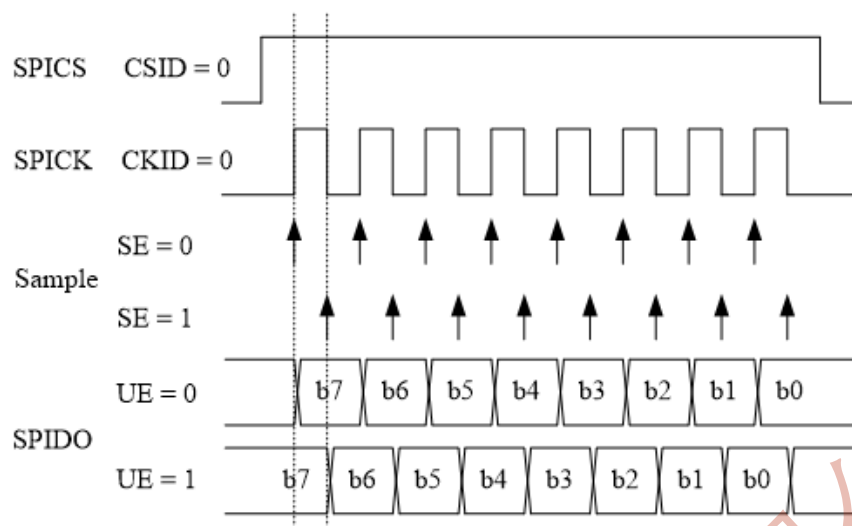


图 1-1 传输波形1

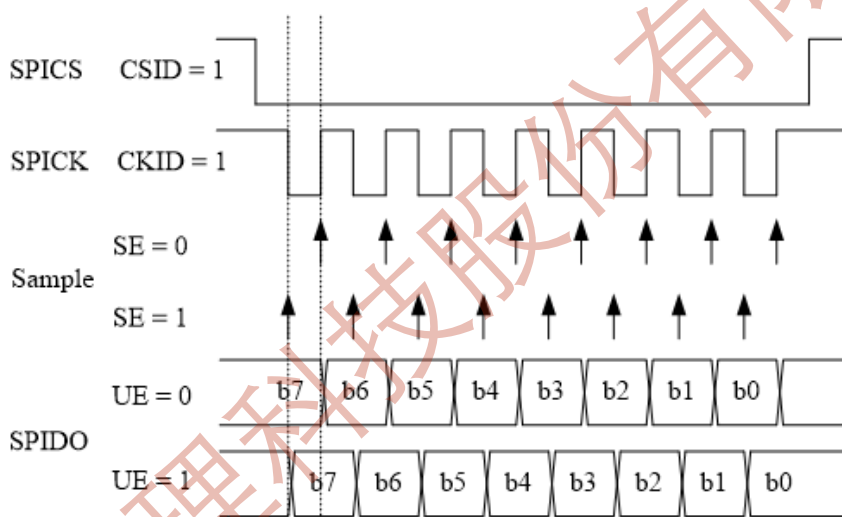


图 1-2 传输波形2

# 18 16位GPTIMER

## 18.1 概述

Timer是一个集成了定时/计数/捕获功能于一体的多功能定时器（计数位宽参见芯片规格）。它的计数时钟驱动源可以选择片内时钟或片外信号。它带有一个可配置的最高达64的异步预分频器，用于扩展定时时间或片外信号的最高频率。同时具有上升沿/下降沿捕获功能，可以方便的对片外信号的高电平/低电平宽度进行测量。

且每个Timer都具有独立的CAP\_FLT硬件模块，使能后可用于去除掉从片外进入TIMER CAP 信号上的窄脉冲信号（多用于提升红外接收解码的质量）；CAP FLT模块使用一个固定的时基对TIMER CAP进入的信号进行采样，改变CAP\_FLT时基时钟的产生可兼容不同的系统工作状态，也可在一定范围内调整对CAP信号的过滤效果，滤波范围如下：

- 必须连续4次采样均为‘1’时，输出信号才会变为‘1’；
- 必须连续4次采样均为‘0’时，输出信号才会变为‘0’

脉宽小于3倍时基的窄脉冲将被滤除

TIMER 计数时钟源，CAP\_FLT时基时钟源 参见 Clock\_system 文档；

## 18.2 控制寄存器

| 寄存器列表    | TIMER0   | TIMER1   | TIMER2   |
|----------|----------|----------|----------|
| Tx_CON   | T0_CON   | T1_CON   | T2_CON   |
| Tx_CNT   | T0_CNT   | T1_CNT   | T2_CNT   |
| Tx_PRD   | T0_PRD   | T1_PRD   | T2_PRD   |
| Tx_PWM   | T0_PWM   | T1_PWM   | T2_PWM   |
| Tx_IRFLT | T0_IRFLT | T1_IRFLT | T2_IRFLT |

该芯片只有TIMER0-TIMER2

## 18.3 寄存器说明

### 18.3.1 Tx\_CON: timer x control register

| Bit   | Name         | RW | Default | RV | Description                                                          |
|-------|--------------|----|---------|----|----------------------------------------------------------------------|
| 31-17 | RESERVED     | -  | -       | -  | 预留                                                                   |
| 16    | DUAL_EDGE_EN | rw | 0       | -  | DUAL_EDGE_EN:双边沿捕获模式，当MODE=2或3时，使能该位即变成双边沿捕获模式，在上沿和下沿会TxCNT的值写入TxPR。 |
| 15    | PND          | r  | 0       | -  | PND: 中断请求标志，当timer溢出或产生捕获动作时会被硬件置1，需要由软件清0。                          |

| Bit   | Name       | RW | Default | RV | Description                                                                                                                                                                                                                                                                                                                  |
|-------|------------|----|---------|----|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 14    | PCLR       | w  | x       | -  | PCLR: 软件在此位写入‘1’将清除PND中断请求标志。                                                                                                                                                                                                                                                                                                |
| 13-10 | SSEL       | rw | 0x0     | -  | SSEL3-0: TIMER驱动源选择见图1-2<br>【注意】: 选中的时钟需要使用上面PSET分频到(lsb_clk/2)以下!                                                                                                                                                                                                                                                           |
| 9     | PWM_INV    | rw | 0       | -  | PWM_INV: PWM信号输出反向。                                                                                                                                                                                                                                                                                                          |
| 8     | PWM_EN     | rw | 0       | -  | PWM_EN: PWM信号输出使能。此位置1后, 相应IO口的功能将会被PWM信号输出替代。                                                                                                                                                                                                                                                                               |
| 7-4   | PSET       | rw | 0x0     | -  | PSET3-0: 预分频选择位<br>0000: 分频倍数为1;<br>0001: 分频倍数为4;<br>0010: 分频倍数为16;<br>0011: 分频倍数为64;<br>0100: 分频倍数为2;<br>0101: 分频倍数为8;<br>0110: 分频倍数为32;<br>0111: 分频倍数为128;<br>1000: 分频倍数为256;<br>1001: 分频倍数为1024;<br>1010: 分频倍数为4096;<br>1011: 分频倍数为16384;<br>1100: 分频倍数为512;<br>1101: 分频倍数为2048;<br>1110: 分频倍数为8192;<br>1111: 分频倍数为32768; |
| 3-2   | CAP_FLT_EN | rw | 0x0     | -  | 捕获信号滤波使能 (仅捕获模式使用)<br>0: 从IO 直接输入到TIMER CAP;<br>1: 从IO 输入后先经过FLT后再到TIMER CAP (需配合IRFLT寄存器使用)                                                                                                                                                                                                                                 |
| 1-0   | MODE       | rw | 0x0     | -  | MODE1-0: 工作模式选择<br>00: TIMER关闭;<br>01: 定时/计数模式;<br>10: IO口上升沿捕获模式 (当IO上升沿到来时, 把TxCNT的值捕捉到TxPR中);<br>11: IO口下降沿捕获模式 (当IO下降沿到来时, 把TxCNT的值捕捉到TxPR中)。                                                                                                                                                                            |

### 18.3.2 Tx\_CNT: timer x counter register

| Bit  | Name   | RW | Default | Description |
|------|--------|----|---------|-------------|
| 15-0 | Tx_CNT | rw | x       | Timer的计数寄存器 |

### 18.3.3 Tx\_PR: timer x period register

| Bit  | Name  | RW | Default | Description |
|------|-------|----|---------|-------------|
| 15-0 | Tx_PR | rw | x       | Timer的周期寄存器 |

在定时/计数模式下，当Tx\_CNT = Tx\_PR时，Tx\_CNT会被清0。

在上升沿/下降沿捕获模式下，TxPR是作为捕获寄存器使用的，当捕获发生时，Tx\_CNT的值会被复制到Tx\_PR中。而此时Tx\_CNT自由的由0->(2<sup>32</sup>-1)->0计数，不会和Tx\_PR进行比较清0。

#### 18.3.4 Tx\_PWM: timer x PWM register

| Bit  | Name   | RW | Default | Description    |
|------|--------|----|---------|----------------|
| 15-0 | Tx_PWM | rw | x       | Timer的PWM设置寄存器 |

在PWM模式下，此寄存器的值决定PWM输出的占空比。占空比N的计算公式如下：

$$N = (Tx\_PWM / Tx\_PR) * 100\%$$

此寄存器带有缓冲，写此寄存器的动作不会导致不同步状态产生的PWM波形占空比瞬间过大或过小的问题

#### 18.3.5 Tx\_IRFLT: timer x PWM register

| Bit | Name     | RW | Default | Description                                                                                                                                                                                                                                                                                                                       |
|-----|----------|----|---------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 7-4 | PSEL     | rw | x       | PSEL[3-0]: 时基发生器分频选择<br>0000: 分频倍数为1;<br>0001: 分频倍数为2;<br>0010: 分频倍数为4;<br>0011: 分频倍数为8;<br>0100: 分频倍数为16;<br>0101: 分频倍数为32;<br>0110: 分频倍数为64;<br>0111: 分频倍数为128;<br>1000: 分频倍数为256;<br>1001: 分频倍数为512;<br>1010: 分频倍数为1024;<br>1011: 分频倍数为2048;<br>1100: 分频倍数为4096;<br>1101: 分频倍数为8192;<br>1110: 分频倍数为16384;<br>1111: 分频倍数为32768; |
| 3-2 | TSRC     | rw | x       | TSRC[1-0]: 时基发生器驱动源选择，见Clock_System 文档                                                                                                                                                                                                                                                                                            |
| 1   | RESERVED | -  | -       | 预留                                                                                                                                                                                                                                                                                                                                |
| 0   | FLT_EN   | rw | 0       | FLT_EN: FLT使能<br>0: 关闭IRFLT;<br>1: 打开IRFLT; ;                                                                                                                                                                                                                                                                                     |

PSEL选定的分频倍数N和TSRC选定的驱动时钟的周期Tc共同决定了IRFLT用于采样信号的时基

Ts:

$$Ts = Tc * N$$

如选择24MHz的时基时钟，并且分频倍数为512时，Ts = 21.3us，所有小于(21.3\*3=63.9us)的窄脉冲信号，均会被滤除；

珠海市杰理科技股份有限公司

## 19 UART

### 19.1 概述

支持接收带循环Buffer的DMA模式和普通模式。

UART在DMA接收的时候有一个循环Buffer，UTx\_RXSADR表示它的起始，UTx\_RXEADR表示它的结束。同时，在接收过程中，会有一个**超时计数器（UTx\_OTCNT）**，如果在指定的时间里没有收到任何数据，**则超时中断就会产生**。超时计数器是在收到数据的同时自动清空。

### 19.2 特殊注意点

【注意】：

- OT\_PND触发流程，满足一下（1-4）顺序，则可产生OT\_PND：
  - 写UTx\_OTCNT，数值不为0启动OT功能；
  - 收到n个byte数据；
  - 从最后一个下降沿开始，等待OT超时（每来一个下降沿都会重新计时，时钟与接收baud\_clk一致）；
  - 当OTCNT与超时时间相等，OT\_PND置1（只能通过CLR\_OTPND清零）。
- 读接收的数据数量时，先将RDC该bit写1，接着asm(“csync”)清空流水线，然后软件查询RDC\_OVER置1，置1后即可读取HRXCNT数值，RDC\_OVER只在RDC写1之后生效，平常为无效状态（不管0或1）；

### 19.3 数字模块控制寄存器

#### 19.3.1 TX\_CON0: uart tx control register 0

| Bit   | Name     | RW | Default | RV | Description                                    |
|-------|----------|----|---------|----|------------------------------------------------|
| 31-16 | RESERVED | -  | -       | -  | 预留                                             |
| 15    | TPND     | r  | 1       | -  | TPND:TX Pending                                |
| 14    | RESERVED | -  | -       | -  | 预留                                             |
| 13    | CL RTPND | w  | 0       | -  | CL RTPND: 清空TX Pending                         |
| 12-3  | RESERVED | -  | -       | -  | 预留                                             |
| 2     | TXIE     | rw | 0       | 1  | TXIE:TX中断允许<br>当TX Pending为1，而且TX中断允许为1，则会产生中断 |
| 1     | RESERVED | -  | -       | -  | 预留                                             |
| 0     | UTTXEN   | rw | 0       | 1  | UTTXEN:UART模块发送使能                              |

#### 19.3.2 RX\_CON0: uart rx control register 0

| Bit   | Name      | RW | Default | RV | Description                                          |
|-------|-----------|----|---------|----|------------------------------------------------------|
| 31-15 | RESERVED  | -  | -       | -  | 预留                                                   |
| 14    | RPND      | r  | 0       | -  | RPND:RX Pending& Dma_Wr_Buf_Empty, 数据接收不完Pending不会为1 |
| 13    | RESERVED  | -  | -       | -  | 预留                                                   |
| 12    | CLRRPND   | w  | 0       | -  | CLRRPND: 清空 RX Pending                               |
| 11    | OTPND     | r  | 0       | -  | OTPND:OverTime Pending                               |
| 10    | CLR_OTPND | w  | 0       | -  | CLR_OTPND: 清空OTPND                                   |
| 9     | RESERVED  | -  | -       | -  | 预留                                                   |
| 8     | RDC_OVER  | r  | 0       | -  | 用于判断写RDC后数据是否已存入内存标志, RDC写1时会清零, 当数据存入内存后置1。         |
| 7     | RDC       | w  | 0       | -  | RDC: 写1时, 将已经收到的数目写到UTx_HRXCNT, 已收到的数目清零写0无效         |
| 6     | RX_MODE   | rw | 0       | -  | RXMODE (*): 读模式选择<br>0: 普通模式, 不用DMA;<br>1: DMA模式。    |
| 5     | OT_IE     | rw | 0       | 1  | OTIE:OT中断允许<br>0:不允许;<br>1: 允许。                      |
| 4     | DIVS      | rw | 0       | -  | DIVS:前3分频选择, 0为4分频, 1为3分频                            |
| 3     | RXIE      | rw | 0       | 1  | RXIE:RX中断允许<br>当RX Pending为1, 而且RX中断允许为1, 则会产生中断     |
| 2     | RESERVED  | -  | -       | -  | 预留                                                   |
| 1     | UTRXEN    | rw | 0       | 1  | UTRXEN:UART模块接收使能                                    |
| 0     | RESERVED  | -  | -       | -  | 预留                                                   |

### 19.3.3 TX\_CON1: uart tx control register 1

| Bit   | Name     | RW | Default | RV | Description          |
|-------|----------|----|---------|----|----------------------|
| 31-16 | RESERVED | -  | -       | -  | 预留                   |
| 15    | CTSPND   | r  | 0       | 0  | CTSPND: CTS中断pending |
| 14    | RESERVED | -  | -       | -  | 预留                   |



| Bit   | Name       | RW | Default | RV | Description                                     |
|-------|------------|----|---------|----|-------------------------------------------------|
| 13    | CLR_CTSPND | w  | 0       | -  | CLR_CTSPND: 清除CTS pending                       |
| 12-10 | RESERVED   | -  | 0       | -  | 预留                                              |
| 9     | TX_INV     | rw | 0       | 0  | Tx发送数据电平取反<br>0: 不取反<br>1: 取反                   |
| 8     | RESERVED   | -  | -       | -  | 预留                                              |
| 7     | CTS_INV    | rw | 0       | -  | CTS流控有效电平选择（对方允许我方发送数据）<br>0: 高电平有效<br>1: 低电平有效 |
| 6-4   | RESERVED   | -  | -       | -  | 预留                                              |
| 3     | CTSIE      | rw | 0       | -  | CTSIE: CTS中断使能<br>0: 禁止中断;<br>1: 中断允许           |
| 2     | CTSE       | rw | 0       | -  | CTSE:CTS使能<br>0: 禁止CTS硬件流控制<br>1: 允许CTS硬件流控制    |
| 1-0   | RESERVED   | -  | 0       | -  | 预留                                              |

#### 19.3.4 RX\_CON1: uart rx control register 1

| Bit   | Name       | RW | Default | RV | Description                         |
|-------|------------|----|---------|----|-------------------------------------|
| 31-15 | RESERVED   | -  | -       | -  | 预留                                  |
| 14    | RTSPND     | r  | 0       | 0  | RTSPND: RST pending                 |
| 13    | RESERVED   | -  | -       | -  | 预留                                  |
| 12    | CLR_RTSPND | w  | 0       | -  | CLRRTS: 清除RTS<br>0: N/A<br>1: 清空RTS |
| 11-9  | RESERVED   | -  | 0       | -  | 预留                                  |
| 8     | RX_INV     | rw | 0       | 0  | Rx接收数据电平取反<br>0: 不取反<br>1: 取反       |
| 7     | RESERVED   | -  | -       | -  | 预留                                  |

| Bit | Name       | RW | Default | RV | Description                                   |
|-----|------------|----|---------|----|-----------------------------------------------|
| 6   | RTS_INV    | rw | 0       | -  | RTS流控有效电平选择（允许对方发送数据）<br>0: 高电平有效<br>1: 低电平有效 |
| 5   | RX_BYPASS  | rw | 1       | 1  | Rx接收数据通路直通使能<br>0: 滤波输入<br>1: 直通输入            |
| 4   | RX_DISABLE | rw | 1       | 0  | 关闭数据接收<br>0: 开启输入（正常模式）<br>1: 关闭输入（输入固定为1）    |
| 3-1 | RESERVED   | -  | -       | -  | 预留                                            |
| 0   | RTSE       | rw | 0       | -  | RTSE:RTS使能<br>0: 禁止RTS硬件流控制<br>1: 允许RTS硬件流控制  |

### 19.3.5 UTx\_CON2: uart x control register 2

| Bit  | Name       | RW | Default | RV | Description                                   |
|------|------------|----|---------|----|-----------------------------------------------|
| 15-3 | RESERVED   | -  | -       | -  | 预留                                            |
| 8    | CHK_PND    | r  | 0       | -  | 校验中断标志                                        |
| 7    | CLR_CHKPND | w  | 0       | -  | 校验中断CHK_PND清除，置1清除                            |
| 6    | CHK_IE     | rw | 0       | 1  | 校验中断使能，置1使能                                   |
| 5-4  | CHECK_MODE | rw | 0       | -  | 校验模式选择：<br>0: 常0<br>1: 常1<br>2: 偶校验<br>3: 奇校验 |
| 3    | CHECK_EN   | rw | 0       | 0  | 校验功能使能位，置1使能                                  |
| 2    | RB8        | r  | 0       | 0  | RB8:9bit模式时，RX接收的第9位                          |
| 1    | RESERVED   | -  | -       | -  | 预留                                            |
| 0    | M9EN       | rw | 0       | 0  | M9EN: 9bit模式使能                                |

### 19.3.6 UTx\_BAUD: uart x baudrate register

| Bit  | Name     | RW | Default | RV  | Description |
|------|----------|----|---------|-----|-------------|
| 15-0 | UTx_BAUD | w  | 0x0     | 0x0 | 注释如下所示      |

uart 的UTx\_DIV的整数部分

串口频率分频器因子(UTx\_DIV)的整数部分

当DIVS=0时，

$$\text{Baudrate} = \text{Freq\_sys} / ((\text{UTx\_BAUD}+1) * 4 + \text{BAUD\_FRAC})$$

当DIVS=1时，

$$\text{Baudrate} = \text{Freq\_sys} / ((\text{UTx\_BAUD}+1) * 3 + \text{BAUD\_FRAC})$$

(Freq\_sys是apb\_clk,指慢速设备总线的时钟，非系统时钟)

### 19.3.7 UTx\_BUF: uart x data buffer register

| Bit  | Name     | RW | Default | RV | Description                                             |
|------|----------|----|---------|----|---------------------------------------------------------|
| 31-8 | RESERVED | -  | 0       | -  | 预留                                                      |
| 7-0  | UT_BUF   | rw | 0x0     | -  | uart的收发数据寄存器<br>写UTx_BUF可启动一次发送；<br>读UTx_BUF可获得已接收到的数据。 |

### 19.3.8 UTx\_TXADR: uart x TX DMA buffer register

| Bit   | Name      | RW | Default | RV  | Description                                            |
|-------|-----------|----|---------|-----|--------------------------------------------------------|
| 31-25 | RESERVED  | -  | 0       | -   | 预留                                                     |
| 24-0  | UTx_TXADR | rw | 0x0     | 0x0 | DMA发送数据的指针<br>写入时作为DMA发送数据的<br>起始地址，读出时时DMA正<br>在操作的位置 |

UTx\_TXADR的读出值不是操作的实时地址

### 19.3.9 UTx\_TXCNT: uart x TX DMA counter register

| Bit  | Name      | RW | Default | RV  | Description                                                                                                     |
|------|-----------|----|---------|-----|-----------------------------------------------------------------------------------------------------------------|
| 31-0 | UTx_TXCNT | rw | 0x0     | 0x0 | 写UTx_TXCNT，控制器产生一次DMA的操作，同时清空中断（写该寄存器需要开启RX_EN才有效），当uart需要发送的数据达到UTx_TXCNT的值，控制器会停止发送数据的操作，同时产生中断（UTx_CON[15]）。 |

### 19.3.10 UTx\_RXCNT: uart x RX DMA counter register

| Bit | Name | RW | Default | RV | Description |
|-----|------|----|---------|----|-------------|
|-----|------|----|---------|----|-------------|

| Bit  | Name      | RW | Default | RV  | Description                                                                                  |
|------|-----------|----|---------|-----|----------------------------------------------------------------------------------------------|
| 31-0 | UTx_RXCNT | rw | 0x0     | 0x0 | 写UTx_RXCNT，控制器产生一次DMA的操作，同时清空中断，当uart需要接收的数据达到UTx_RXCNT的值，控制器会停止接收数据的操作，同时产生中断（UTx_CON[14]）。 |

### 19.3.11 UTx\_RXSADR: uart x RX DMA start address register

| Bit   | Name       | RW | Default | RV  | Description                       |
|-------|------------|----|---------|-----|-----------------------------------|
| 31-25 | RESERVED   | -  | 0       | -   | 预留                                |
| 24-0  | UTx_RXSADR | rw | 0x0     | 0x0 | DMA接收数据时，循环buffer的开始地址，需4byte地址对齐 |

### 19.3.12 UTx\_RXEADR: uart x RX DMA end address register

| Bit   | Name       | RW | Default | RV  | Description                                                                        |
|-------|------------|----|---------|-----|------------------------------------------------------------------------------------|
| 31-25 | RESERVED   | -  | 0       | -   | 预留                                                                                 |
| 24-0  | UTx_RXEADR | rw | 0x0     | 0x0 | DMA接收数据时，循环buffer的结束地址。需4byte地址对齐<br>【注意】：UTx_RXSADR=UTx_RXEADR时没有循环buffer，接收地址会递增 |

### 19.3.13 UTx\_HRXCNT: uart x have RX DMA counter register

| Bit  | Name       | RW | Default | RV  | Description                                           |
|------|------------|----|---------|-----|-------------------------------------------------------|
| 31-0 | UTx_HRXCNT | r  | 0x0     | 0x0 | 当设这RDC（UTx_CON[7]）=1时，串口设备会将当前总共收到的字节数记录到UTx_HRXCNT里。 |

### 19.3.14 UTx\_OTCNT: uart x Over Timer counter register

| Bit  | Name      | RW | Default | RV  | Description                                                                                                                                 |
|------|-----------|----|---------|-----|---------------------------------------------------------------------------------------------------------------------------------------------|
| 31-0 | UTx_OTCNT | rw | 0x0     | 0x0 | 设置串口设备在等待RX下降沿的时间，在设置的时间内没收到RX下降沿，则产生OT_PND<br>Time(ot)= Time(uart_clk) * UTx_OTCNT;<br>例如：接收uart_clk的频率为10MHz，UTx_OTCNT = 10，那么，OT的时间就为1ms |

### 19.3.15 UTx\_ERR\_CNT: uart x error byte counter register

| Bit  | Name        | RW | Default | RV | Description                                                                                                                                                         |
|------|-------------|----|---------|----|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 31-0 | UTx_ERR_CNT | r  | 0x0     | -  | <p>校验错误数量计数，当打开CHECK_EN后，当第一次校验出错时，该寄存器会记录当前byte对应的数量。</p> <p><b>【注意】</b>：只记录第一次出错的数量，清除CHK_PND会重新记录。</p> <p>若清除CHK_PND后，没有出现新的校验错误数据，则该寄存器会一直保持上一次记录的错误数量计数数据。</p> |

## 20 UART\_LITE

### 20.1 概述

uart\_lite不支持DMA以及OT\_CNT超时，不支持CTS/RTS，仅作为debug uart使用

### 20.2 数字模块控制寄存器

#### 20.2.1 TX\_CON0: uart tx control register 0

| Bit   | Name     | RW | Default | RV | Description                                        |
|-------|----------|----|---------|----|----------------------------------------------------|
| 31-16 | RESERVED | -  | -       | -  | 预留                                                 |
| 15    | TPND     | r  | 1       | -  | TPND:TX Pending                                    |
| 14    | RESERVED | -  | -       | -  | 预留                                                 |
| 13    | CLRTPND  | w  | 0       | -  | CLRTPND: 清空TX Pending                              |
| 12-3  | RESERVED | -  | -       | -  | 预留                                                 |
| 2     | TXIE     | rw | 0       | 1  | TXIE:TX中断允许<br>当TX Pending为1，而且TX中断允许为1，<br>则会产生中断 |
| 1     | RESERVED | -  | -       | -  | 预留                                                 |
| 0     | UTTXEN   | rw | 0       | 1  | UTTXEN:UART模块发送使能                                  |

#### 20.2.2 RX\_CON0: uart rx control register 0

| Bit   | Name     | RW | Default | RV | Description                                        |
|-------|----------|----|---------|----|----------------------------------------------------|
| 31-15 | RESERVED | -  | -       | -  | 预留                                                 |
| 14    | RPND     | r  | 0       | -  | RPND:RX Pending，数据接收不完Pending不会为1                  |
| 13    | RESERVED | -  | -       | -  | 预留                                                 |
| 12    | CLRRPND  | w  | 0       | -  | CLRRPND: 清空 RX Pending                             |
| 11-5  | RESERVED | -  | -       | -  | 预留                                                 |
| 4     | DIVS     | rw | 0       | -  | DIVS:前3分频选择，0为4分频，1为3分频                            |
| 3     | RXIE     | rw | 0       | 1  | RXIE:RX中断允许<br>当RX Pending为1，而且RX中断允许为1，<br>则会产生中断 |

| Bit | Name     | RW | Default | RV | Description       |
|-----|----------|----|---------|----|-------------------|
| 2   | RESERVED | -  | -       | -  | 预留                |
| 1   | UTRXEN   | rw | 0       | 1  | UTRXEN:UART模块接收使能 |
| 0   | RESERVED | -  | -       | -  | 预留                |

### 20.2.3 TX\_CON1: uart tx control register 1

| Bit   | Name     | RW | Default | RV | Description                   |
|-------|----------|----|---------|----|-------------------------------|
| 31-16 | RESERVED | -  | -       | -  | 预留                            |
| 15-10 | RESERVED | -  | 0       | -  | 预留                            |
| 9     | TX_INV   | rw | 0       | 0  | Tx发送数据电平取反<br>0: 不取反<br>1: 取反 |
| 8-0   | RESERVED | -  | 0       | -  | 预留                            |

### 20.2.4 RX\_CON1: uart rx control register 1

| Bit  | Name       | RW | Default | RV | Description                                |
|------|------------|----|---------|----|--------------------------------------------|
| 31-9 | RESERVED   | -  | -       | -  | 预留                                         |
| 8    | RX_INV     | rw | 0       | 0  | Rx接收数据电平取反<br>0: 不取反<br>1: 取反              |
| 7    | RESERVED   | -  | -       | -  | 预留                                         |
| 5    | RX_BYPASS  | rw | 1       | 1  | Rx接收数据通路直通使能<br>0: 滤波输入<br>1: 直通输入         |
| 4    | RX_DISABLE | r  | 1       | 0  | 关闭数据接收<br>0: 开启输入（正常模式）<br>1: 关闭输入（输入固定为1） |
| 3-0  | RESERVED   | -  | -       | -  | 预留                                         |

### 20.2.5 UTx\_CON2: uart x control register 2

| Bit  | Name     | RW | Default | RV | Description |
|------|----------|----|---------|----|-------------|
| 15-3 | RESERVED | -  | -       | -  | 预留          |
| 8    | CHK_PND  | r  | 0       | -  | 校验中断标志      |

| Bit | Name         | RW | Default | RV | Description                               |
|-----|--------------|----|---------|----|-------------------------------------------|
| 7   | CLR_CHKPNPND | w  | 0       | -  | 校验中断CHK_PND清除，置1清除                        |
| 6   | CHK_IE       | rw | 0       | 1  | 校验中断使能，置1使能                               |
| 5-4 | CHECK_MODE   | rw | 0       | -  | 校验模式选择：<br>0：常0<br>1：常1<br>2：偶校验<br>3：奇校验 |
| 3   | CHECK_EN     | rw | 0       | 0  | 校验功能使能位，置1使能                              |
| 2   | RB8          | r  | 0       | 0  | RB8:9bit模式时，RX接收的第9位                      |
| 1   | RESERVED     | -  | -       | -  | 预留                                        |
| 0   | M9EN         | rw | 0       | 0  | M9EN：9bit模式使能                             |

### 20.2.6 UTx\_BAUD: uart x baudrate register

| Bit  | Name     | RW | Default | RV  | Description |
|------|----------|----|---------|-----|-------------|
| 15-0 | UTx_BAUD | w  | 0x0     | 0x0 | 注释如下所示      |

uart 的UTx\_DIV的整数部分

串口频率分频器因子(UTx\_DIV)的整数部分

当DIVS=0时，

$$\text{Baudrate} = \text{Freq\_sys} / ((\text{UTx\_BAUD} + 1) * 4 + \text{BAUD\_FRAC})$$

当DIVS=1时，

$$\text{Baudrate} = \text{Freq\_sys} / ((\text{UTx\_BAUD} + 1) * 3 + \text{BAUD\_FRAC})$$

(Freq\_sys是apb\_clk,指慢速设备总线的时钟，非系统时钟)

### 20.2.7 UTx\_BUF: uart x data buffer register

| Bit  | Name     | RW | Default | RV | Description                                             |
|------|----------|----|---------|----|---------------------------------------------------------|
| 31-8 | RESERVED | -  | 0       | -  | 预留                                                      |
| 7-0  | UT_BUF   | rw | 0x0     | -  | uart的收发数据寄存器<br>写UTx_BUF可启动一次发送；<br>读UTx_BUF可获得已接收到的数据。 |

### 20.2.8 UTx\_ERR\_CNT: uart x error byte counter register

| Bit | Name | RW | Default | RV | Description |
|-----|------|----|---------|----|-------------|
|-----|------|----|---------|----|-------------|



| Bit  | Name        | RW | Default | RV | Description                                                                                                                                                  |
|------|-------------|----|---------|----|--------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 31-0 | UTx_ERR_CNT | r  | 0x0     | -  | <p>校验错误数量计数，当打开CHECK_EN后，当第一次校验出错时，该寄存器会记录当前byte对应的数量。</p> <p>【注意】：只记录第一次出错的数量，清除CHK_PND会重新记录。</p> <p>若清除CHK_PND后，没有出现新的校验错误数据，则该寄存器会一直保持上一次记录的错误数量计数数据。</p> |

珠海市杰理科技股份有限公司

## 21 AUDIO LINK

### 21.1 概述

1. Audio Link接口(以下简称ALNK)是一个通用的双声道音频接口, 用于连接片外的DAC或ADC, 连接信号有MCLK, SCLK, LRCK, DATA。支持IIS/左对齐/右对齐/DSP0/DSP1共5种模式, 原生支持16/24bit数据位宽, 对18/20/32bit位宽的设备可提供兼容支持, 具备2条独立通道。ALNK通过DMA的方式与片内系统进行数据连接, 不论输入或输出, 每条通道占用buffer的大小可由软件配置。
2. MCLK是音频接口的主时钟, Delta-Sigma类型的DAC/ADC都是需要此信号的。MCLK可由ALNK提供给DAC/ADC, 也可由DAC/ADC提供给ALNK。
3. ALNK可配置为主机模式或从机模式。主机模式是指SCLK和LRCK由本模块提供, 此时需从外部提供相应采样率参考时钟。有3种方式可供选择:
  1. 挂接22.5792MHz晶振以支持44.1KHz组别的采样率。
  2. 挂接24.576MHz晶振以支持48/32KHz组别的采样率。
5. 从机模式是指SCLK和LRCK由外部DAC/ADC提供, 此时采样率由外部提供的LRCK确定, 因此芯片不需外接晶振来提供采样率参考时钟。

### 21.2 特殊注意点

ALNK具备2条独立的通道, 可相互独立地工作, 每条通道都可独立配置为输入或输出, 也可配置为不同的连接模式。需注意的是它们共用了MCLK/SCLK/LRCK信号, 因此使用上有一定的限制。根据LRCK信号的不同, 将IIS/左对齐/右对齐定义为基本模式, DSP0/DSP1定义为扩展模式。

1. 4条通道只能同时工作于基本模式或扩展模式。不可一些通道工作于基本模式, 另一些通道工作于扩展模式。
2. 基本模式下每条通道都只支持双声道立体声。扩展模式下每条通道均可支持单声道或立体声, 这由硬件自动适应, 当每帧的SCLK时钟个数大于等于立体声所需的时钟个数时, 该通道工作于立体声状态, 否则工作于单声道状态(只有左声道)。

### 21.3 ALNK支持的采样率配置

表格 1-1 ALNK支持的采样率设置和MCKD关系的列表

| SR(KHz) | 64fs | 128fs | 192fs        | 256fs         | 384fs        | 512fs         | 768fs         |
|---------|------|-------|--------------|---------------|--------------|---------------|---------------|
| 8       |      |       |              |               | 3.072M<br>可用 |               | 6.144M 推荐     |
| 11.025  |      |       |              | 2.8224M<br>可用 |              | 5.6448M<br>推荐 |               |
| 12      |      |       |              | 3.072M 可用     |              | 6.144M 推荐     |               |
| 16      |      |       | 3.072M<br>可用 |               | 6.144M<br>可用 |               | 12.288M<br>推荐 |

| SR(KHz) | 64fs           | 128fs          | 192fs         | 256fs          | 384fs         | 512fs          | 768fs         |
|---------|----------------|----------------|---------------|----------------|---------------|----------------|---------------|
| 22.05   |                | 2.8224M<br>可用  |               | 5.6448M<br>可用  |               | 11.2896M<br>推荐 |               |
| 24      |                | 3.072M 可<br>用  |               | 6.144M 可<br>用  |               | 12.288M<br>推荐  |               |
| 32      |                |                | 6.144M<br>可用  |                | 12.288M<br>可用 |                | 24.576M<br>推荐 |
| 44.1    |                | 5.6448M<br>可用  |               | 11.2896M<br>可用 |               | 22.5792M<br>推荐 |               |
| 48      |                | 6.144M 可<br>用  |               | 12.288M<br>可用  |               | 24.576M<br>推荐  |               |
| 64      |                |                | 12.288M<br>可用 |                | 24.576M<br>推荐 |                | 49.152M<br>可用 |
| 88.2    | 5.6448M<br>可用  | 11.2896M<br>可用 |               | 22.5792M<br>推荐 |               | 45.1584M<br>可用 |               |
| 96      | 6.144M 可<br>用  | 12.288M<br>可用  |               | 24.576M<br>推荐  |               | 49.152M<br>可用  |               |
| 128     |                |                | 24.576M<br>推荐 |                | 49.152M<br>可用 |                |               |
| 176.4   | 11.2896M<br>可用 | 22.5792M<br>推荐 |               | 45.1584M<br>可用 |               |                |               |
| 192     | 12.288M<br>可用  | 24.576M<br>推荐  |               | 49.152M<br>可用  |               |                |               |

## 21.4 数字模块控制寄存器

### 21.4.1 ALNK\_CON0: control register 0

| Bit | Name  | RW | Default | RV | Description                                                               |
|-----|-------|----|---------|----|---------------------------------------------------------------------------|
| 15  | FLAG3 | r  | 0       | x  | 通道x dma buffer flag<br>0: 当前正在使用buf0, buf1可被读写<br>1: 当前正在使用buf1, buf0可被读写 |
| 14  | FLAG2 | r  | 0       | x  |                                                                           |
| 13  | FLAG1 | r  | 0       | x  |                                                                           |
| 12  | FLAG0 | r  | 0       | x  |                                                                           |

| Bit | Name     | RW | Default | RV | Description                                                                |
|-----|----------|----|---------|----|----------------------------------------------------------------------------|
| 11  | ALNKE    | rw | 0       | 1  | ALNK模块使能<br>0: 模块关闭<br>1: 模块打开                                             |
| 10  | SCKINV   | rw | 0       | 0  | SCLK边沿选择<br>0: SCLK的下降沿更新数据, 上升沿采样数据<br>1: SCLK的上升沿更新数据, 下降沿采样数据           |
| 9   | F32E     | rw | 0       | x  | 主机模式下, 每帧数据的SCLK个数<br>0: 64 SCLKs per-frame<br>1: 32 SCLKs per-frame       |
| 8   | MOE      | rw | 0       | 0  | MCLK输出时钟使能<br>0: 不输出MCLK至对应IO端口<br>1: 输出MCLK至对应IO端口                        |
| 7   | SOE      | rw | 0       | 0  | SCLK/LRCK时钟输出使能<br>0: 不输出SCLK/LRCK至对应IO端口<br>1: 输出SCLK/LRCK至对应IO端口         |
| 6   | DSPEN    | rw | 0       | x  | ALNK模块工作模式选择<br>0: ALNK工作于基本模式 (IIS/左对齐/右对齐)<br>1: ALNK工作于扩展模式 (DSP0/DSP1) |
| 5-0 | RESERVED | -  | -       | -  | 预留                                                                         |

#### 21.4.2 ALNK\_CON1: control register 1

| Bit   | Name  | RW | Default | RV | Description                                                                                                                                                                                                                             |
|-------|-------|----|---------|----|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15    | T3DIR | rw | 0       | x  | TxDIR: 通道x方向设置<br>0: 发送 1: 接收<br>TxLEN: 通道x数据位宽设置 0: 16bit 1: 24bit<br>TxMOD: 通道x工作模式设置, 与DSPEN一起决定该通道的工作模式<br>DSPEN TxMOD<br>x 0 不使用通道x<br>0 1 IIS (数据延后1bit)<br>0 2 左对齐<br>0 3 右对齐<br>1 1 DSP0 (数据延后1bit)<br>1 2 DSP1 others 预留, 不可设置 |
| 14    | T3LEN | rw | 0       | x  |                                                                                                                                                                                                                                         |
| 13-12 | T3MOD | rw | 0x0     | x  |                                                                                                                                                                                                                                         |
| 11    | T2DIR | rw | 0       | x  |                                                                                                                                                                                                                                         |

| Bit | Name  | RW | Default | RV | Description |
|-----|-------|----|---------|----|-------------|
| 10  | T2LEN | rw | 0       | x  |             |
| 9-8 | T2MOD | rw | 0x0     | x  |             |
| 7   | T1DIR | rw | 0       | x  |             |
| 6   | T1LEN | rw | 0       | x  |             |
| 5-4 | T1MOD | rw | 0x0     | x  |             |
| 3   | T0DIR | rw | 0       | x  |             |
| 2   | T0LEN | rw | 0       | x  |             |
| 1-0 | T0MOD | rw | 0x0     | x  |             |

#### 21.4.3 ALNK\_CON2: control register 2

| Bit  | Name     | RW | Default | RV | Description                                |
|------|----------|----|---------|----|--------------------------------------------|
| 15-8 | RESERVED | -  | -       | -  | 预留                                         |
| 7    | PND3     | r  | 0       | x  | 通道x Pending, 当dma buf0或buf1被使用完毕后, 此位被硬件置1 |
| 6    | PND2     | r  | 0       | x  |                                            |
| 5    | PND1     | r  | 0       | x  |                                            |
| 4    | PND0     | r  | 0       | x  |                                            |
| 3    | CPND3    | w  | -       | x  | 写1清除通道x Pending, 写0无效                      |
| 2    | CPND2    | w  | -       | x  |                                            |
| 1    | CPND1    | w  | -       | x  |                                            |
| 0    | CPND0    | w  | -       | x  |                                            |

#### 21.4.4 ALNK\_CON3: control register 3

| Bit | Name | RW | Default | RV | Description |
|-----|------|----|---------|----|-------------|
|-----|------|----|---------|----|-------------|

| Bit | Name  | RW | Default | RV  | Description                                                                                                                                                                                                                                                      |
|-----|-------|----|---------|-----|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 7-5 | LRDIV | rw | 0x0     | x   | ALNK采样率设置<br>000: 从外部输入LRCK（即从机模式）<br>001: LRCK为MCKD的1/64, 即64fs<br>010: LRCK为MCKD的1/128, 即128fs<br>011: LRCK为MCKD的1/192, 即192fs<br>100: LRCK为MCKD的1/256, 即256fs<br>101: LRCK为MCKD的1/384, 即384fs<br>110: LRCK为MCKD的1/512, 即512fs<br>111: LRCK为MCKD的1/768, 即768fs |
| 4-2 | MDIV  | rw | 0x0     | x   | MCLK前置分频器设置<br>000: MCKD为MCLK的1分频<br>001: MCKD为MCLK的2分频<br>010: MCKD为MCLK的4分频<br>011: MCKD为MCLK的8分频<br>1xx: MCKD为MCLK的16分频                                                                                                                                       |
| 1-0 | MSRC  | rw | 0x0     | 0x3 | 设置MCLK的来源<br>00: 从外部输入MCLK<br>01: 系统时钟<br>10: OSC CLK<br>11: PLL ALNK CLK                                                                                                                                                                                        |

#### 21.4.5 ALNK\_ADRX: alnk dma start address register

ALNK DMA操作起始地址寄存器，在使用ALNK之前，必须由软件初始化对齐至4Byte。4个通道的ALNK DMA操作起始地址可以独立设置，其中，x取0-3。

#### 21.4.6 ALNK\_LEN: alnk dma sample length register

通道0-3 DMA数据长度设置寄存器。不同缓存模式下的含义不同。

##### 1. 乒乓缓存模式

ALNK DMA样点长度寄存器，在使用ALNK之前，必须由软件初始化为2的倍数，允许写入 值为2-32768，超出此范围的设置值可能导致不可预料的错误。

例如，当ALNK\_LEN设置为100时，则ALNK每条通道每次DMA过程消耗100个样点。此时每通道DMA buffer占用的空间为：

16bit data:  $100 * 2(\text{CH}) * 2(\text{Byte}) * 2(\text{dual buffer}) = 800 \text{ Byte}$   
24bit data:  $50 * 2(\text{CH}) * 4(\text{Byte}) * 2(\text{dual buffer}) = 800 \text{ Byte}$

【注意】：2(CH)代表左、右声道。

## 21.5 数据通路

### 21.5.1 乒乓缓存

ALNK的数据都是通过DMA的方式与片内系统连接的，使用了dual-buffer（乒乓缓冲）的方式，每条通道的buf0/buf1容纳样点数由ALNK\_LEN寄存器指定，总buffer需求为：

16bit data:  $ALNK\_LEN * 2(CH) * 2(Byte) * 2(dual\ buffer) = ALNK\_LEN * 8\ Byte$   
24bit data:  $ALNK\_LEN / 2 * 2(CH) * 4(Byte) * 2(dual\ buffer) = ALNK\_LEN * 8\ Byte$

当四条通道一起使用时，共需要4倍大小的buffer。

特别的：

1. 上述的ALNK\_LEN在此处只能理解为16bit数据宽度下的样点数，当数宽度为24bit时，样点数为ALNK\_LEN/2；

2. 在单声道状态下，只有左声道会被使用到，右声道将被停用。

16bit数据宽度时，数据组织如图1-1所示。

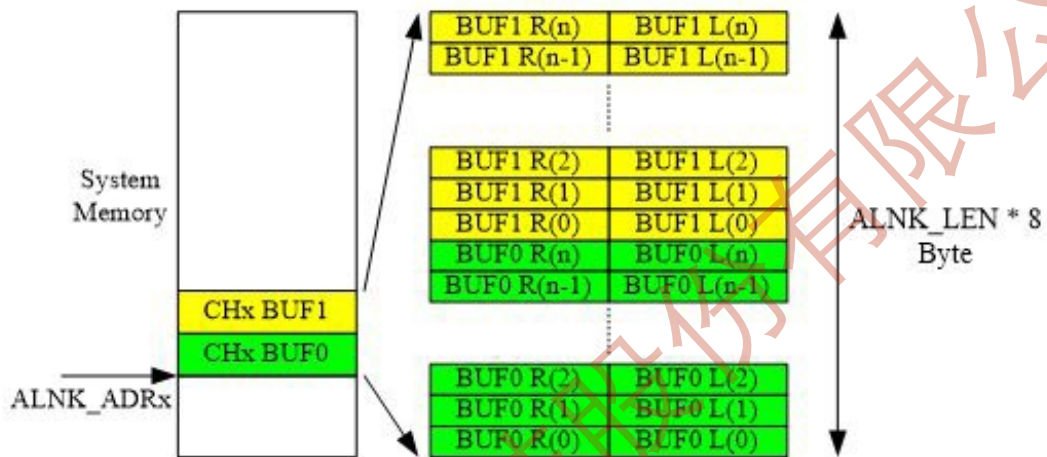


图 1-1 16bit数据宽度乒乓缓冲结构示意图

24bit数据宽度时，数据组织如图1-2所示。

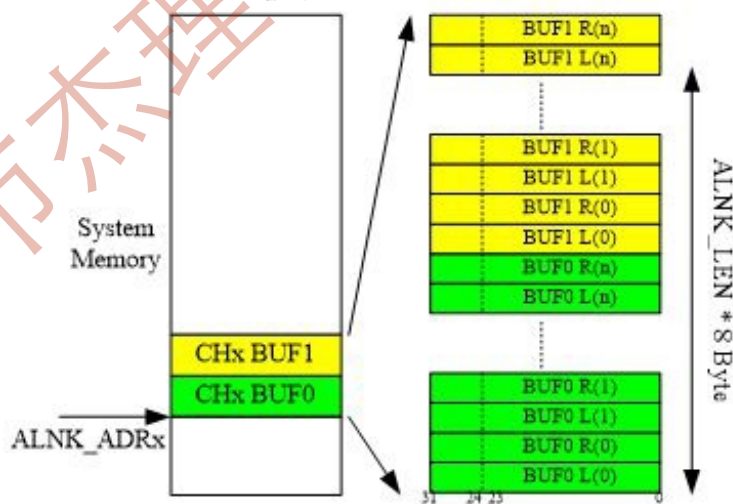


图 1-2 24bit数据宽度乒乓缓冲结构示意图